Manual for Security Engineers

A Reference Guide:

- How to read this book

Part I — Foundations

1. The identity problem
2. Cryptographic primitives
3. Trust foundations & token binding

Interlude — The complete picture

Part II — The JOSE Layer

4. JWT anatomy
5. JWS — signing
6. JWE — encryption
7. JWK & JWKS

Part III — Delegated Authorization: OAuth 2.0

8. Roles & terminology
9. Grants
10. PKCE — Proof Key for Code Exchange
11. Token endpoint, introspection, revocation
12. Scopes, audience, and resource indicators

Part IV — OAuth 2.1 & Security BCP

13. What OAuth 2.1 consolidates
14. Browser-based apps
15. Native apps

Part V — OpenID Connect

16. OIDC vs OAuth
17. ID Token claims
18. UserInfo endpoint
19. Discovery & Dynamic Client Registration
20. Response types & flows
21. Logout

Part VI — Social & Federated Identity

22. The federation model
23. Account linking
24. Just-in-time provisioning
25. Provider quirks

Part VII — User Lifecycle

26. Registration
27. Email verification
28. Password handling
29. Login
30. Recovery
31. Profile, consent, and audit
32. Account closure

Part VIII — Multi-Factor & Passwordless

33. MFA factor classes
34. TOTP & HOTP
35. WebAuthn / FIDO2
36. Passkeys
37. Magic links & email OTP
38. Step-up authentication

Part IX — AAAF: Authentication, Authorization, Accounting

39. The acronym, expanded
40. Authentication architecture
41. Authorization models
42. PDP / PEP / PIP / PAP
43. Scope vs role vs permission
44. Accounting / audit
45. Service-to-service authorization

Part X — Trust Distribution & Token Lifecycle

46. JWKS endpoint design
47. Key rotation strategy
48. Token validation algorithm
49. Token revocation

Part XI — Sender-Constrained & Strong Auth

50. DPoP — Demonstrating Proof-of-Possession
51. Mutual TLS (mTLS)
52. FAPI 2.0 profile
53. PAR & JAR

Part XII — Future-Looking Protocols

54. GNAP — Grant Negotiation and Authorization Protocol
55. Verifiable Credentials & Self-Sovereign Identity
56. CAEP & SSF — out-of-band session signals

Part XIII — Operational Security

57. The OAuth threat catalog
58. Hardened defaults
59. Secret & key custody
60. Rate limiting & abuse
61. Logging, monitoring, IR

Part XIV — Putting it together: the OneAuth reference architecture

62. Layer map
63. Build order
64. Closing — design tenets

Appendices

Appendix A — RFC index
Appendix B — Claim & header registry
Appendix C — Cipher suite recommendations (2026)
Appendix D — Glossary

A Reference Guide:

Identity, trust, and access — the layered stack. Conceptual reference, standards-grounded, not codebase-specific.

How to read this book

This document is a **standalone, conceptual reference** for the layers that make up an identity, trust, and access stack — JOSE, OAuth, OIDC, federated identity, user lifecycle, MFA, AAAF, key distribution, sender-constrained tokens, and what comes next. It is self-contained. It teaches you what each layer is *for*, what standard defines it, what an attacker tries against it, and what a correct implementation looks like.

It is written for the engineer who has to hold the whole stack in their head — the person who must know why PKCE exists *and* why JWE differs from JWS *and* why federated `sub` is provider-scoped *and* why DPoP is the SPA-friendly alternative to mTLS *and* how AAAF responsibilities split across a microservice mesh. If you only need to ship a feature, the code-&-test is faster. If you need to make a correct security decision, this is the document to read.

Audience

- Security engineers reviewing the architecture.
- Implementers extending an AS into new layers (MFA, JWE, GNAP).
- Auditors and reviewers checking the design against current best practice.
- Senior backend engineers building services that consume tokens from an AS.

Prerequisites

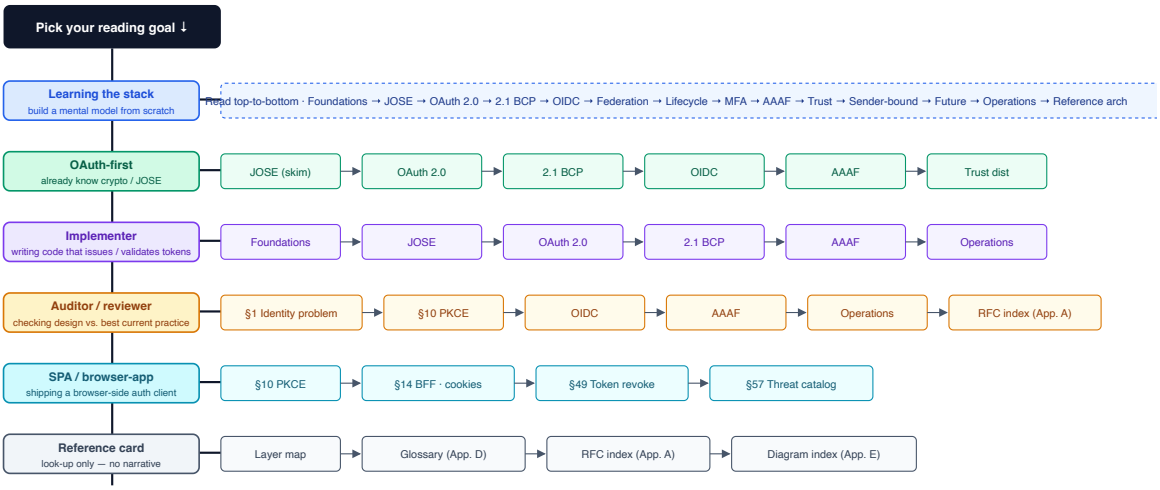
You should be comfortable with HTTP, TLS, JSON, public-key cryptography concepts (signing, verifying, key pairs), and RESTful APIs. You do **not** need prior OAuth/OIDC experience — Parts I–III build it up.

Reading paths

The book reads top-to-bottom. Most readers do not need every chapter; pick the branch that matches your problem and follow the chapters left-to-right.

Reading paths — branch by goal, then read chapters left-to-right

Six branches off the same root. Pick one based on what you are trying to do.



Forward references like "(§14)" point to the chapter where a topic is treated in depth — feel free to follow them.

Layer map

Group shading: foundations · protocols · users · operations & trust · ops & reference.

Glossary (front-loaded)

The full glossary lives in **Appendix D**. The following acronyms are used so often that you should know them before you start.

Term	Expansion	One-line meaning
AS	Authorization Server	The service that issues tokens.
RS	Resource Server	The API that consumes tokens.
RP	Relying Party	An OIDC client; the app that *relies on* the IdP for identity.
OP	OpenID Provider	An OIDC AS; the service the RP relies on.

Term	Expansion	One-line meaning
IdP	Identity Provider	Generic term for the entity that authenticates users (synonymous with OP in OIDC contexts).
RO	Resource Owner	The end user, in OAuth terminology.
AAAF	Authentication, Authorization, Accounting (Framework)	The three security responsibilities, separated.
JOSE	JSON Object Signing & Encryption	The IETF working group output: JWT/JWS/JWE/JWK/JWA.
`kid`	Key ID	A JOSE header that pins which key signed/encrypted a token.
`sub`	Subject	The user's stable identifier in a token.
`aud`	Audience	The intended recipient of a token.
`iss`	Issuer	The AS/OP that minted a token.
ACR	Authentication Context Class Reference	"How was the user authenticated?"
AMR	Authentication Methods References	"Which methods were used?" (e.g. `pwd`, `otp`, `hwk`).
PDP / PEP	Policy Decision / Enforcement Point	Where authorization is decided vs. where it is enforced.

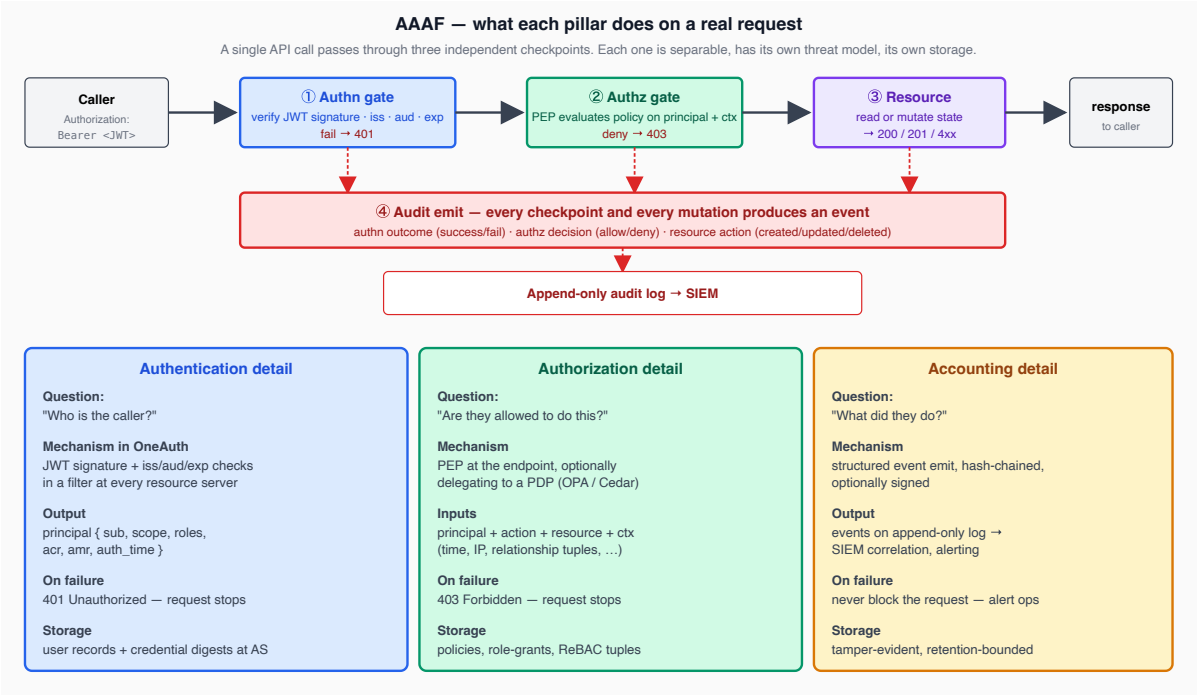
Part I — Foundations

1. The identity problem

Every request must answer three independent questions:

1. **Who is the caller?** (authentication)
2. **Are they allowed to do this?** (authorization)
3. **What did they do?** (accounting)

Conflating them is the most common architectural mistake — different threat models, different lifecycles, different storage.



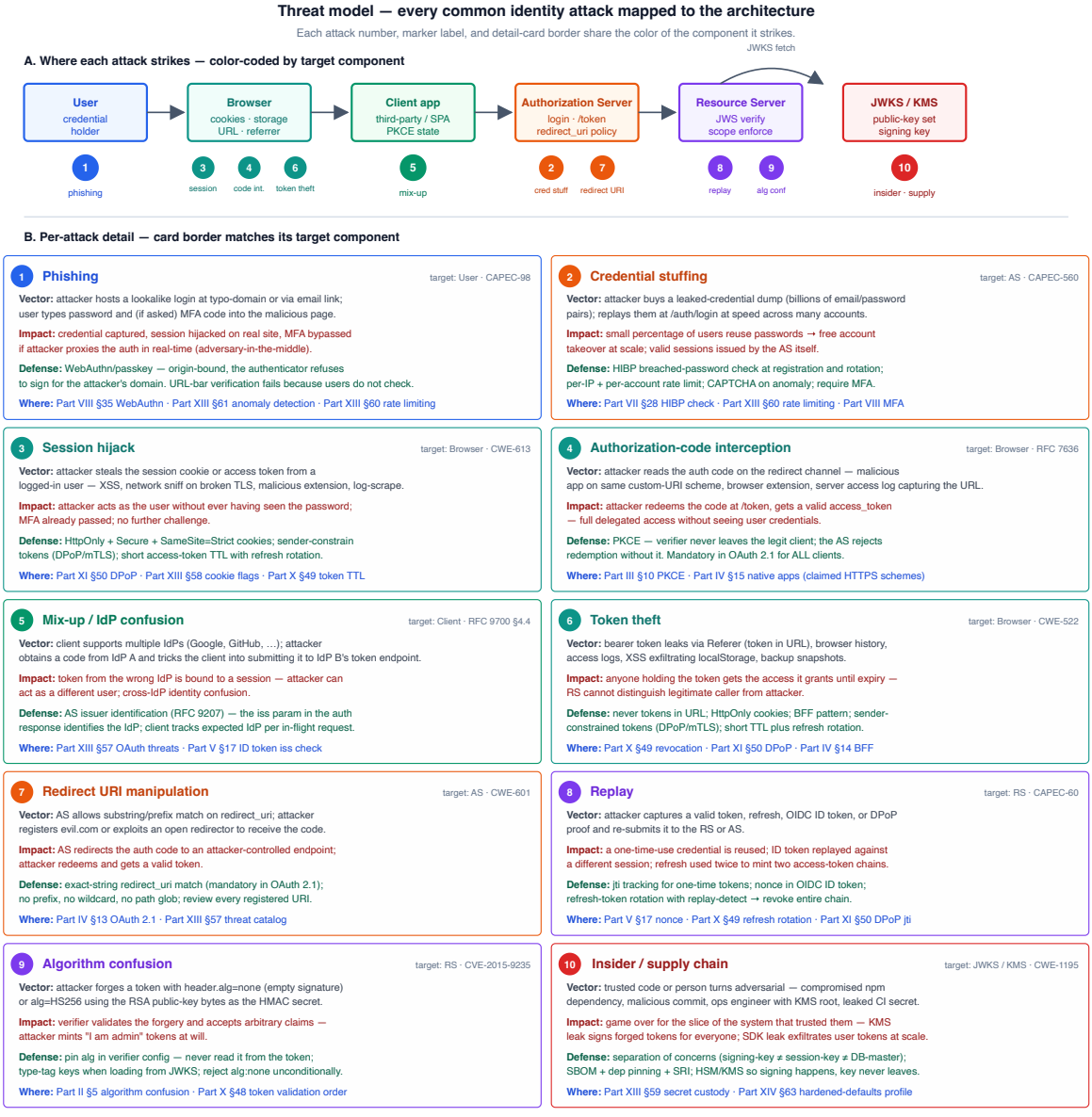
- **Authentication** is a truth claim. The caller asserts an identity; the system accepts or rejects. Output: a verified principal with strength claims (`sub`, `acr`, `amr`).
- **Authorization** is a policy decision. Given a principal, action, resource, and context, decide allow/deny. Separate the *decision* (PDP — OPA, Cedar, in-process) from the *enforcement* (PEP at the resource server).
- **Accounting** is a historical record. Every authn event, authz decision, and resource mutation lands in an immutable log. Done after-the-fact, it is never done well.

Why centralize identity

One login surface (single MFA enrollment, single password-reset path), consistent token semantics across services (every service speaks `sub` / `aud` / `scope`), single point of revocation (rotate the signing key — every service stops trusting old tokens), smaller hardening surface (one set of password digests, one set of OAuth client secrets).

The cost is operational coupling to the AS. The cure is offline verification through public-key cryptography (Part X) — JWKS-cached resource servers continue serving even when the AS is down.

Threat model overview



References

- NIST SP 800-63-3 *Digital Identity Guidelines* — the decomposition of identity assurance into IAL/AAL/FAL.
- NIST SP 800-207 *Zero Trust Architecture* — why every request gets re-evaluated, regardless of network position.
- IETF RFC 4949 *Internet Security Glossary, Version 2*.

2. Cryptographic primitives

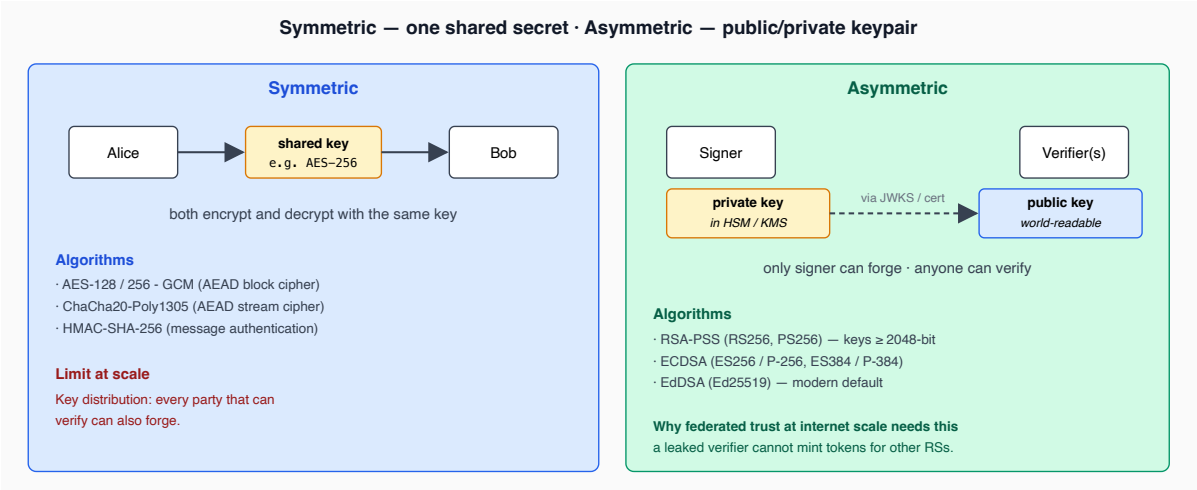
Identity-layer protocols are dense applications of a handful of primitives. You do not need to be a cryptographer to operate an identity stack, but you do need to know which primitive solves which problem so you can read a token, an error log, or a config file and immediately know what is being claimed.

Symmetric vs asymmetric

Symmetric primitives use one shared key (e.g., AES, HMAC, ChaCha20). Fast, small, but require key distribution: every party that can verify can also forge.

Asymmetric primitives use a keypair: one private (sign / decrypt), one public (verify / encrypt). Slower, larger keys and signatures, but the public key can be distributed openly. The asymmetric property is what makes federated trust at internet scale possible: the AS holds a private signing key, and any service in the world can fetch the public key and verify a token without ever talking to the AS.

A correctly built AS keeps the three properties separate: **token signing is always asymmetric** (RS256, ES256, or EdDSA) so any consumer can verify offline; **password storage is one-way** (Argon2id; see below); **service-to-service confidentiality** is delegated to TLS at the transport layer.



Hash functions

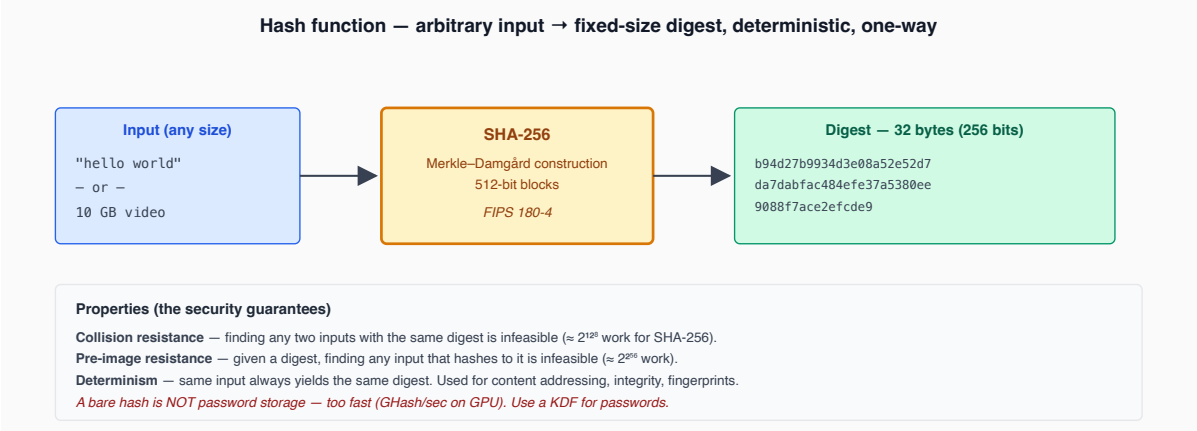
A cryptographic hash takes arbitrary input and returns a fixed-size, deterministic, irreversible digest. Three properties matter:

- **Collision resistance** — finding two inputs with the same digest is infeasible ($\approx 2^{128}$ work for SHA-256).
- **Pre-image resistance** — given a digest, finding an input that hashes to it is infeasible.

- **Second pre-image resistance** — given an input, finding a *different* input with the same digest is infeasible.

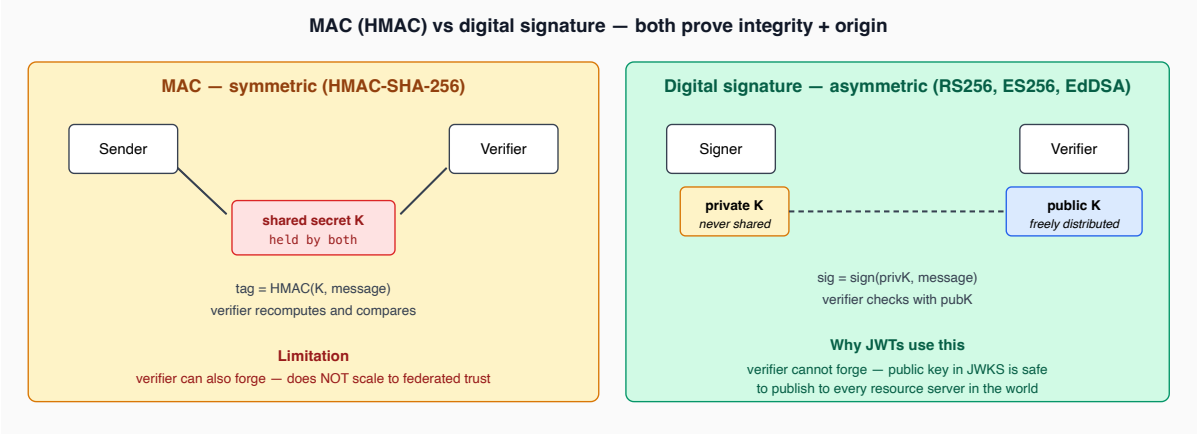
SHA-256 / SHA-512 throughout JOSE: PKCE `S256` method, JWS thumbprints, `at_hash`, JWK thumbprints per RFC 7638. MD5 and SHA-1 are broken for collision resistance — never use them.

A bare SHA-256 of a password is crackable at billions/sec on a GPU. Password storage requires a KDF (next).



MACs (Message Authentication Codes)

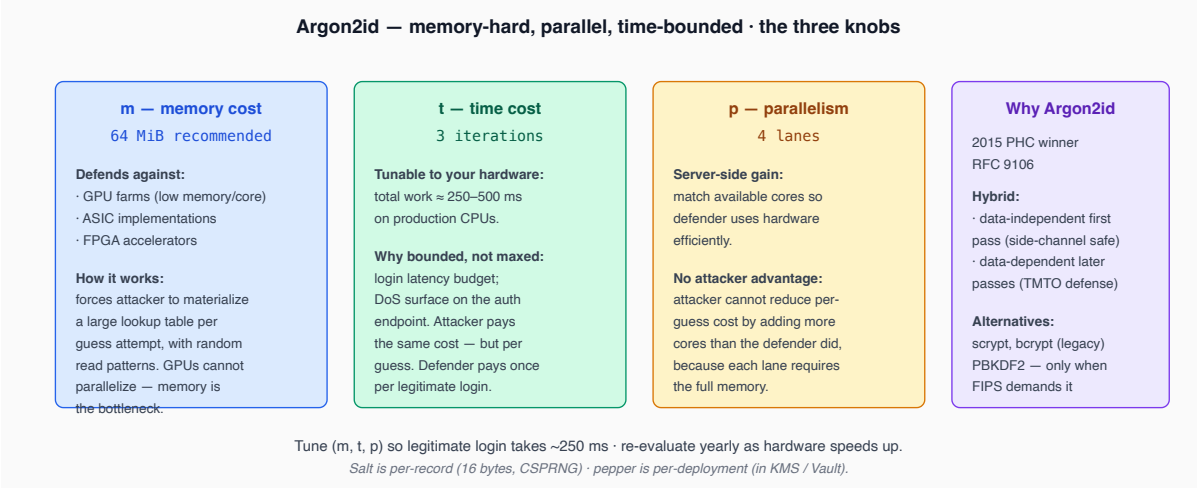
HMAC-SHA-256 is the standard MAC. JWS HS256 uses it but is unsuitable for federated AS deployments — verifier needs the secret (defeats federated trust) and is the seat of the algorithm-confusion attack (§5).



KDFs (Key Derivation Functions)

Use Argon2id for passwords (RFC 9106). Three tunable parameters: memory, time, parallelism. Acceptable fallbacks: scrypt, bcrypt cost ≥ 12; PBKDF2 only for FIPS.

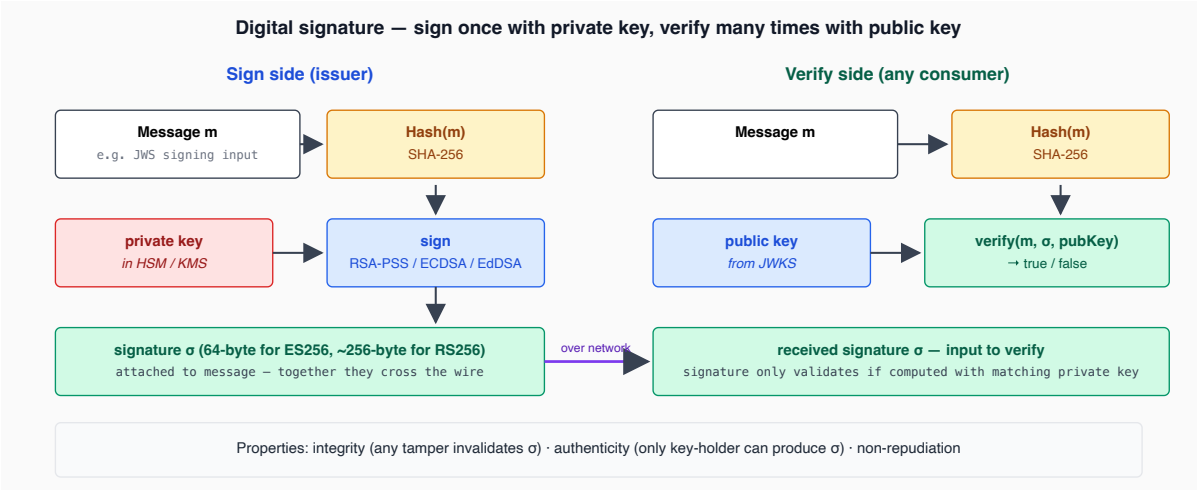
Salt is per-record (rainbow-table defense). Pepper is per-deployment in KMS (DB-only leak still uncrackable).



Digital signatures

A signature is the asymmetric analogue of a MAC: signer holds the private key, verifier holds the public key, only the signer can forge. Four families an AS typically picks from:

- **RSA-PKCS#1 v1.5** (RS256 / RS384 / RS512) — historical default. RSA keys ≥ 2048-bit. Large signatures, fast verify, slow sign. Common interop choice; the OneAuth repo example uses RS256.
- **RSASSA-PSS** (PS256 / PS384 / PS512) — RSA with randomized padding. More conservative against future attacks. Drop-in when consumers support it.
- **ECDSA** (ES256 over P-256, ES384 over P-384) — elliptic-curve. Smaller keys (256-bit) and signatures (~64 bytes), fast both ways. Use deterministic ECDSA (RFC 6979) to avoid nonce-reuse compromise.
- **EdDSA** (Ed25519) — modern default. Fastest, smallest, deterministic by design. Adopt once consumer tooling stabilizes.

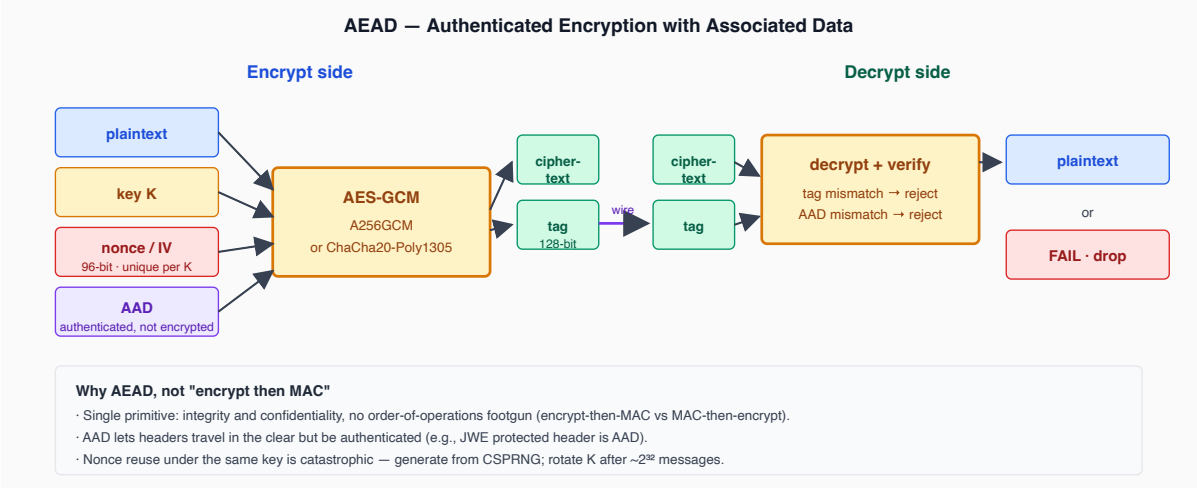


AEAD ciphers

For the JWE layer (§6) — confidentiality and integrity in one primitive. Two families:

- **AES-GCM** (A128GCM, A192GCM, A256GCM) — standard, hardware-accelerated on x86 (AES-NI) and ARM. The 96-bit IV must never repeat under the same key. Rotate the key after roughly 2^{32} messages.
- **ChaCha20-Poly1305** (RFC 7539) — software-friendly, constant-time on platforms without AES hardware. Registered in JOSE.

Nonce reuse under the same key is catastrophic — generate IVs from CSPRNG; track count for constructed nonces.

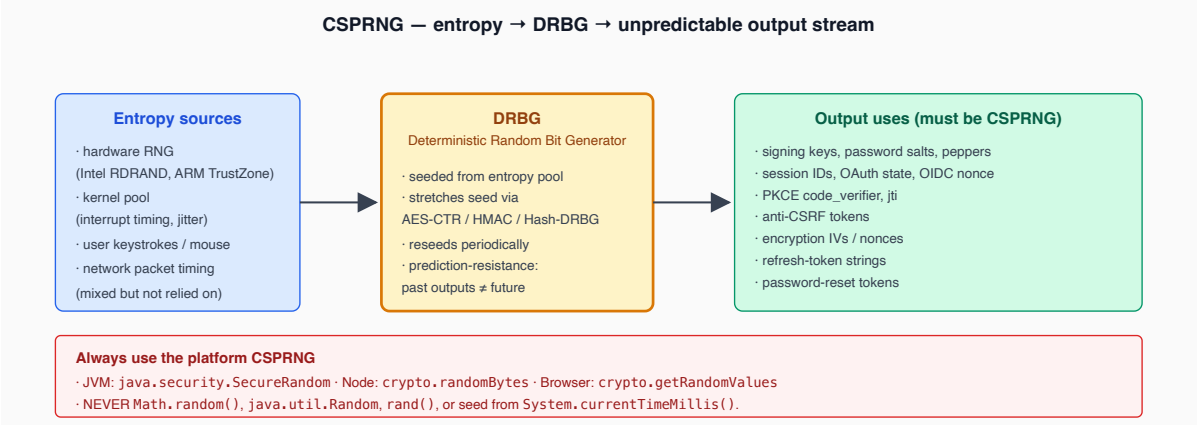


Random number generation

Almost every security failure that is not a primitive failure is a randomness failure. Always use the platform CSPRNG: `java.security.SecureRandom`, `crypto.randomBytes`, `crypto.getRandomValues`. Never `Math.random()`, `java.util.Random`, or seeds from `System.currentTimeMillis()`.

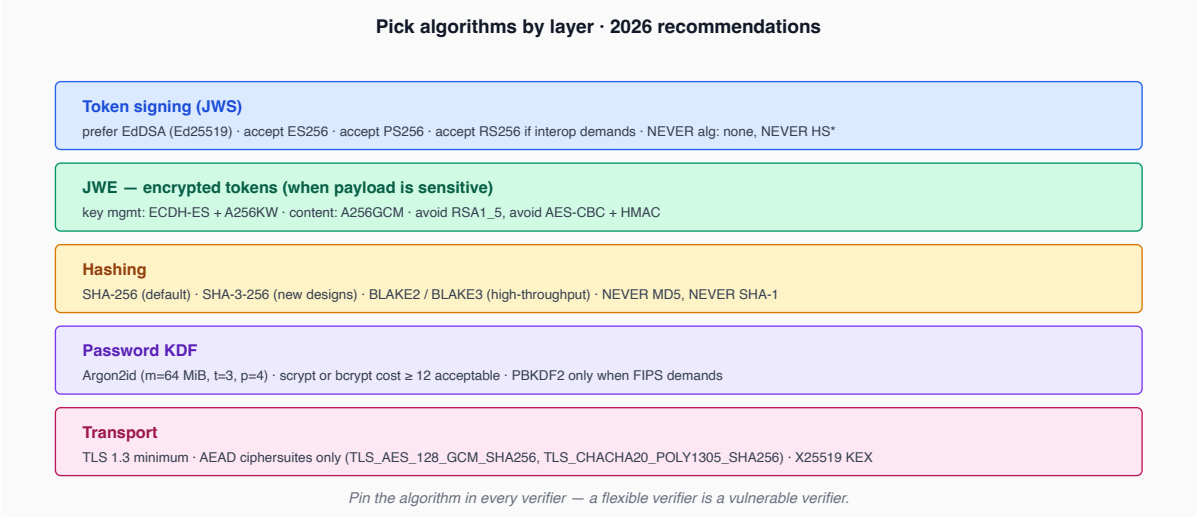
Required for: signing-key generation, password salts, session identifiers, OAuth `state`, OIDC `nonce`, PKCE `code_verifier`, `jti`, CSRF tokens, encryption IVs, refresh-token strings, password-reset tokens.

Plain randomness suffices for: UI animation delays, retry-backoff jitter, telemetry sampling.



Cipher-suite recommendations (2026)

Full table in Appendix C.



Threat model

Attack	Mitigation
Brute-force password guessing	Argon2id with high memory cost; rate limiting (Part XIII).
Rainbow-table cracking	Per-record salts (mandatory), per-deployment peppers (optional).
Algorithm downgrade	Pin algorithm in the verifier; never accept `alg: none`.
Weak randomness	Platform CSPRNG only; review every `Random` use in security-sensitive code.
Nonce reuse	Deterministic ECDSA (RFC 6979); careful AES-GCM IV management.

References

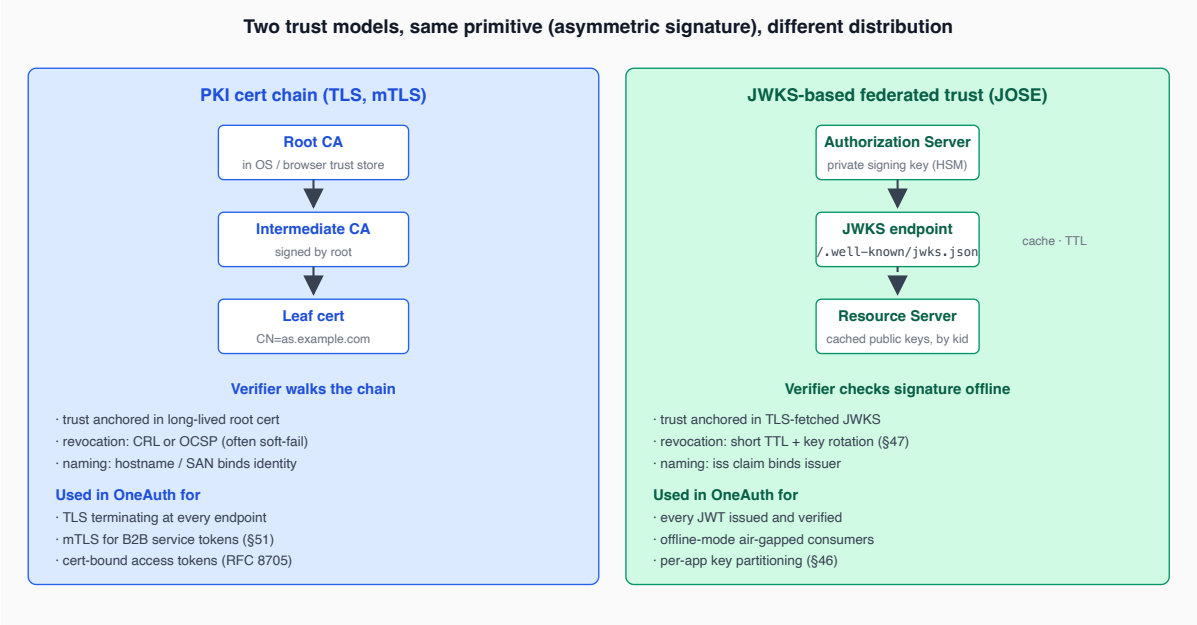
- IETF RFC 7515 *JSON Web Signature*.
- IETF RFC 7518 *JSON Web Algorithms*.
- IETF RFC 8032 *EdDSA*.
- IETF RFC 6979 *Deterministic ECDSA*.
- IETF RFC 9106 *Argon2*.
- NIST SP 800-90A *Random Bit Generators*.

3. Trust foundations & token binding

Identity systems run on **trust** — the recipient of a credential decides whether to accept it. Two trust models matter:

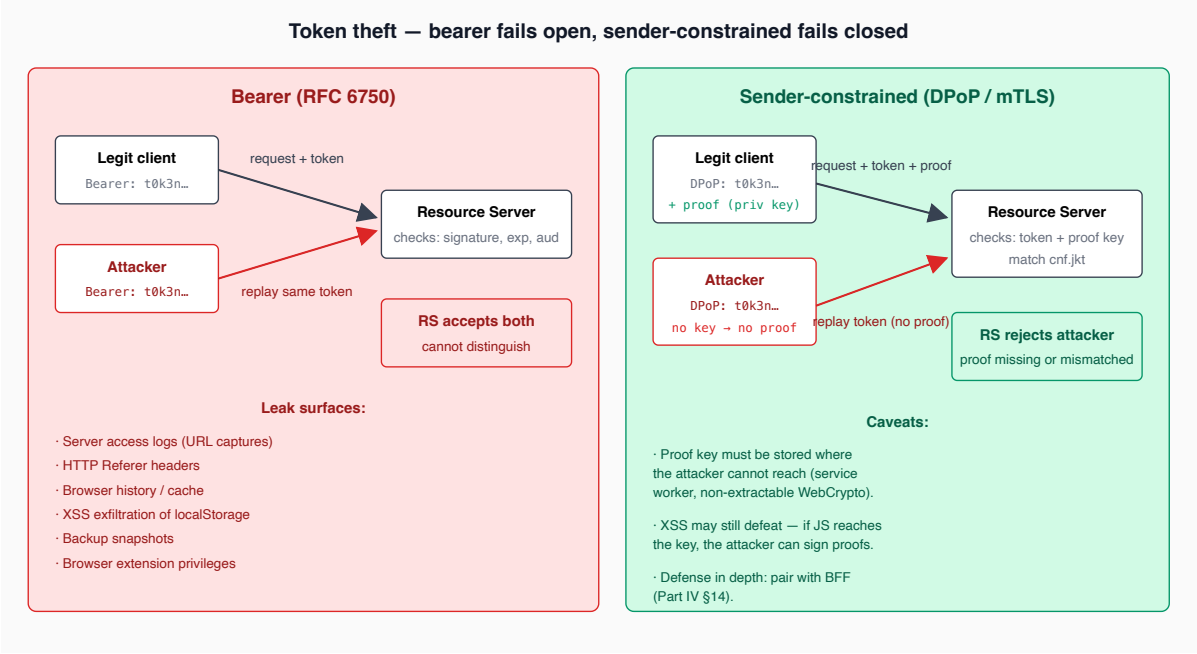
- **PKI / certificate chain** (used by TLS) — anchor in a root CA installed in the trust store; intermediate CAs sign leaf certificates; verifiers walk the chain to a known root.
- **Federated trust via JWKS** (used by OAuth/OIDC) — anchor in the AS's published public-key set at a well-known HTTPS URL; resource servers cache the JWKS and verify tokens offline.

Both are asymmetric: a private key signs, a public key verifies. The difference is how trust roots are distributed and how revocation is reasoned about. PKI relies on long-lived root certs and CRL/OCSP to revoke; JWKS relies on short-TTL caches and key rotation (Part X).



Bearer tokens

A **bearer token** grants access to whoever holds it — possession is sufficient. The recipient checks the token's signature, expiry, and audience, but not who is presenting it. Bearer tokens (RFC 6750) are the historical default of OAuth 2.0 and remain the most widely deployed token form.



Sender-constrained tokens

A **sender-constrained token** binds to a key the legitimate client controls — the resource server requires a proof of possession on every request. A stolen token is useless without the key. Three mechanisms in active use:

- **DPoP** (RFC 9449) — per-request proof JWT signed by a client-generated key; AS binds the access token to the public-key thumbprint via `cnf.jkt`. Implementable in pure JavaScript (Web Crypto). Full treatment in §50.
- **Mutual TLS** (RFC 8705) — client X.509 cert at TLS handshake; AS binds token to cert thumbprint. Heavyweight, common in B2B and FAPI. §51.
- **Token Binding** (RFC 8471) — TLS-layer key binding. Effectively dead (Chrome dropped support).

Why bearer is fragile

Once a bearer token leaves the legitimate client, anyone who has it can present it. Common leak surfaces:

- HTTP Referer headers (token in URL query string).
- Browser history and disk caches.
- Server access logs that capture full URLs.
- XSS exfiltration from a vulnerable SPA.
- Backup snapshots of localStorage / database / log files.
- Mobile-OS clipboard managers, browser-extension privileges.

Why we still ship bearer tokens

Bearer is the universal default — every JOSE library, reverse proxy, and SDK validates it without ceremony. For typical first-party / behind-TLS deployments, bearer is sufficient *if* TTLs are short, refresh tokens are rotated, transport is always TLS, and tokens never enter URLs. Roadmap promotes DPoP for SPAs and mTLS for B2B (Part XI).

Threat model

Attack	Bearer	Sender-constrained
Token theft from log / cache / backup	Game over	Useless without key
MITM with broken TLS	Replay accepted	Replay rejected (proof mismatch)
XSS exfil of localStorage	Game over	Game over if key co-located — DPoP needs an isolated key store

DPoP is not an XSS substitute. If JS can read the token, it can probably reach the proof key. The defense against XSS is strict CSP plus a backend-for-frontend so tokens never reach the browser at all (§14).

References

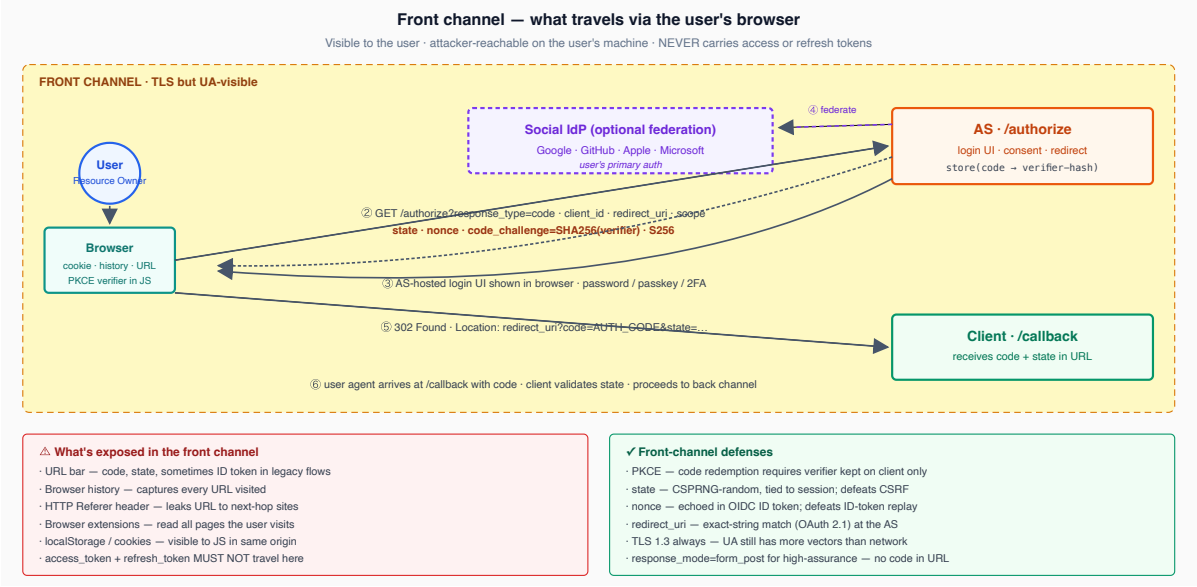
- IETF RFC 6750 *OAuth 2.0 Bearer Token Usage*.
- IETF RFC 9449 *DPoP*.
- IETF RFC 8705 *Mutual-TLS Client Authentication*.
- IETF RFC 9700 *OAuth 2.0 Security Best Current Practice* §2.1 (sender constraint).

Interlude — The complete picture

Before zooming into JOSE, OAuth, OIDC, and AAAF separately, look at how the pieces fit together. The system has **two distinct execution channels** — *front* and *back* — and each carries different credentials with different threat models. Splitting them is the first concept that has to land.

Front channel — via the user's browser

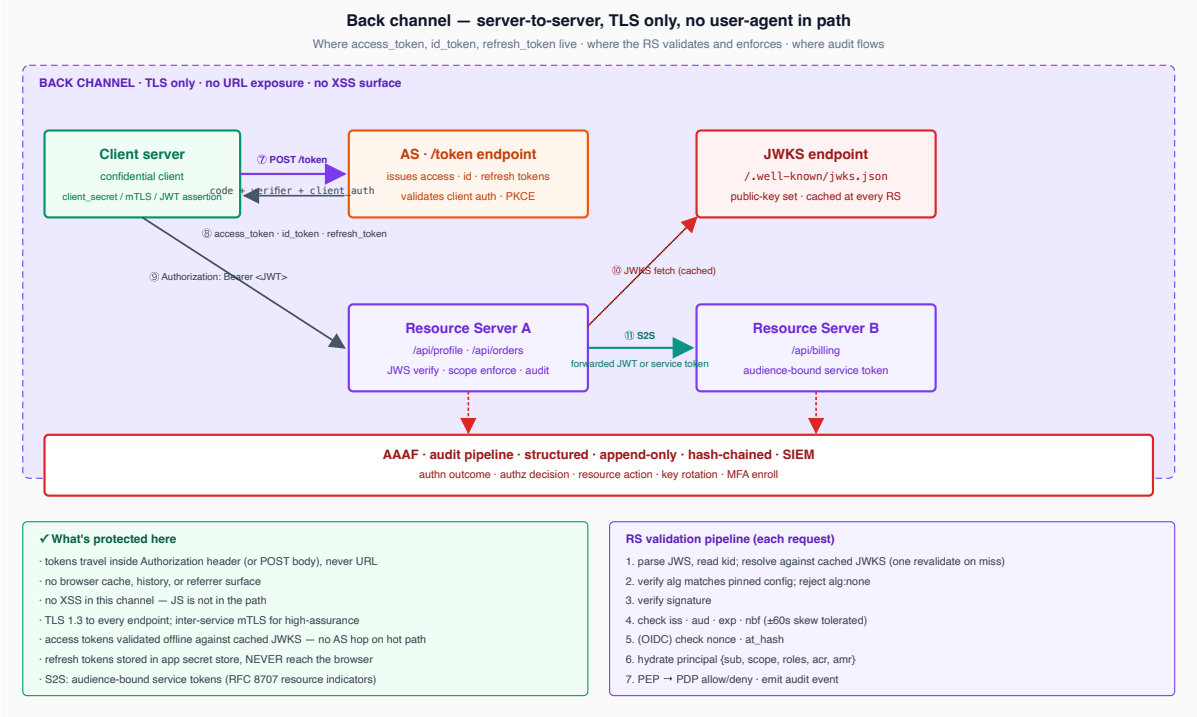
The browser, the user, the user's machine. Visible to extensions, to history, to the URL bar, to anyone watching the screen. Anything sent here is potentially seen by an attacker. **The front channel never carries access or refresh tokens** — only short-lived authorization codes (which are useless without the PKCE verifier, which never leaves the client).



The front channel ends when the user agent arrives at the client's `/callback` URL with a `code`. The client now hands off to the back channel.

Back channel — server-to-server

Direct between servers. TLS only. No user-agent, no URL exposure, no XSS surface. **All access, ID, and refresh tokens live here.** Resource servers verify tokens offline against the cached JWKS — they never call the AS on the hot path. Audit events flow out to the SIEM from every layer.



Three credentials, three audiences

Three different tokens are in flight at once. Conflating them is the most common misuse of OAuth and OIDC:

- **id_token** — answers *"who is the user"*. Consumed by the *client*. Lives only in the back channel between AS and client.
- **access_token** — answers *"what may be done on whose behalf"*. Consumed by the *resource server*. Travels in the `Authorization: Bearer` header.
- **refresh_token** — long-lived credential to mint new access tokens. Consumed by the *AS only*. NEVER sent to RSs, NEVER reaches the browser (in BFF deployments).

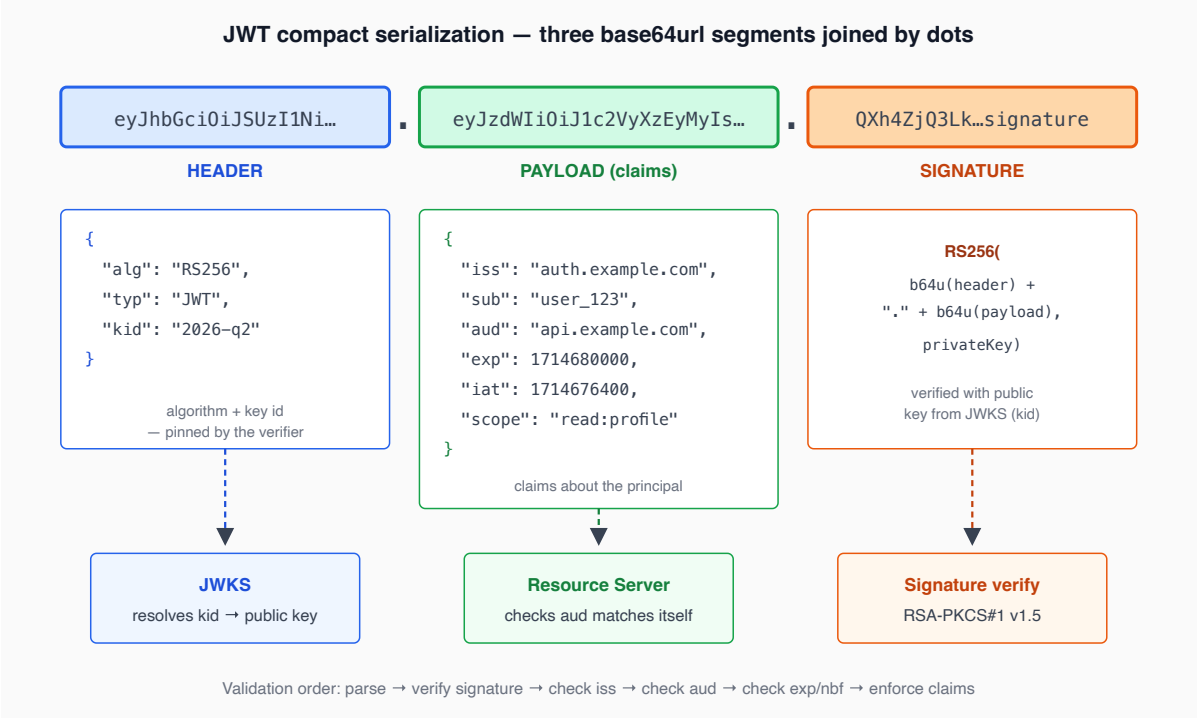
Part II — The JOSE Layer

JOSE — JSON Object Signing and Encryption — is the IETF working group whose output gives us JWT, JWS, JWE, JWK, JWKS, and JWA. Every credential a modern AS issues is a JOSE artifact. This part walks the family bottom-up so the rest of the book has firm footing.

4. JWT anatomy

A JWT (JSON Web Token, RFC 7519) is a *claims container*. It is not, by itself, signed or encrypted — JWT only specifies the claims and the JSON encoding. To get integrity (signing) you wrap it in a JWS. To get confidentiality you wrap it in a JWE. "JWT" almost always means "JWS-signed JWT", which is the *compact serialization* of three base64url-encoded parts joined by dots:

```
<base64url(header)>.<base64url(payload)>.<base64url(signature)>
```



Header

The header is a JSON object naming:

- `alg` — the signing algorithm (e.g., `RS256`, `ES256`, `EdDSA`). **Always pin** the expected `alg` in the verifier; never trust the header to dictate which algorithm to use (Part II §5, algorithm-confusion).
- `typ` — the type. Typically `JWT`. RFC 8725 (JWT Best Current Practices) recommends a more specific type like `at+jwt` (RFC 9068) for OAuth 2.0 access tokens.
- `kid` — Key ID. The `kid` is what lets the verifier pick the right public key out of a JWKS containing many. The rotation strategy in Part X §47 hinges on `kid`.
- `cty` — Content Type. Used when the payload is itself a JOSE artifact (nested JWT).

Payload (claims)

Payload claims are split into:

- **Registered claims** (RFC 7519 §4.1) — `iss`, `sub`, `aud`, `exp`, `nbf`, `iat`, `jti`. These have prescribed semantics. You should not redefine them.
- **Public claims** — registered in the IANA JWT Claims registry (e.g., `email`, `email_verified`, `name`, `roles`, `scope`).
- **Private claims** — application-specific, namespaced by URI to avoid collision (`https://oneauth.example.com/claims/tenant_id`).

Claim	Meaning	Notes
`iss`	Issuer	Must match what the verifier expects. URL form preferred (`https://auth.example.com`).
`sub`	Subject	Stable identifier of the principal *within the issuer*. Not necessarily a user ID — could be a service.
`aud`	Audience	Who the token is for. Verifier MUST check that it appears in `aud`. May be a string or array.
`exp`	Expiration	Unix seconds. Hard cutoff.
`nbf`	Not before	Unix seconds. Token invalid before this.
`iat`	Issued at	Unix seconds.
`jti`	JWT ID	Unique token identifier. Used for replay defenses, denylists.
`scope` / `scp`	Scopes	OAuth scopes granted (RFC 9068).
`cnf`	Confirmation	Sender-constraint binding (Part XI).
`acr` / `amr`	Auth context / methods	OIDC strength markers.
`nonce`	Nonce	OIDC ID-token replay defense.

Signature

The signature is computed over `base64url(header) + "." + base64url(payload)` using the algorithm in the header. RFC 7515 spells out the exact byte sequence — getting this wrong is a common source of "valid token rejected" bugs (forgetting that base64url, not base64, is required, or that the dot separator is part of the signed input).

Compact vs JSON serialization

JWS (and JWE) have two serializations:

- **Compact** — three (JWS) or five (JWE) base64url segments joined by dots. URL-safe. The default for tokens.
- **JSON** — a structured JSON object. Supports multiple signatures / recipients. Used in document-signing contexts.

Most production AS implementations (the OneAuth repo example included) use compact serialization throughout.

Threat model

Attack	Mitigation
Forged claim	Signature verification (Part II §5).
Stale token	`exp` enforcement plus short TTL plus refresh rotation (Part X).
Replay	`jti` tracking for one-time tokens; `nonce` for ID tokens (Part V §17).
Audience confusion	Resource server MUST check `aud` against its own identifier.

References

- IETF RFC 7519 *JSON Web Token*.
- IETF RFC 8725 *JWT Best Current Practices*.
- IETF RFC 9068 *JWT Profile for OAuth 2.0 Access Tokens*.

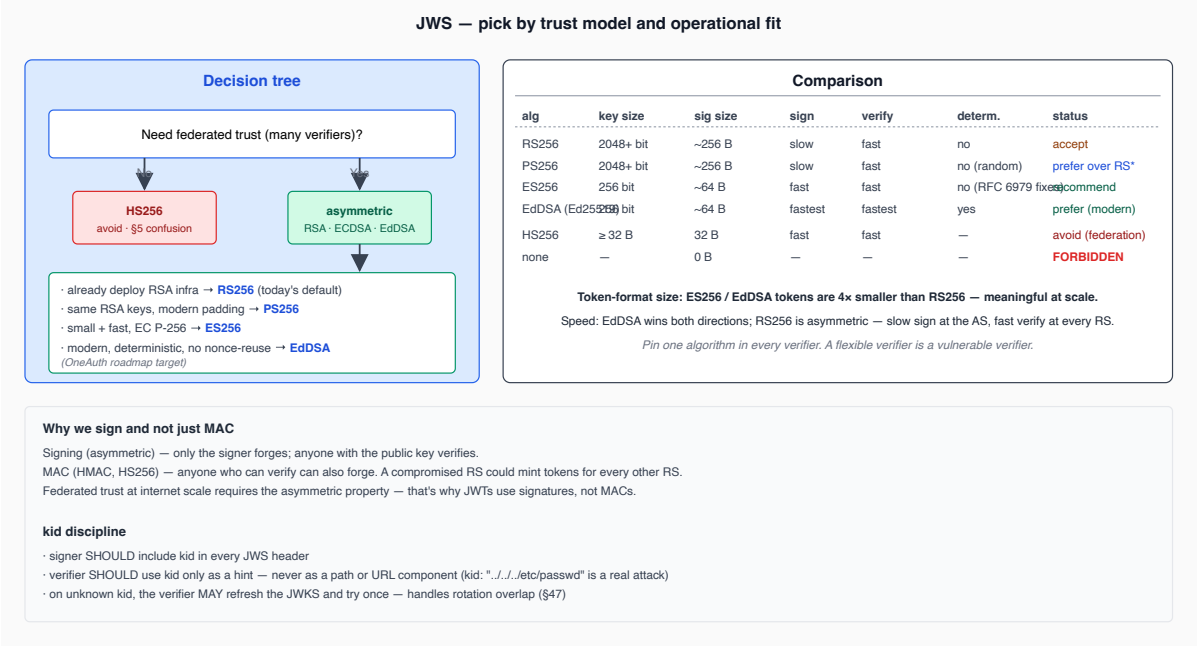
---

5. JWS — signing

JWS (RFC 7515) is the *integrity* layer. It binds a payload to a private key. A successful verification tells you two things: the payload is byte-for-byte what the signer signed, and the signer holds the key matching the public key you used to verify.

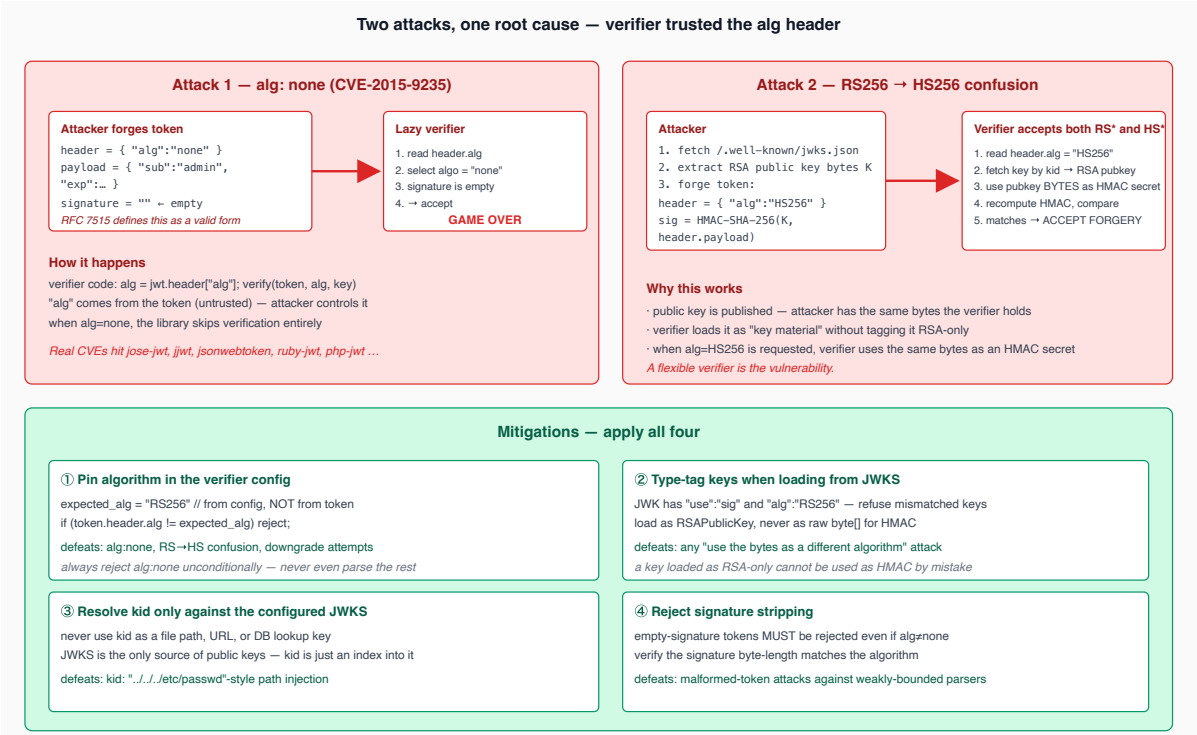
It does **not** tell you the payload is fresh, in-scope, intended for you, or that the signer is the issuer you expect. Those are claim-level checks, performed *after* signature verification. The validation order matters and is laid out in Part X §48.

Algorithm choice



Algorithm-confusion attacks

Two classic JWT attacks both come from one root cause — **trusting the `alg` field in the unverified header**. The fix in both cases is to pin the expected algorithm in the verifier's configuration and reject everything else.



`kid` discipline

`kid` is how the verifier resolves a public key from a multi-key JWKS. The rules:

- The signer SHOULD include `kid` in every JWS header.
- The verifier SHOULD use `kid` only as a *hint* — never trust it to locate keys outside the configured JWKS. (An attacker who can submit `kid: ../../../etc/passwd` against a poorly-coded verifier has just gained file read.)
- On unknown `kid`, the verifier MAY refresh the JWKS and try again exactly once. This handles the rotation-overlap case (Part X §47).

Why we sign and not just MAC

Signing (asymmetric) means anyone can verify, only the signer can forge. MAC (symmetric) means anyone who can verify can also forge. For federated trust at internet scale, signing is the only viable choice: every resource server can hold a public key, and a compromised resource server cannot mint tokens for other resource servers.

Threat model

Attack	Mitigation
`alg: none`	Hard-reject.
Algorithm confusion	Pin algorithm; tag keys with their algorithm.
Key substitution via `kid`	Resolve `kid` only against the configured JWKS.
Signature stripping	Tokens with empty signatures must be rejected even if `alg: none` is rejected.
Length-extension	Not applicable to RSA / ECDSA / EdDSA; relevant only to constructions like raw SHA-256 of `secret || message`.

References

- IETF RFC 7515 *JWS*.
- Tim McLean, *Critical vulnerabilities in JSON Web Token libraries* (2015) — the original `alg: none` write-up.
- Auth0, *Critical vulnerabilities in JSON Web Token libraries* (2015 follow-up on algorithm confusion).
- OWASP, *JSON Web Token Cheat Sheet*.

---

6. JWE — encryption

JWE (RFC 7516) is the confidentiality layer — only the recipient's private key can decrypt the payload. JWS-signed tokens are public-readable (anyone who can decode base64url

sees the claims), which is acceptable for `sub` / `email` / `roles` but unacceptable for medical, financial, or residency-bound data.

An advanced pattern: **nested tokens** — a JWS wrapped in a JWE. The outer JWE protects the payload from intermediaries; the inner JWS preserves issuer-binding once the recipient decrypts.

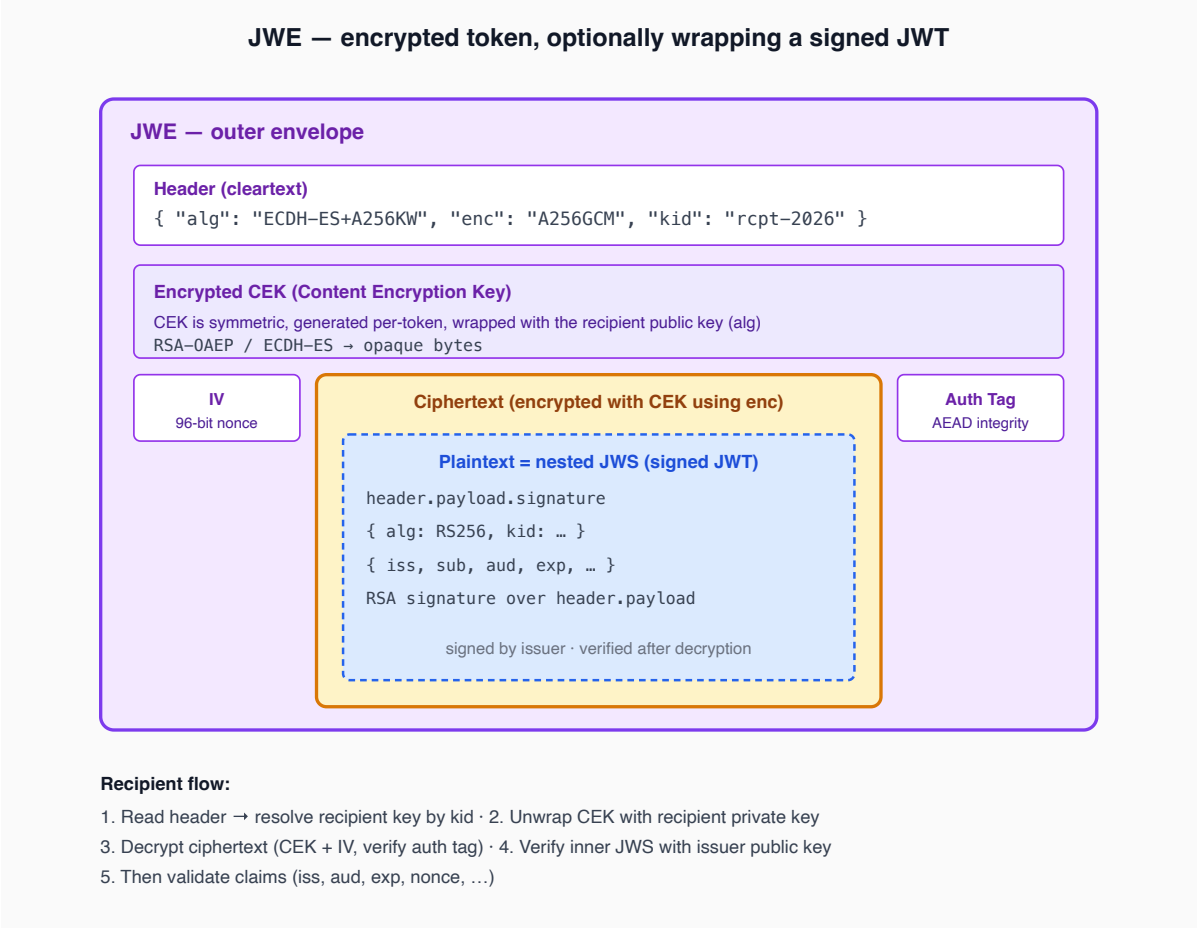
JWE structure

Compact form: `<header>.<encrypted_key>.<iv>.<ciphertext>.<auth_tag>`

The two-layer key construction is the heart of JWE:

- A fresh **Content Encryption Key (CEK)** is generated per token. The CEK encrypts the payload using an AEAD cipher (`enc` parameter, e.g., `A256GCM`).
- The CEK itself is wrapped (encrypted) for the recipient using their public key (`alg` parameter, e.g., `ECDH-ES` , `RSA-OAEP`).

This mirrors PGP and TLS — per-message symmetric content key, asymmetrically wrapped. Fast (AEAD on bulk payload), key-bounded (only the recipient can unwrap the CEK).



Algorithm choice

JWE has two algorithm parameters.

`alg` — key management:

`alg`	What it does	When to use
`ECDH-ES` (or `ECDH-ES+A256KW`)	Static-ephemeral ECDH with the recipient's EC keypair.	Modern default.
`RSA-OAEP-256`	Wraps CEK with the recipient's RSA public key.	Backwards-compat with RSA-only consumers.
`A256KW`	Symmetric key-wrap with a shared secret.	Closed systems; rare in federated identity.
`dir`	No CEK wrap; pre-shared symmetric key directly.	Trusted internal hops only.

`enc` — content encryption:

`enc`	Cipher	Notes
`A128GCM` / `A256GCM`	AES-GCM	Default. Hardware-accelerated.
`A128CBC-HS256` / `A256CBC-HS512`	AES-CBC + HMAC	Legacy; avoid for new designs.
`XC20P`	XChaCha20-Poly1305	Software-friendly; less universally supported.

Recommended defaults: `alg=ECDH-ES+A256KW`, `enc=A256GCM`.

When to use JWE

Use JWE when **any** of these is true:

1. The payload contains data intermediaries (proxies, gateways, logs, browsers) must not see.
2. Compliance requires "encryption at rest" *and* tokens transit through systems considered "rest" (caches, queues).
3. The token traverses a federation chain in which intermediate parties should not learn user attributes.
4. The OIDC profile mandates it (e.g. FAPI 2.0 certifications).

Do not use JWE merely "for security" if the payload is not sensitive — it adds operational cost (recipient key management, decrypt on every request) without proportionate benefit.

Anti-patterns

- **JWE without authenticating the recipient.** If anyone can fetch the recipient's public key and encrypt, the token is "private" from anyone *but* the AS. That is fine for the AS-to-RS path. It is not "authentication" — JWE alone says nothing about who created the token. Always nest a JWS inside (or sign-then-encrypt) when integrity-of-issuer matters.
- **Sign over ciphertext, not plaintext.** RFC 7515 + RFC 7516 nest correctly: sign the plaintext, then encrypt the JWS. Reversed nesting (encrypt-then-sign over the ciphertext) leaves the ciphertext unsigned and breaks integrity guarantees against malleable AEAD modes.
- **Reusing CEKs.** A fresh CEK per token is non-negotiable. Reusing CEKs is a catastrophic key-management failure.

Threat model

Attack	Mitigation
Eavesdropper between AS and RS	JWE protects payload (TLS protects everything else; JWE survives TLS-termination at intermediaries).
Malicious intermediary	JWE prevents reading; nested JWS prevents tampering.
Compromised recipient private key	All past JWEs to that recipient are now decryptable. Mitigation: short-TTL keys, periodic rotation, `kid` in the recipient's JWKS just like signers.
AEAD nonce reuse	Generate IV from CSPRNG per encryption; track count if doing constructed nonces.

References

- IETF RFC 7516 *JWE*.
- IETF RFC 7518 *JWA* §4 (key management) and §5 (content encryption).
- IETF RFC 7520 *Examples of Protecting Content using JOSE*.

---

7. JWK & JWKS

A **JWK** (JSON Web Key, RFC 7517) is the JSON encoding of a single key. A **JWKS** (JWK Set) is the JSON encoding of a set of keys. The JWKS is what the AS publishes at `/.well-known/jwks.json` and what every resource server caches.

JWK structure

Member	Required	Meaning
`kty`	yes	Key type (`RSA` , `EC` , `OKP` , `oct`).
`use`	recommended	`sig` or `enc` . Pins the key's purpose.
`key_ops`	optional	More granular ops (`sign` , `verify` , `encrypt` , `decrypt` , `wrapKey` , `unwrapKey`).
`alg`	recommended	Pins the algorithm. Mismatched verifier MUST refuse.
`kid`	recommended	Stable identifier. Token headers refer to this.
`x5c` / `x5t` / `x5t#S256`	optional	X.509 cert chain (when bound to PKI).
(kty-specific)	yes	RSA: `n` , `e` . EC: `crv` , `x` , `y` . OKP (EdDSA): `crv` , `x` . oct (HMAC): `k` .

A public-key JWK omits the private parameters (`d` , `p` , `q` , `dp` , `dq` , `qi` for RSA; `d` for EC/OKP). Private JWKs include them. **Never publish a private JWK on a JWKS endpoint.** The `/.well-known/jwks.json` URL is public; everything in it must be safe to be world-readable.

JWK thumbprint (RFC 7638)

The JWK thumbprint is a deterministic SHA-256 of the canonical JSON of the public key. It is stable across serialization quirks and is what DPoP (`jkt` claim) and other key-binding constructions reference. Useful as a `kid` value when you want `kid` to be derivable than assigned.

JWKS endpoint

The JWKS endpoint is a public HTTPS URL returning a JSON document of the form:

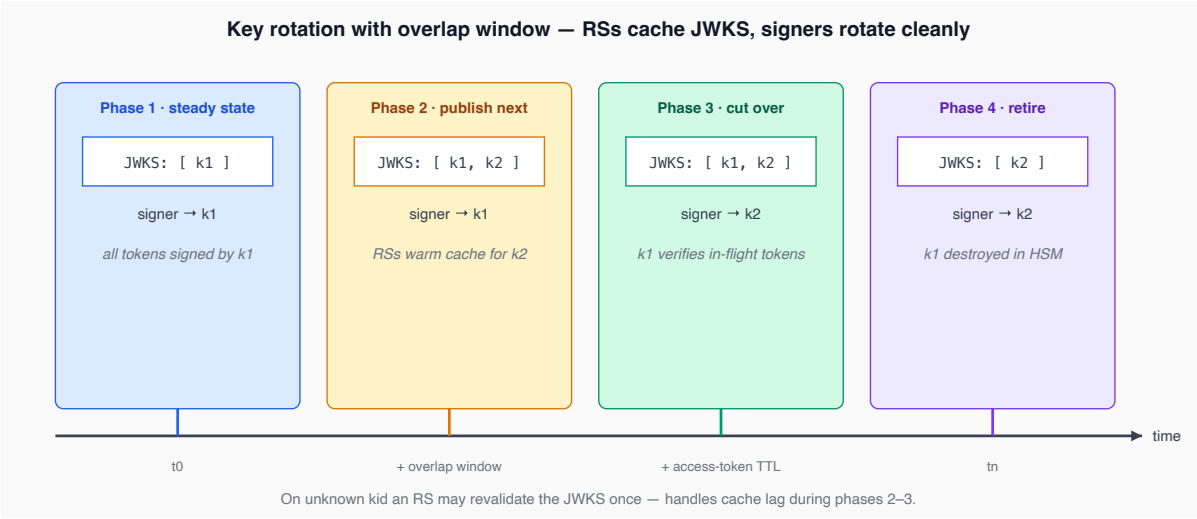
```
{
 "keys": [
 { "kty": "RSA", "kid": "2026-q1", "use": "sig", "alg": "RS256", "n":
        "...", "e": "AQAB" },
 { "kty": "RSA", "kid": "2026-q2", "use": "sig", "alg": "RS256", "n":
        "...", "e": "AQAB" }
 ]
}
```

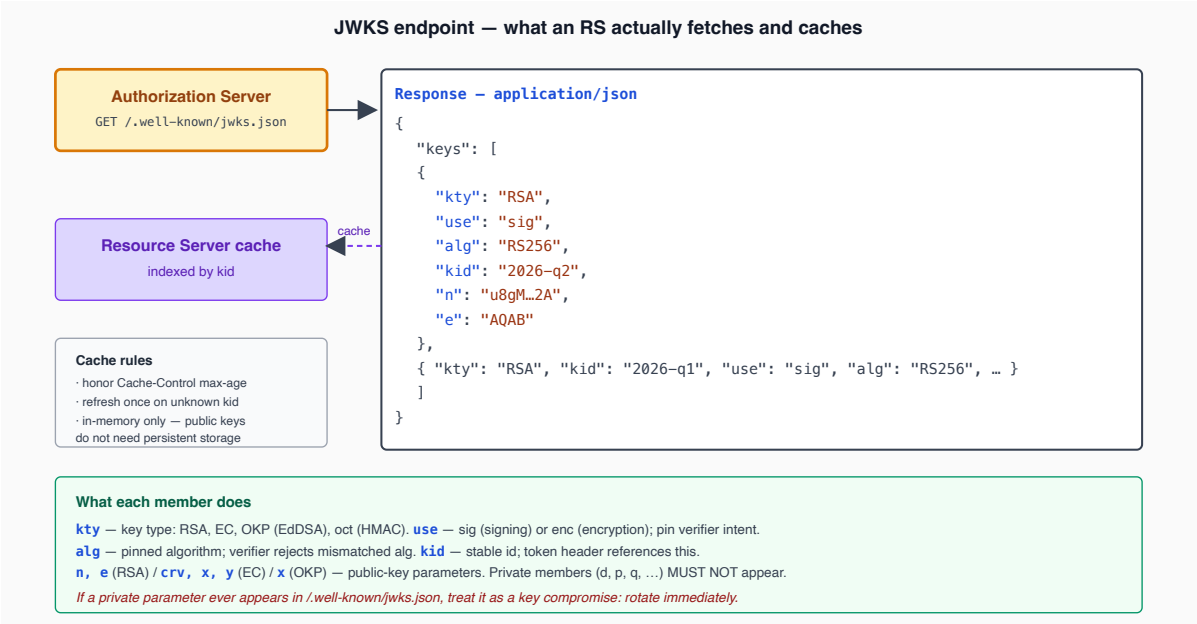
Caching semantics:

- Set `Cache-Control: max-age=...` proportional to your rotation cadence. Common values: `max-age=3600` (1 hour) for high-tempo, `max-age=86400` (24 hours) for slow

rotation.

- Resource servers SHOULD cache the JWKS in memory (and respect `max-age`).
- On unknown `kid`, the resource server MAY revalidate the JWKS exactly once before rejecting the token. This handles rotation overlap (a token signed with the new key arrives before the old cache expires).





Per-tenant / per-app key partitioning

Multi-tenant deployments often partition the JWKS per `appId`: each consuming application sees a JWKS filtered to keys it should trust. This is operationally useful — an `appId` only fetches the keys it needs, reducing payload size — and a defense-in-depth measure: a leaked JWKS for one tenant does not leak signing material for another. (The OneAuth repo example exposes this via `?appId=...` filtering.)

The cost is that the AS must understand `appId` at the JWKS endpoint level. That coupling is local to the AS and does not propagate to the resource servers.

Offline mode

Some resource servers operate in air-gapped or zero-egress environments. For those, the JWKS is shipped as a *file* alongside the application binary, baked at deploy time. The trade-off: zero hot-path dependency on the AS, but rotation requires a redeploy. This mode is appropriate for the lowest-trust environments where outbound HTTPS to the AS is not allowed by policy.

Threat model

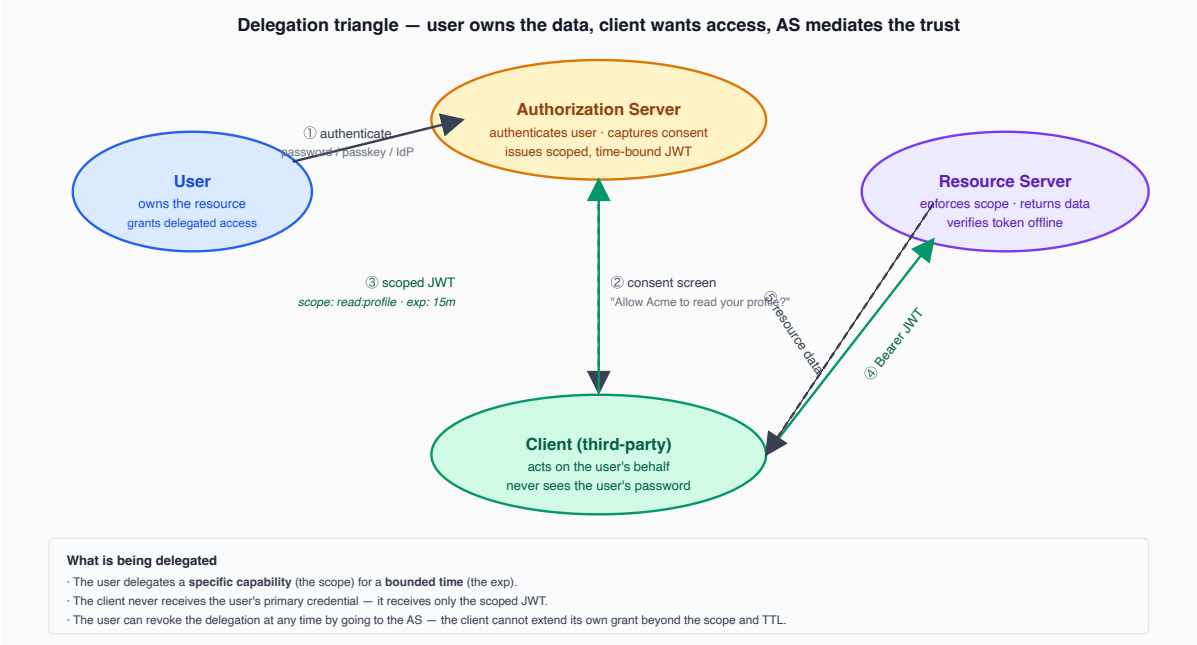
Attack	Mitigation
JWKS spoofing	Always fetch over TLS; HSTS; certificate pinning for very-high-assurance.
Stale JWKS / accepted-after-rotation	TTL discipline; refresh on unknown `kid`.
`kid` injection in token header	Resolve `kid` only against the cached JWKS, never use it as a path or URL component.
Private key leak via misconfigured endpoint	Strict separation: signer never serializes `d` / `p` / `q` to the JWKS publishing path.

References

- IETF RFC 7517 *JSON Web Key*.
- IETF RFC 7638 *JWK Thumbprint*.

Part III — Delegated Authorization: OAuth 2.0

OAuth 2.0 (RFC 6749) is the framework that lets a user grant a third-party application limited access to their resources without handing over their password. It is *not* an authentication protocol — that is a common and dangerous misreading. Authentication on top of OAuth is OpenID Connect (Part V).

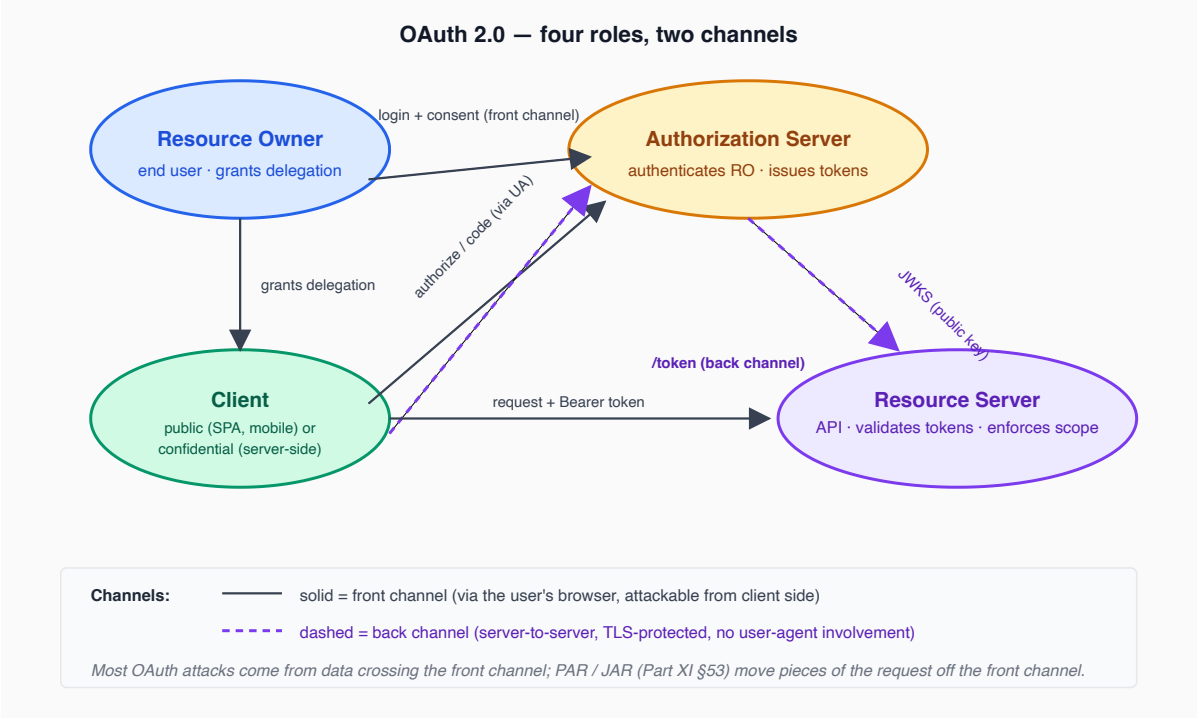


8. Roles & terminology

OAuth defines four roles:

- **Resource Owner (RO)** — the end user who owns the data.
- **Client** — the application asking for access. *Public* clients cannot keep a secret (SPAs, mobile apps); *confidential* clients can (server-side web apps, backend services).
- **Authorization Server (AS)** — the service that authenticates the RO and issues tokens.
- **Resource Server (RS)** — the API that holds the resource and enforces scopes.

the AS issues *capabilities* (tokens), the RS verifies and enforces them. The user grants once, the client acts many times.



Front channel vs back channel

- **Front channel** = via the user's browser. Visible to the user, attackable from the user's machine. Used for the authorization request and (in classic OAuth) the response carrying the code.
- **Back channel** = direct server-to-server. Not visible to the user. Used for the token exchange (code → token), refresh, introspection.

The front-channel security failures (CSRF on `state`, redirect URI manipulation, mix-up, code interception) all come from data crossing the user's browser. Newer mechanisms (PAR, JAR — Part XI §53) move parts of the request from the front channel to the back channel to harden the protocol.

Threat model (high-level; deep dive in Part XIII)

Attack	Vector	Defense
CSRF on the authorization request	Attacker initiates an auth flow on the victim's browser.	`state` parameter, bound to user session.
Redirect URI manipulation	AS redirects the code to an attacker-controlled URI.	Exact-match `redirect_uri` whitelist.
Code interception (public clients)	Attacker observes the code in the redirect.	PKCE (§10).
Token leakage via Referer	Token in URL leaks to next-hop.	Never put tokens in URL; use POST + form_post.

Attack	Vector	Defense
Replay of refresh token	Attacker steals refresh token.	Refresh-token rotation (Part IV §13); sender constraint (Part XI).

References

- IETF RFC 6749 *OAuth 2.0 Authorization Framework*.
- IETF RFC 6750 *Bearer Token Usage*.
- IETF RFC 6819 *OAuth 2.0 Threat Model and Security Considerations*.

---

9. Grants

A **grant** is a flow for getting a token. OAuth 2.0 originally defined four; the subsequent decade has been a story of deprecating the unsafe ones.

Authorization Code Grant

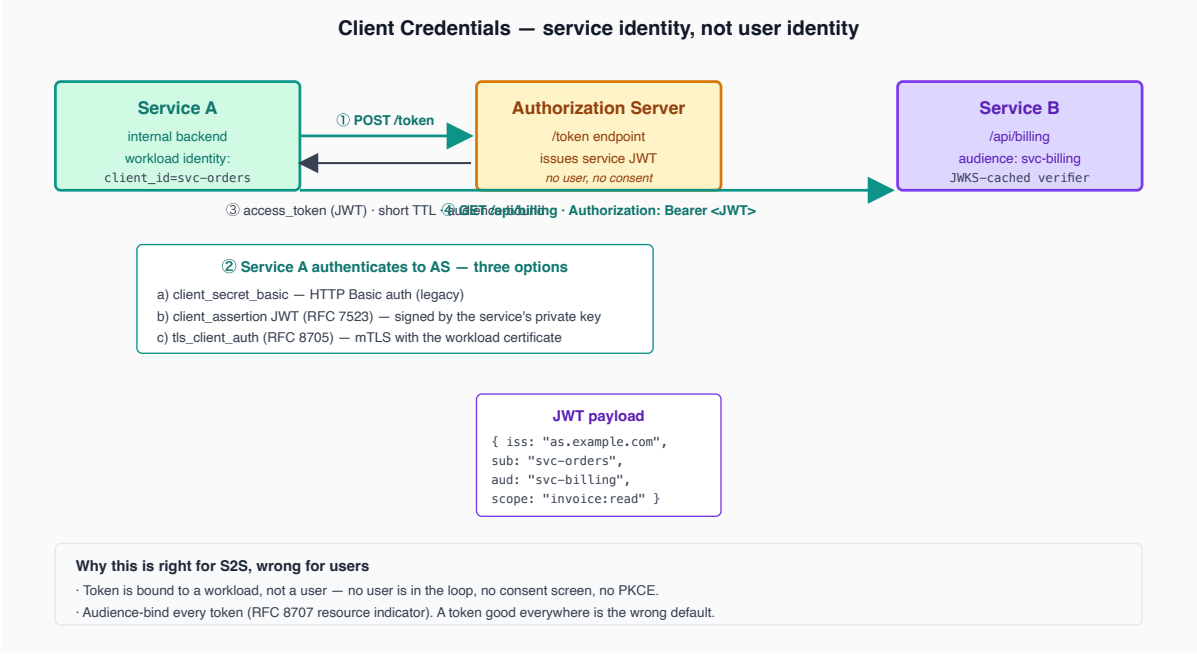
The only grant that should appear in new code. The flow:

1. Client redirects the RO's browser to the AS authorization endpoint with `client_id`, `redirect_uri`, `response_type=code`, `scope`, `state`, and (since 2021) PKCE parameters `code_challenge` and `code_challenge_method`.
2. AS authenticates the RO (login, MFA, consent screen).
3. AS redirects browser back to `redirect_uri` with a short-lived `code` and the echoed `state`.
4. Client server-side POSTs to the AS token endpoint with `code`, `client_id`, `client_secret` (confidential only), `redirect_uri`, and PKCE `code_verifier`.
5. AS returns access token (and ID token, refresh token).

The code is single-use, short-lived (typically 60 seconds), bound to `client_id` and `redirect_uri`, and (with PKCE) bound to a verifier the client proves it knows.

Client Credentials Grant

Machine-to-machine. The client authenticates with its credentials (client secret or, better, signed JWT assertion / mTLS) and gets back an access token. No user, no consent. The standard mechanism for service-to-service calls between trusted backends.



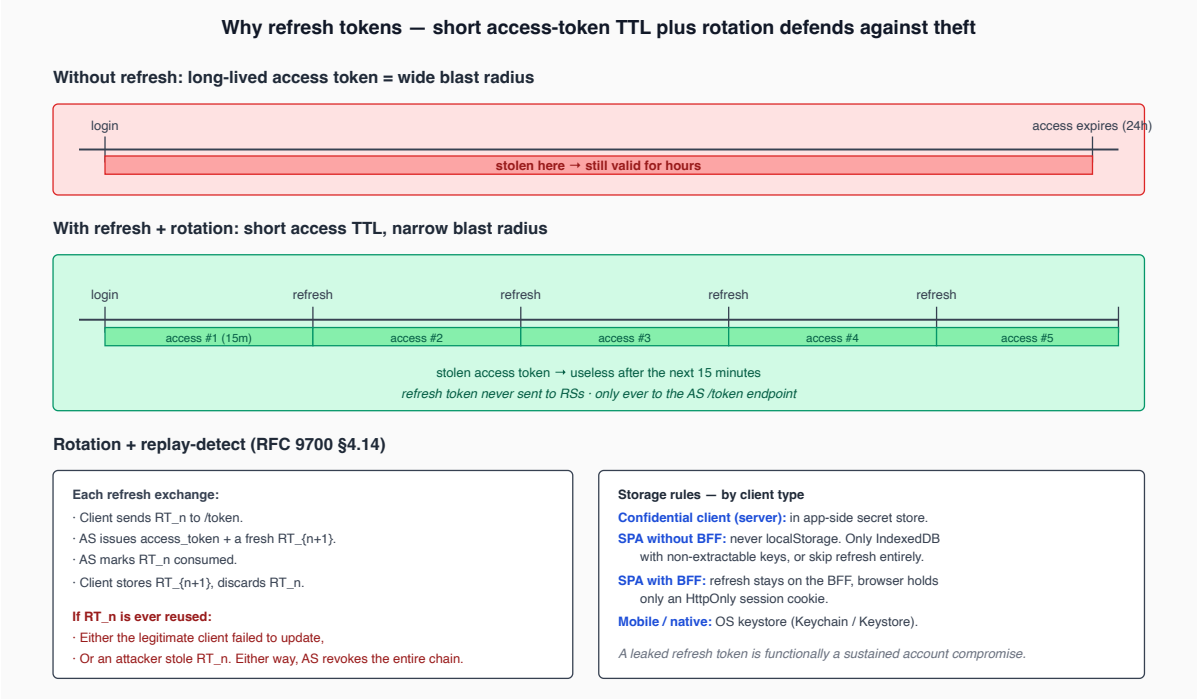
Refresh Token Grant

A refresh token is a long-lived credential the client uses to obtain a new access token without involving the user again. The client posts `grant_type=refresh_token` + the refresh token to the token endpoint and receives a fresh access token (and, ideally, a new refresh token — *rotation*).

The refresh token is the highest-value credential in the system. Its theft is functionally equivalent to a sustained account compromise. Defenses:

- **Short access-token TTL** (5–15 minutes) so the refresh is needed often.
- **Rotation** — every refresh exchange returns a new refresh token and invalidates the old one. Replay of the old token (which an attacker would do) is detected and triggers session-wide revocation (RFC 9700 §4.14).
- **Sender constraint** — DPoP-bound refresh, mTLS-bound refresh.
- **Storage** — confidential client: secret store. Browser: HttpOnly cookie set by a backend-for-frontend, never `localStorage`.

Refresh-token rotation is mandatory in modern deployments — see §63.



Implicit Grant

The original "browser-only" flow: the AS returns the access token directly in the URL fragment, no code exchange. Deprecated by RFC 9700: tokens leak via Referer, browser history, and server logs; without PKCE there is no client authentication; without a code there is no rotation point. **Do not implement. Reject any client trying to use it.**

Resource Owner Password Credentials Grant

The client collects username and password and submits them to the AS. Defeats the entire point of OAuth (the AS is supposed to be the only place that sees the password). Deprecated. The only legitimate use was bootstrapping legacy clients to OAuth; that justification has expired.

Device Authorization Grant

For input-constrained devices: TVs, IoT, CLIs. The device shows a code, the user opens a separate browser, enters the code, authorizes, and the device polls the token endpoint until it gets the token. RFC 8628. Useful pattern when device input is constrained.

CIBA (Client-Initiated Backchannel Authentication)

The client tells the AS to authenticate a known user out-of-band (push notification to phone, SMS); the AS notifies the client when authentication completes. OIDC working group spec. Used in banking and high-assurance B2B.

Threat model

Attack	Targeted grant	Defense
Code interception	Authorization Code	PKCE
Refresh token replay	Refresh	Rotation; sender constraint
Password capture	ROPC	Don't implement ROPC
Token in URL leak	Implicit	Don't implement Implicit

References

- IETF RFC 6749 *OAuth 2.0*.
- IETF RFC 7591 / 7592 *Dynamic Client Registration*.
- IETF RFC 8628 *Device Authorization Grant*.
- OpenID *CIBA Core 1.0*.
- IETF RFC 9700 *OAuth Security BCP*.

---

10. PKCE — Proof Key for Code Exchange

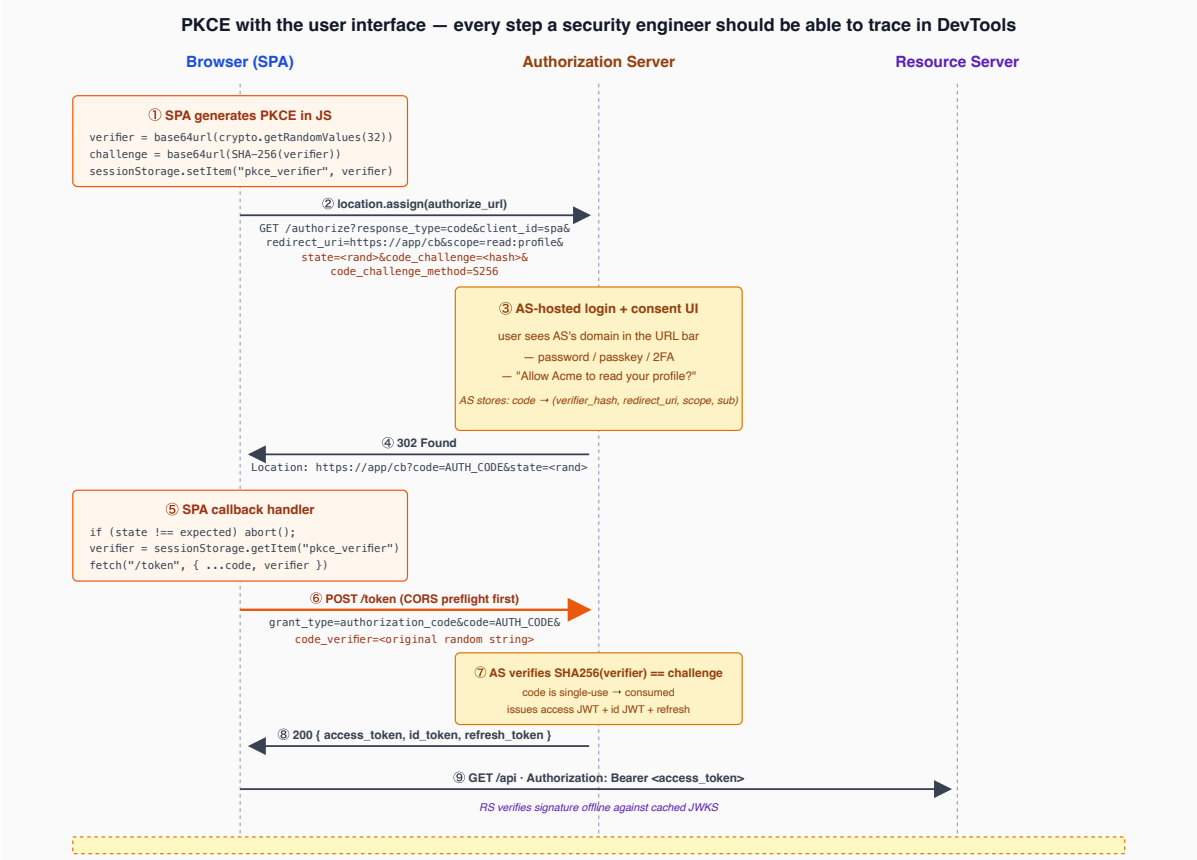
PKCE (RFC 7636, pronounced "pixie") was introduced for native apps that cannot keep a `client_secret`, but RFC 9700 made it mandatory for *all* OAuth 2.0 clients in 2024. The reason is that public-client and confidential-client threat models are not as separate as they once seemed: a confidential client whose redirect endpoint is compromised gains nothing from the secret, and a `client_secret` does not defend against a malicious browser plugin or MITM that captures the code.

What threat does PKCE solve?

The authorization code lives, briefly, in the user's browser between step 3 and step 4 of the auth-code flow. Threats during that window:

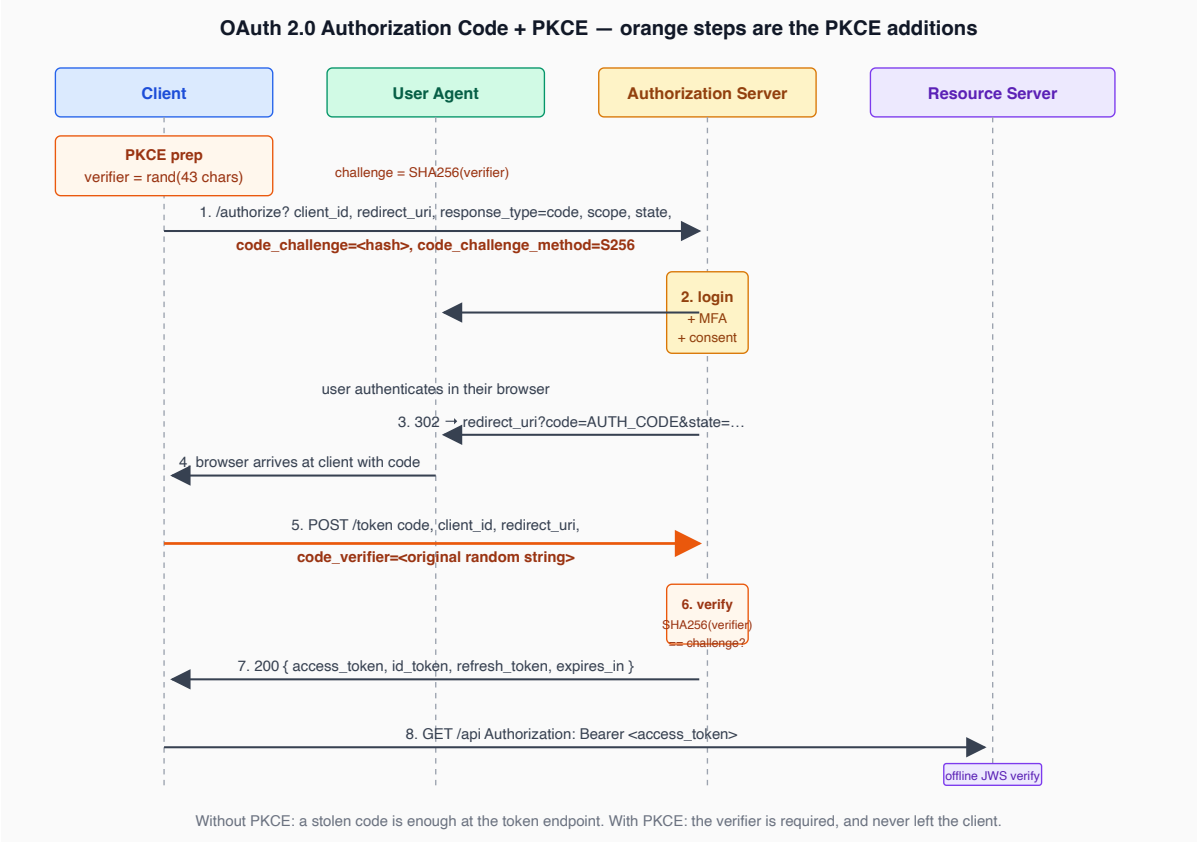
- A malicious mobile app registered for the same custom URL scheme intercepts the redirect.
- A malicious browser extension reads the URL.
- A network observer captures the redirect URL (only possible if TLS is broken — but defense in depth).
- A buggy referrer header leaks the URL to an analytics endpoint.

Without PKCE, the code is sufficient on its own to redeem at the token endpoint. With PKCE, redeeming requires *also* knowing the `code_verifier`, which never leaves the client.



How PKCE works

1. Client generates a high-entropy random string `code_verifier` (43–128 characters, base64url-friendly alphabet).
2. Client computes `code_challenge = base64url(SHA-256(code_verifier))`.
3. Client sends `code_challenge` + `code_challenge_method=S256` with the authorization request. AS stores the challenge against the issued code.
4. Client redeems the code at the token endpoint *with* `code_verifier`. AS recomputes `SHA-256(code_verifier)` and compares to the stored challenge. Match: token is issued. Mismatch: error.



`S256` vs `plain`

Two `code_challenge_method` values are defined: `plain` (challenge = verifier) and `S256` (challenge = SHA-256 hash). `plain` is allowed only for backwards compatibility and provides minimal protection — if the challenge leaks, the verifier leaks. **Always use `S256`. Reject `plain` in the AS.**

Why PKCE for confidential clients too

A confidential client's `client_secret` defends the *token* endpoint against random redemption attempts. PKCE defends the *code* itself against interception in the browser. Different attack surface, different defense, both warranted. RFC 9700 closed the debate: PKCE for everyone.

Threat model

Attack	PKCE response
Code interception (any vector)	Attacker has code but not verifier; redemption fails.
Verifier guessing	43-char minimum; with CSPRNG, 256 bits of entropy. Unguessable.
Hash reversal of challenge	SHA-256 pre-image resistance.

Attack	PKCE response
Verifier reuse across flows	Verifier must be regenerated per flow; AS compares against the specific challenge stored with this code.

References

- IETF RFC 7636 *PKCE*.
- IETF RFC 9700 *Security BCP* §2.1.1.

---

11. Token endpoint, introspection, revocation

Token endpoint

The AS exposes `/oauth/token` (or `/token` in the OIDC convention). It accepts POSTed form-encoded grant requests and returns JSON token responses. Key behaviors:

- Authentication of the client per RFC 6749 §2.3 (Basic auth with client secret, JWT assertion per RFC 7523, mTLS per RFC 8705).
- Validation of the grant type and its parameters.
- Idempotency: retrying the *same* request must not issue duplicate tokens (the code is single-use; a duplicate redemption is a *replay attempt* and SHOULD revoke the original token per RFC 6749 §4.1.2).
- Response: access token, optional ID token, optional refresh token, token type (`Bearer` today; `DPoP` on the roadmap), expiry.

Introspection (RFC 7662)

Introspection is for *opaque* tokens — tokens that are random strings, not JWTs. The RS posts the token to the AS introspection endpoint, the AS returns its claims. Useful when:

- The RS cannot afford the JOSE library footprint.
- The AS wants to revoke tokens before expiry without issuing a JWKS refresh.
- The token format is not exposed to RSs by design (token-as-handle pattern).

When the AS issues JWTs and resource servers verify offline, introspection is unnecessary on the hot path. Some deployments add introspection for environments that demand opaque-token semantics.

Revocation (RFC 7009)

The client posts a token (access or refresh) to `/oauth/revoke`. The AS marks it invalid. Useful at logout, password change, "sign out everywhere" UI affordances.

For JWT access tokens, revocation needs a denylist mechanism (see Part X §49). For refresh tokens, revocation is straightforward — the AS already has a record of every refresh token issued. Refresh-token revocation typically pairs with rotation (§63).

References

- IETF RFC 7662 *Token Introspection*.
- IETF RFC 7009 *Token Revocation*.
- IETF RFC 7523 *JWT Bearer Client Authentication*.

---

12. Scopes, audience, and resource indicators

These three concepts are commonly conflated. They are not the same.

Scope

Scope is a string label for a *capability* the AS grants the client. Scope strings have meaning only within a specific deployment — `read:profile`, `write:repo`, `admin`, `mail.send`. Scopes are requested by the client, presented to the user on the consent screen, and included in the access token. The RS decides what each scope grants; the AS only mediates the grant.

Common conventions:

- Verb-noun (`read:profile`, `write:profile`, `delete:profile`).
- Hierarchical (`mail.send`, `mail.read`, with implicit-include semantics defined per AS).
- Special scopes: `openid` (triggers OIDC), `offline_access` (triggers refresh token issuance), `email`, `profile`.

Audience

`aud` is the intended recipient of the *token*. It is named in the token itself. The RS verifies it matches its own identifier and rejects otherwise.

A scope says *what* the token can do; an audience says *which RS* can accept the token. A token with `scope=read:profile` and `aud=api.example.com` does not work against `api.partner.com` even if that partner also defines a `read:profile` scope.

Resource indicators (RFC 8707)

When a client wants a token specifically for one of several APIs the AS protects, it asks via the `resource` parameter on the authorization or token request. The AS issues a

token with the requested resource as `aud`. This narrows the audience and reduces blast radius if a token is stolen.

Pattern: a single AS, multiple RSs (`api.example.com`, `billing.example.com`, `support.example.com`). Without resource indicators, every token is good for every RS that trusts the AS — bad. With resource indicators, the client requests `resource=https://billing.example.com` and gets a token only that RS will accept.

Scope vs role vs permission (preview)

- **Scope** is a *delegation label* the user grants to a client.
- **Role** is a *user attribute* the AS maintains (e.g., `admin`, `editor`).
- **Permission** is a *fine-grained allow* the RS evaluates per request.

Scopes and roles are coarse and live in the token. Permissions are fine and live in the policy engine. Part IX §43 unpacks the distinction with examples.

Threat model

Attack	Mitigation
Scope inflation by the client	AS limits which scopes a client may request; user consent confirms.
Audience confusion	RS verifies `aud` strictly.
Token reuse across RSs	Resource indicators narrow audience.

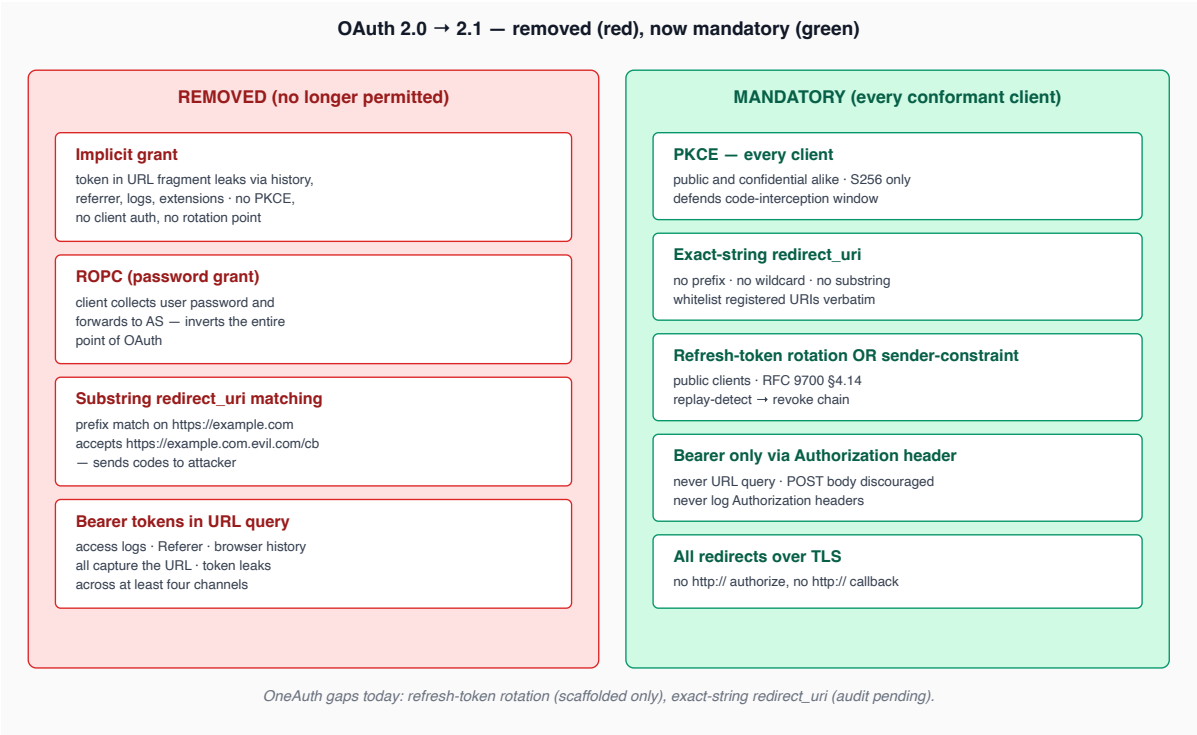
References

- IETF RFC 6749 §3.3 *Access Token Scope*.
- IETF RFC 8707 *Resource Indicators for OAuth 2.0*.

Part IV — OAuth 2.1 & Security BCP

13. What OAuth 2.1 consolidates

OAuth 2.1 (draft-ietf-oauth-v2-1) is not a new protocol — it consolidates the security guidance accumulated since 2012 (RFC 6819, 7636, 7591, 8252, 9068, 9700, 8707) and removes the dangerous parts of 2.0.



OneAuth gaps today: refresh-token rotation (scaffolded only), exact-string redirect_uri (audit pending).

Why the deprecations were necessary

- **Implicit** — token in URL fragment leaks across at least four channels (history, referrer, logs, extensions). No PKCE, no client auth, no rotation point.
- **ROPC** — every reason OAuth was invented says "the client should never see the password". ROPC inverts that.
- **Substring `redirect_uri` matching** — an attacker who can register `https://evil.example.com/oauth/callback` while the AS allows prefix match on `https://example.com` may be able to cause the AS to send codes to `https://example.com.evil.example.com/oauth/callback`.

References

- *The OAuth 2.1 Authorization Framework* — IETF Internet-Draft (current).
- IETF RFC 9700 *OAuth 2.0 Security Best Current Practice*.

---

14. Browser-based apps

A browser-based application (single-page app) cannot keep a secret from itself. Anything in the JS context is reachable by any other JS in the same context — including anything an attacker injects via XSS, a malicious dependency, or a compromised CDN. The OAuth working group has wrestled with this since the implicit-grant deprecation. Current guidance (draft-ietf-oauth-browser-based-apps):

Pattern A: SPA holds tokens (the historical default)

- Authorization Code + PKCE flow.
- Tokens stored in `localStorage` or in-memory.
- Refresh-token rotation if used at all.

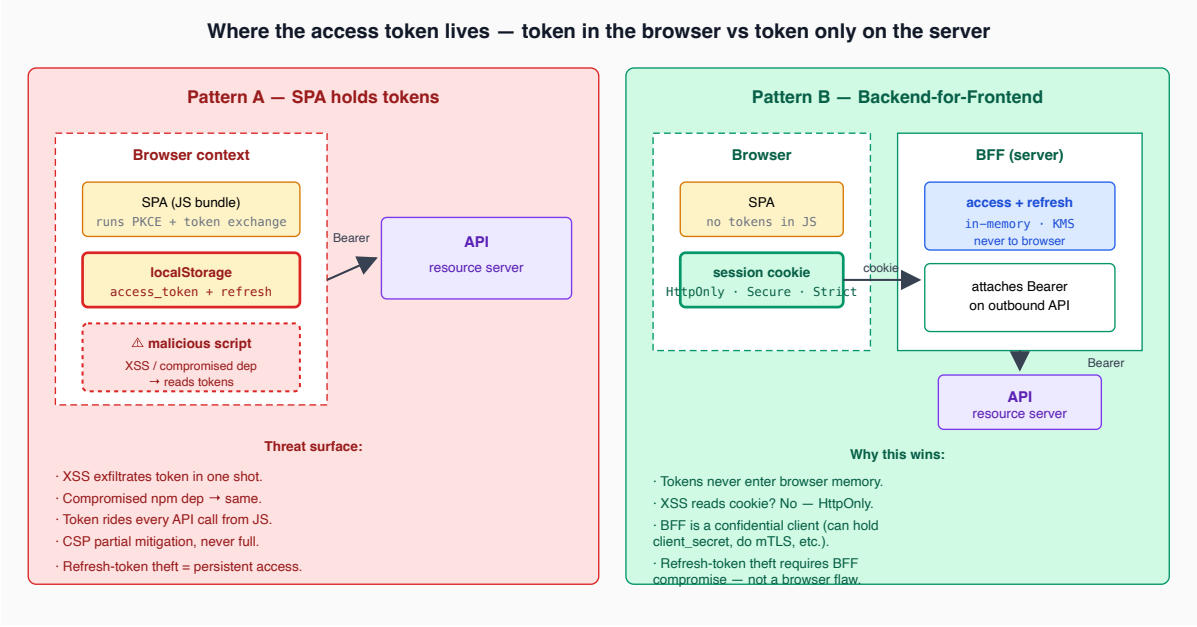
This works against passive attackers (network MITM, server-log scrape) but fails against active attackers (XSS) — once an attacker can run JS in the SPA's origin, they steal the access token, the refresh token, and the DPoP key in one shot. Mitigations are partial: CSP, Trusted Types, dependency hygiene, no inline event handlers. None of them eliminate the failure mode.

Pattern B: Backend-for-Frontend (BFF) — recommended

- The browser holds *no tokens at all*.
- A small server-side component (the BFF) sits in front of the SPA. It owns the OAuth flow, the access token, the refresh token. The browser holds a session cookie (HttpOnly, Secure, SameSite=Strict) bound to BFF-side state.
- API calls from the SPA go to the BFF, which adds the `Authorization: Bearer ...` header server-side.

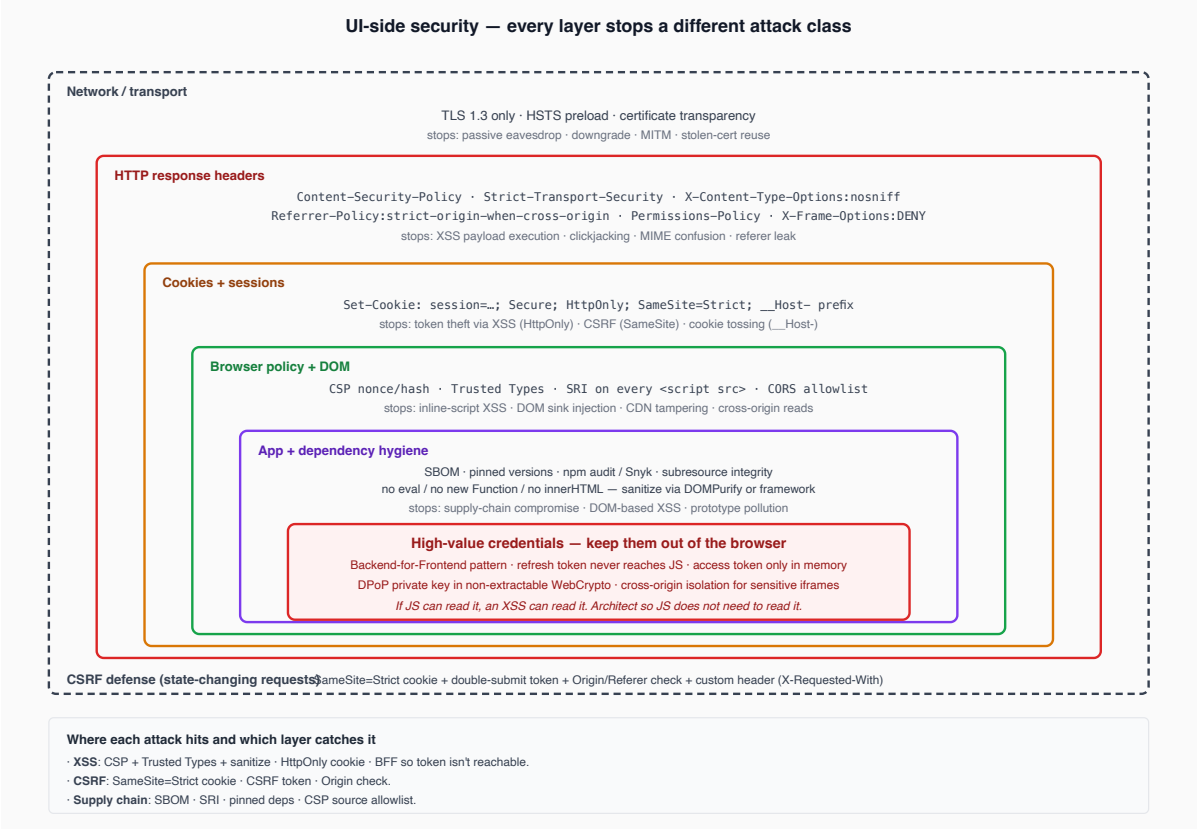
The BFF is functionally a confidential OAuth client, even though the user-facing app is a SPA. The XSS-equivalent attacker now has to compromise the *BFF* to steal tokens — a much higher bar than reading `localStorage`.

BFF is the recommended pattern for any SPA that handles non-trivial data. The session cookie is the only credential in the browser; the access token never leaves the BFF process.



Pattern C: Service worker + isolated key store

A more experimental design: tokens live in a service worker's scope, isolated from the page's JS heap. Web Crypto-bound DPoP keys live in IndexedDB with `non-extractable: true`. Attractive in theory; deployment ergonomics are still rough in 2026. Track for future iterations of the spec.



Threat model

Attack	A: SPA-holds-tokens	B: BFF
Network MITM	TLS	TLS
Token in URL	Don't put it there	Not in URL
XSS exfiltration	Game over	Session cookie HttpOnly — script cannot read
CSRF	OAuth `state`	Cookie SameSite + CSRF token
Subresource compromise (CDN)	Game over	Game over (still need CSP, SRI)

References

- IETF *OAuth 2.0 for Browser-Based Apps* (Internet-Draft).
- *OAuth 2.0 BFF Pattern* — community guidance.

---

15. Native apps

A native (mobile / desktop) app has its own footguns. The historical bad pattern was *embedded webviews* — the app shows the OAuth login inside its own webview, which means the user is typing the IdP password into a UI fully controlled by the app developer. RFC 8252 forbids this and prescribes:

- Use the **system browser** (or a system-managed in-app browser tab: Safari View Controller on iOS, Custom Tabs on Android). The IdP login UI is owned by the OS, not the app.
- Use **claimed HTTPS schemes** (App Links on Android, Universal Links on iOS) for the redirect URI when possible. These bind the redirect to a verified domain ownership and prevent another app from claiming the same scheme.
- Use **PKCE always**.

Custom URI schemes (`com.example.app://`) are allowed but inferior — any app on the device may register the same scheme, leading to redirect interception. Claimed HTTPS is strictly stronger.

References

- IETF RFC 8252 *OAuth 2.0 for Native Apps*.

Part V — OpenID Connect

OpenID Connect (OIDC, OpenID Foundation) is an *identity layer* on top of OAuth 2.0. OAuth answers "may this client take this action on the user's behalf"; OIDC answers "who is the user". The OIDC working group reuses the OAuth flows almost wholesale and adds:

- **ID Token** — a JWT carrying authentication claims about the user.
- **UserInfo endpoint** — a back-channel call for additional user attributes.
- **Discovery** — a `/.well-known/openid-configuration` document listing every endpoint, key, and supported feature.
- **Logout flows** — RP-initiated, front-channel, back-channel.

If your problem is "log this user in", you want OIDC. If your problem is "let this client write to that API", you want OAuth. Most real systems want both — they run OIDC for login and OAuth for API authorization, on the same AS.

16. OIDC vs OAuth

an OAuth access token is **not** evidence of authentication. It is evidence that *some* user, sometime, granted the client permission to do something. The token does not say *who* the user is, *when* they authenticated, or *how strongly*. Many high-profile breaches have come from RSs treating an OAuth access token as an "I am logged in as X" claim.

OIDC fixes this with the **ID Token**: a separately-issued JWT, audience-bound to the *client*, signed by the OP, carrying authentication claims (`sub` , `auth_time` , `acr` , `amr` , `nonce`). The RP validates the ID token to establish the user's identity. The access token, separately, is presented to the RS for resource access.

Mental model:

- ID token: "this user is X, authenticated at time T using methods M". Goes to the *client*.
- Access token: "the bearer is allowed to do scope S on resource R until time E". Goes to the *resource server*.
- Refresh token: "the bearer can ask for new access tokens". Goes to the *client* (storage as in §9).

The client checks the ID token and signs the user in; it then uses the access token to call APIs.

References

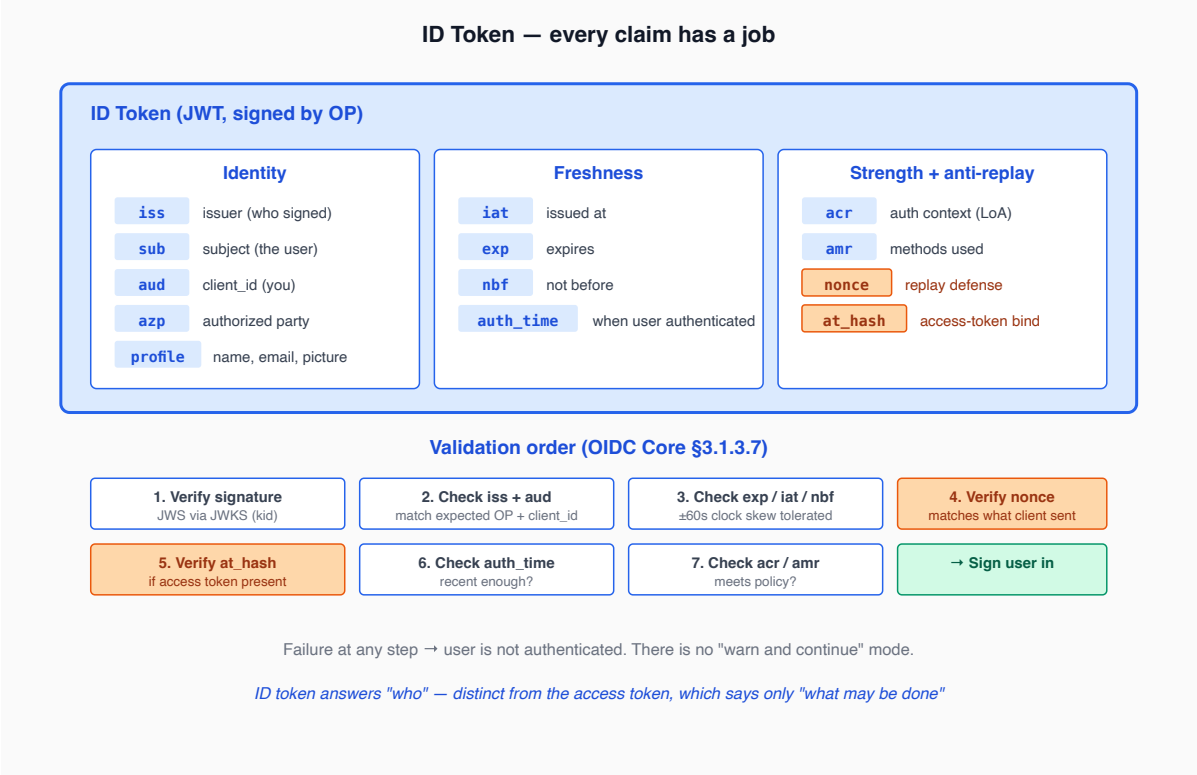
- *OpenID Connect Core 1.0*.
- *OAuth 2.0 vs OpenID Connect: Identity vs Delegation —*

---

17. ID Token claims

The ID token is a JWT (typically JWS-signed; JWE possible). Mandatory and important claims:

Claim	Purpose
`iss`	The OP's identifier. Verifier MUST match against expected.
`sub`	Subject identifier, locally unique within the OP.
`aud`	The client_id of the RP (the receiving client).
`exp`	Expiry.
`iat`	Issued at.
`auth_time`	When the user actually authenticated (may be earlier than `iat` if the OP has a session).
`nonce`	Echoes a value the client sent in the auth request. Replay defense.
`acr`	Authentication Context Class Reference. Strength of authentication.
`amr`	Authentication Methods References. Array of methods used.
`azp`	Authorized Party. When the audience is broader than the immediate client.
`at_hash`	Hash of the access token's first half. Binds ID token to access token.



`nonce` — replay defense

The client generates a random `nonce` and includes it in the authorization request. The OP echoes it back in the ID token. The client verifies `nonce` matches the expected value before trusting the token.

Without `nonce`, a stolen ID token from another browser session could be replayed against the client. With `nonce`, the client knows *this* token came from *this* flow.

`acr` and `amr` — strength signals

`acr` is a string identifying an *authentication policy* — for example, `"urn:mace:incommon:iap:silver"` (InCommon Silver), or `"http://schemas.openid.net/pape/policies/2007/06/multi-factor"`. The OP and RP agree on what each ACR value means.

`amr` is an array of mechanism names: `pwd`, `otp`, `hwk`, `sms`, `face`, `fpt`, `pin`. Use it to distinguish "user with password only" from "user with password + WebAuthn". Step-up authentication (Part VIII §38) is built on these.

ID token validation algorithm

OIDC Core §3.1.3.7 spells out the canonical validation order. The short form:

1. If the ID token is encrypted (JWE), decrypt with the client's private key.
2. Verify the JWS signature using the OP's public key (resolved via `kid` against the JWKS).

3. Verify `iss` matches the expected OP.
4. Verify `aud` contains the client's `client_id`.
5. Verify `exp` is in the future, `iat` is not absurd, `auth_time` (if present) is recent enough.
6. Verify `nonce` matches the value the client sent.
7. If `at_hash` is present, verify it matches `SHA-256(access_token)` truncated.
8. Check `acr` and `amr` against the client's policy.

A failure at any step means the user is not signed in. There is no "warn and continue" mode.

References

- *OIDC Core 1.0* §2 (ID Token), §3.1.3.7 (Validation).

---

18. UserInfo endpoint

UserInfo is an OAuth-protected endpoint at the OP that returns a JSON document of user claims. The client calls it with the access token; the OP returns claims like `email`, `name`, `picture`, `email_verified`, plus any custom claims the OP defines.

Why have UserInfo when the ID token already carries claims? Three reasons:

1. **Size discipline.** The ID token is sent on every request that needs identity binding; keeping it small matters. Heavy claims (full profile, addresses, custom attributes) belong in UserInfo, fetched once per session.
2. **Freshness.** UserInfo claims are read live; ID-token claims are frozen at issuance.
3. **Authorization separation.** Some claims (e.g., medical attributes) require explicit consent and can be gated by additional scopes returned at UserInfo time but not in the ID token.

Response can be:

- Plain JSON (default).
- JWS-signed JSON (when the client requested signed response).
- JWE-encrypted JWS-signed JSON (when high-assurance).

References

- *OIDC Core 1.0* §5.3.

---

19. Discovery & Dynamic Client Registration

Discovery

The OP publishes a JSON document at `/.well-known/openid-configuration` listing every endpoint, supported algorithm, supported scope, supported response type, supported claim, and the URL of the JWKS. The client fetches this once and caches it; the client never has to be hardcoded with endpoint URLs.

When discovery is missing, RPs must be configured with explicit endpoint URLs. Adding the discovery document is a small amount of work and high payoff for interoperability — every off-the-shelf OIDC library will then auto-configure against the OP.

Dynamic Client Registration (RFC 7591)

The OP exposes a `/register` endpoint accepting client metadata (name, redirect URIs, logo, scopes wanted) and returning a `client_id` (and optionally `client_secret`). Useful for:

- Open ecosystems where third parties self-register.
- Multi-tenant deployments with many short-lived clients.

Inverse-style management (RFC 7592) lets the client read/update/delete its own registration via a registration access token.

Many deployments skip DCR and configure clients statically; it is appropriate to add only when third-party clients need to onboard themselves.

References

- *OIDC Discovery 1.0*.
- IETF RFC 7591 *OAuth 2.0 Dynamic Client Registration*.
- IETF RFC 7592 *OAuth 2.0 Dynamic Client Registration Management*.

---

20. Response types & flows

OIDC supports several `response_type` values, controlling what comes back from the authorize endpoint:

`response_type`	Returned via redirect	Notes
`code`	code only	Authorization Code Flow. **Recommended.** Token exchanged at the back channel.

`response_type`	Returned via redirect	Notes
`id_token`	id_token only	Implicit Flow (no access token). Discouraged — front-channel ID token.
`id_token token`	id_token + access_token	Implicit Flow. Discouraged.
`code id_token`	code + id_token	Hybrid Flow. Useful when client needs immediate identity *and* will exchange code for access.
`code token`	code + access_token	Hybrid. Niche.
`code id_token token`	all three	Hybrid. Niche.

`response_mode`

Independently of `response_type`, the `response_mode` controls *how* the response is delivered:

- `query` — URL query string. Default for `code`. Visible in browser history.
- `fragment` — URL fragment. Default for implicit. Not sent to server in Referer.
- `form_post` — POSTed by an auto-submitting form. Recommended for high-assurance flows: keeps the response out of the URL entirely.

Recommended: `response_type=code`, `response_mode=form_post`, with PKCE. This is the modern high-assurance configuration.

References

- *OIDC Core 1.0* §3.
- *OAuth 2.0 Form Post Response Mode*.

---

21. Logout

OIDC defines three logout flows:

RP-initiated logout

The RP redirects the user to the OP's `end_session_endpoint` with `id_token_hint` (so the OP knows which session) and `post_logout_redirect_uri`. The OP terminates the session and redirects back. Simple, common, sufficient for most cases.

Front-channel logout

When the user logs out at the OP, the OP iframes-loads each RP's `frontchannel_logout_uri`. Each RP terminates its own session. Simple but unreliable: third-party cookie blocking, ad blockers, and sandboxed iframes break it.

Back-channel logout

When the user logs out at the OP, the OP POSTs a Logout Token (a JWT) to each RP's `backchannel_logout_uri`. RPs terminate the session server-side. More reliable than front-channel, requires RPs to expose a server endpoint reachable from the OP.

Recommended: implement RP-initiated and back-channel; skip front-channel.

Threat model

Attack	Mitigation
Logout CSRF (forced logout)	`id_token_hint` ensures the OP is logging out the right session.
Stolen logout token	Logout token is a signed JWT with `events` claim; replay-protected by `jti`.
Incomplete logout (RP keeps session)	Back-channel logout: server-side termination.

References

- *OIDC RP-Initiated Logout 1.0*.
- *OIDC Front-Channel Logout 1.0*.
- *OIDC Back-Channel Logout 1.0*.

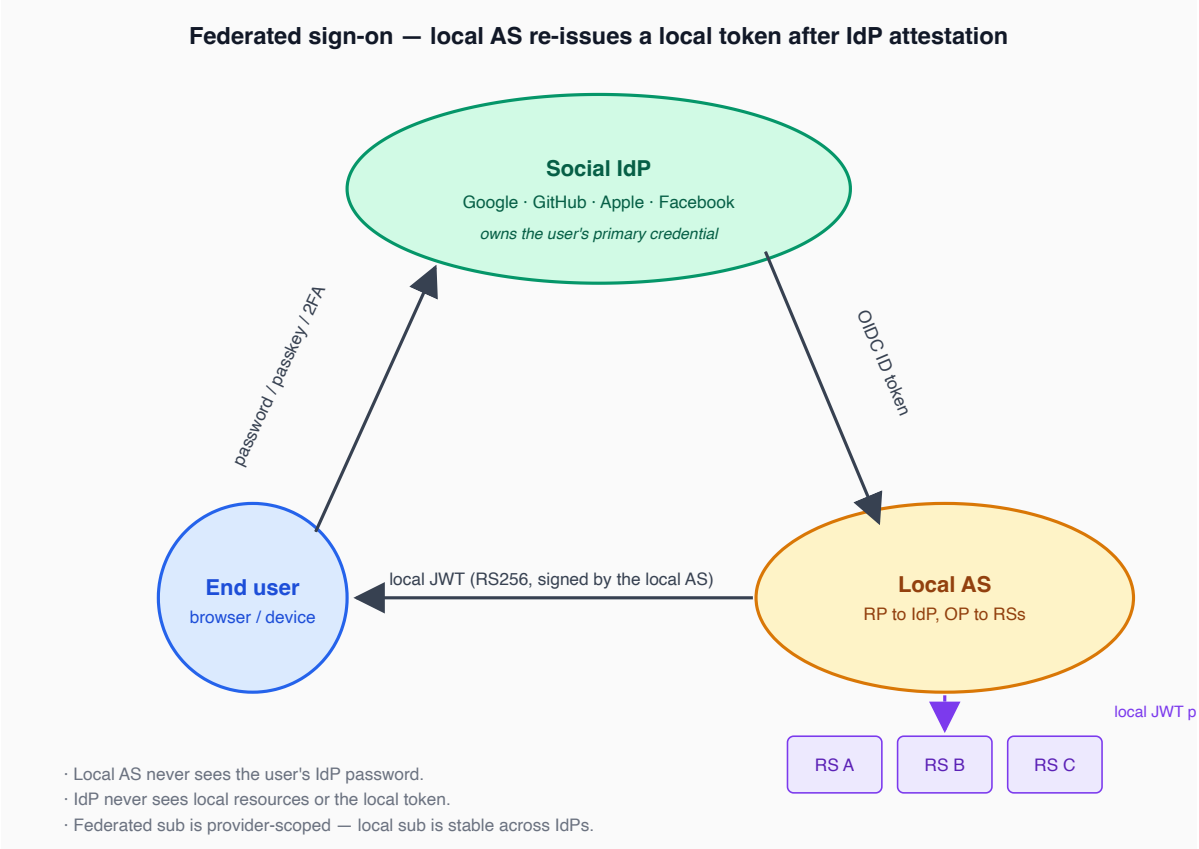
Part VI — Social & Federated Identity

22. The federation model

In a federated sign-in, the local AS does not own the user's primary credential. The user authenticates to a *social IdP* (Google, GitHub, Facebook, Apple, Microsoft) over OAuth/OIDC; the social IdP returns identity claims; the local AS validates those claims and either creates or attaches a local user, then mints a *local* token for the local resource servers.

The trust chain:

```
End user ──[ password / passkey / 2FA ]──▶ Social IdP
 │
 │ ID token / userinfo
 ▼
 Local AS
 │
 │ Local JWT (RS256, locally-signed)
 ▼
 Local Resource Servers
```



Why the local AS issues a local token

Several reasons not to forward the IdP's token:

- **Audience.** IdP tokens are audience-bound to the local AS as the IdP-side client; passing them to RSs would fail audience checks.
- **Lifetime.** IdP tokens have IdP-determined TTLs; the local AS sets policy.
- **Revocation.** Local AS can revoke; cannot revoke an IdP-issued token.
- **Privacy / data minimization.** The local token carries only the claims your RSs need; it does not propagate IdP-only metadata.
- **Stable subject.** The local `sub` is the AS's user ID; the federated `sub` is provider-scoped and may change (e.g., a user's GitHub `sub` is stable; a Facebook `sub` per-app).

Federated `sub` is provider-scoped

The IdP's `sub` is unique within *that IdP*. The same user signing in via Google and via GitHub will have two different `sub` values. The local AS must be able to *link* these to one local user. That is the topic of §23.

References

- *OIDC Core 1.0* §2 (Subject Identifier types: `public` vs `pairwise`).

- *NIST SP 800-63C Federation*.

---

23. Account linking

The most underestimated source of bugs in federated sign-in. The naive policy — "match on email" — looks fine and is wrong.

The problem

User signs up via Google (verified email `alice@example.com`). User later returns and clicks "Sign in with GitHub". GitHub returns email `alice@example.com` (which the user typed into GitHub, GitHub does not verify it). What does the local AS do?

- **Wrong policy: silently link.** Now any attacker who creates a GitHub account with a target email gets access to the target's local account.
- **Wrong policy: silently create new account.** Now Alice has two local accounts; cross-account state is split.
- **Right policy: require proof.** If the new IdP's email matches an existing local user, require a verification step (send a code to the email, or have Alice sign in with the *original* IdP first and then attach GitHub from her settings).

Verified vs unverified email

A claim is only as trustworthy as the issuer.

Provider	`email_verified` semantics
Google	Almost always `true`; Google verified at signup.
GitHub	Optional; user can set arbitrary primary email and toggle verification.
Facebook	`true` if the user verified to Facebook.
Apple	`true` always (Apple ID requires verification).
Microsoft	Depends on tenant config.

A correct AS treats `email_verified=false` claims as *unverified* and never auto-links. Only verified claims are eligible.

Linking policy

1. New IdP sign-in → fetch IdP claims.
2. Look up local user by `(provider, provider_sub)` first. Match → sign in.
3. No match → look up by `(email)` *only if* `email_verified=true` from the new IdP.

4. Email match → require user to *complete a step-up*: sign in with their existing factor (original IdP or password) to confirm intent.
5. After step-up, link the new IdP to the existing local user (insert `(provider, provider_sub)`).
6. No email match → create a new local user, store first IdP as the sign-in path.

References

- Google Identity, *Account Linking with OAuth*.
- *OIDC Core 1.0* §5.7 (Claim Stability and Uniqueness).

---

24. Just-in-time provisioning

JIT provisioning means the local user record is *created on first sign-in* rather than provisioned in advance. The first time a user signs in via Google, the AS creates the local user from the IdP claims (`name`, `email`, `picture`) and stores the federated identifier `(google, sub)`. Subsequent sign-ins find the existing user and sign in.

Trade-offs:

- **For:** zero-friction onboarding, no preregistration step.
- **Against:** harder to enforce invitation-only access; possible for typo'd emails to create stranded accounts.

JIT applies equally to social sign-on and password sign-up. Optional invitation gating belongs behind a config flag for closed-membership systems.

---

25. Provider quirks

Each social provider has its idiosyncrasies. The reference impl handles them in provider-specific user-info adapters.

Provider	Quirk
Google	Returns OIDC ID token; `sub` is stable; `email_verified` is reliable. Use `hd` claim to constrain to a Google Workspace domain.
GitHub	OAuth 2.0, not OIDC. Email is *not* returned by default — must request the `user:email` scope and call `GET /user/emails`. May not be verified. `id` is stable; treat it as `sub`.
Facebook	OAuth 2.0 plus Graph API. Returns short-lived tokens; exchange for long-lived if needed. `id` is **app-scoped** (different per registered app).

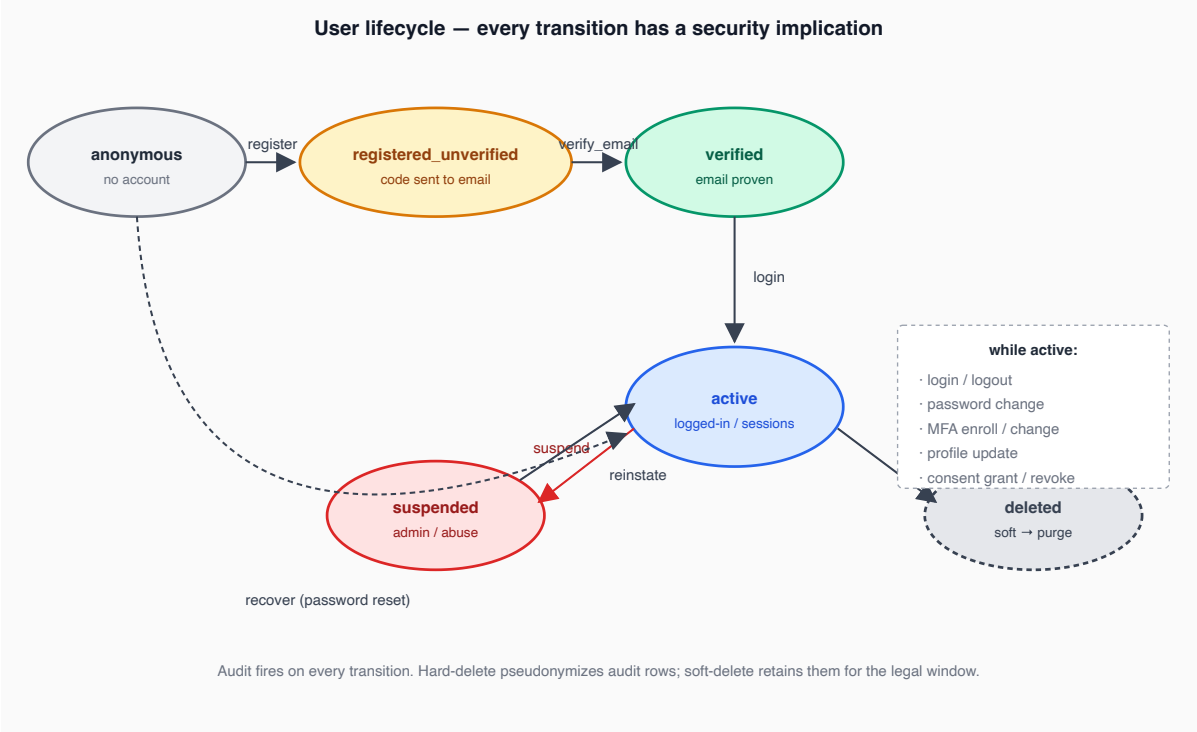
Provider	Quirk
Apple	OIDC. Email may be a relayed `@privaterelay.appleid.com` alias — store it as-is, do not try to resolve to "real" email. Name is returned **only on first authorization**; persist immediately.
Microsoft	OIDC over Azure AD. Multi-tenant — pay attention to `tid` (tenant ID) in addition to `oid` (object ID).

References

- Provider-specific docs (linked in their respective adapters).

Part VII — User Lifecycle

The user lifecycle is the longest-running concern of any identity system. It spans from "the user does not exist" through registration, verification, active use, occasional recovery, and eventual closure. Every state transition has a security implication.



26. Registration

The first transition: from "no account" to "account, pending verification". Registration is the most-attacked endpoint in any auth system — it is unauthenticated, accepts user input, and creates state. Defenses, in layers:

Form design

- **Email + password** is the lowest-friction baseline. Optional username for display.
- **No security questions.** Knowledge-based-auth is broken.
- **No "confirm email" field.** The verification email is the confirmation. A typo'd email creates a stranded account, which is a feature of email verification, not a bug.
- **Constant-time email-existence checks.** Returning "this email is already in use" is a user-enumeration oracle. Better: accept the registration, send a "you already have an account, here's a sign-in link" email to the existing address.

Anti-bot

- **CAPTCHA / hCaptcha** — adds latency, accessibility issues, mostly defeats the cheapest scrapers. Worth it on the registration endpoint.
- **Rate limiting** per IP, per ASN, per country (Part XIII §60).
- **Heuristics** — registrations from data-center IPs, obvious throwaway emails, headless-browser fingerprints.

Email validation depth

Layer the checks:

1. Syntactic (RFC 5322 / 6531 — pragmatically, a permissive regex is fine).
2. MX lookup of the domain.
3. SMTP probe (deliverability) — heavy, rate-limited at the provider.
4. Disposable-domain check (against published lists).
5. **Verification email send.** This is the canonical proof of control.

Reservation pattern

If usernames are user-chosen and unique, race conditions matter. Two registrations for `alice` arriving simultaneously must produce one success and one collision error. Use a unique constraint at the DB level and treat the upsert exception path as the collision case.

Threat model

Attack	Mitigation
User enumeration via "email exists" error	Symmetric responses; verification step is required regardless.
Mass account creation	Rate limit + CAPTCHA + IP reputation.
Email injection (header / payload)	Strict email validation; sanitize template inputs.
Disposable-email signup farms	Disposable-domain blocklists.

References

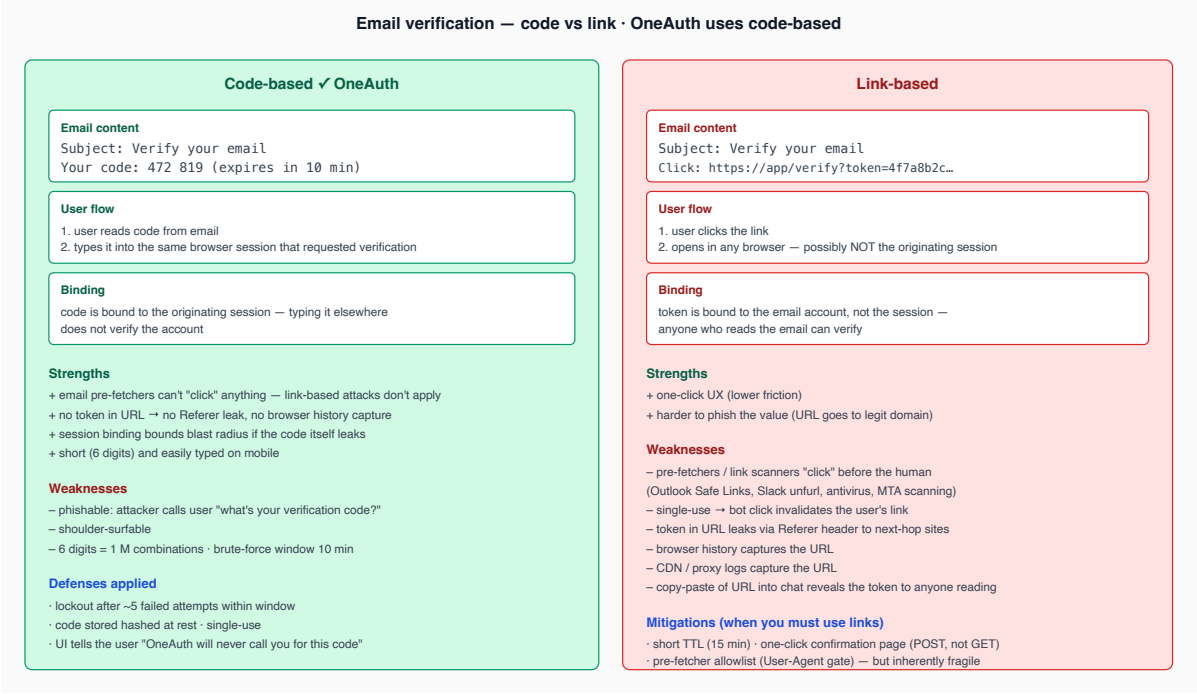
- OWASP *Authentication Cheat Sheet*.
- NIST SP 800-63B §5.

---

27. Email verification

The user receives an email containing either a link or a code. Clicking the link / entering the code proves control of the email and transitions the account from `registered_unverified` to `verified`.

Code-based vs link-based



Code-based verification is the recommended default: six digits, valid for 10 minutes, single-use, throttled to a small number of resend requests per hour.

Code construction

- Six base-10 digits (one million combinations).
- CSPRNG-generated.
- Stored hashed in the database (the database leak should not yield codes).
- Single-use: invalidate on first successful or failed verification.
- Time-bound: ten-minute window, configurable.

Resend throttling

Allowing unlimited resends turns the verification email into a free-mail service for attackers. Limits:

- Per account: 3 resends per hour.
- Per IP: 10 resends per hour.
- Per email-domain: thousands per hour (for legitimate corporate users).

- Hard cap on total resends per account-lifetime: 50.

Threat model

Attack	Mitigation
Brute-force the six-digit code	Lockout after ~5 failed attempts; 10-minute window.
Email pre-fetcher invalidates link	Use codes, not links.
Verification-email spam-as-DoS	Resend throttling; per-IP rate.
Code phishing	Code is bound to the original session; user is told to ignore unsolicited codes.

References

- NIST SP 800-63B §5.1.
- OWASP *Forgot Password Cheat Sheet*.

---

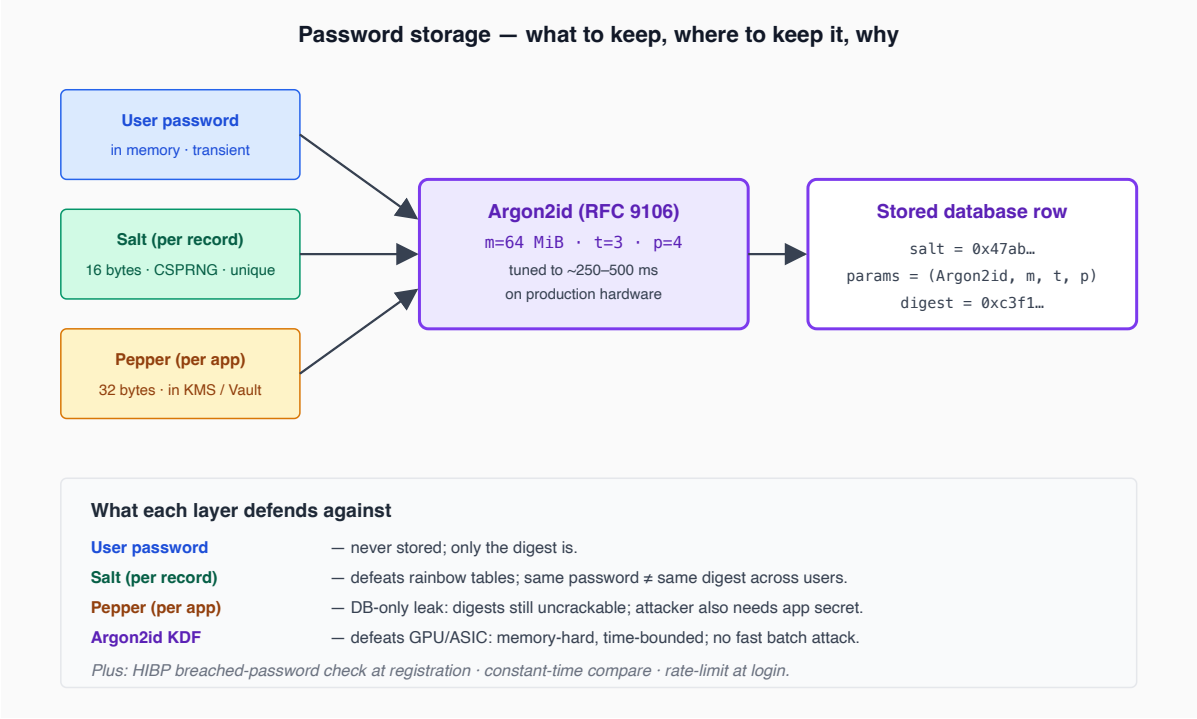
28. Password handling

Passwords are unfashionable but unavoidable. They remain the most widely-used credential and will not vanish in this decade. Storing them safely is non-negotiable.

Storage: KDF, not hash

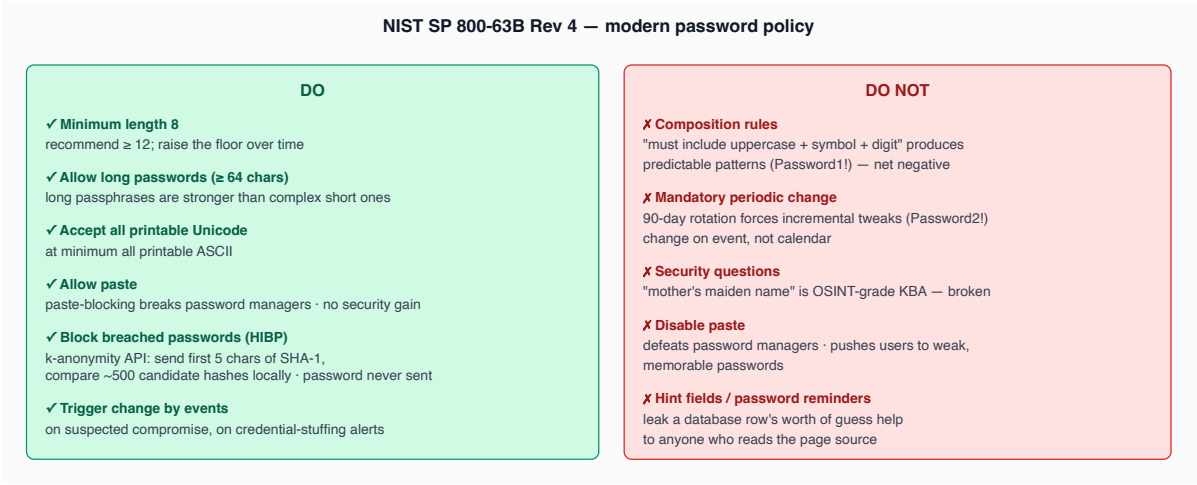
```
stored = (salt, kdf_params, kdf(password, salt, kdf_params, pepper))
```

- **KDF**: Argon2id, parameters tuned to ~250 ms on the production hardware. As a 2026 baseline: `m=64MiB, t=3, p=4`.
- **Salt**: 16 bytes, CSPRNG-generated, unique per record.
- **Pepper**: optional, 32 bytes, stored in the application secret store (KMS or env). Same value for all records of a deployment.



Password policy

Codified by NIST SP 800-63B Rev 4:



Strength feedback

Use **zxcvbn** or similar pattern-detection libraries to give *real* strength feedback, not character-class theater. zxcvbn understands "P@ssw0rd!" is weak even though it satisfies a naïve policy.

Password change vs reset

- **Change**: user is authenticated, knows the old password. Verify old password (constant-time compare), then update.

- **Reset**: user is anonymous, has lost the password. Use the email-verified channel (§30).

Threat model

Attack	Mitigation
Database leak → offline cracking	Argon2id; pepper.
Credential stuffing	Breached-password check; rate limit (Part XIII).
Online brute force	Rate limit; account lockout (with care).
Memory-scraping malware	Out of an AS's scope; mitigated by OS protections.
Password reuse across sites	Breached-password check; user education; passkeys (Part VIII).

References

- NIST SP 800-63B Rev 4 §5.1.1 (Memorized Secrets).
- IETF RFC 9106 *Argon2*.
- HaveIBeenPwned k-anonymity API documentation.
- OWASP *Password Storage Cheat Sheet*.

---

29. Login

The login transition: anonymous → authenticated session. Rules:

- **Verify password in constant time.** KDFs are inherently constant-time per evaluation; the comparison of digests must use a constant-time `equals`. (Almost all KDF libraries handle this.)
- **Generic error messages.** "Invalid email or password" — do not distinguish "no such user" from "wrong password". Distinguishing is a user-enumeration oracle. *Symmetric latency too*: a "no such user" branch that returns in 5ms while "wrong password" takes 250ms (KDF time) is the same oracle. Run a dummy KDF on the no-such-user branch.
- **Lockout strategy with care.** Lock for 15 minutes after 10 failed attempts within an hour, *but* allow the legitimate user to reset without waiting. Hard lockouts that survive a password reset enable account-DoS.
- **Anomaly signals.** Flag (don't block by default): impossible-travel, new device, new ASN. Surface in security-events log; require step-up for sensitive operations than blocking the login.

- **Bind session to IP / device with care.** Strict binding breaks legitimate users on mobile networks. Loose binding gives no security gain. Most teams should not do session-IP binding for end-users; do it for admin sessions only.

Threat model

Attack	Mitigation
User enumeration	Symmetric responses + symmetric latency.
Brute force	Rate limit; lockout; CAPTCHA after N failures.
Credential stuffing	Breached-password check (at registration & rotation); MFA (Part VIII).
Session fixation	Issue a new session ID at login.

References

- OWASP *Authentication Cheat Sheet*.

---

30. Recovery

Password reset is a sensitive flow because it is the *bypass* of the password requirement. The standard recipe:

1. User enters email, requests reset.
2. AS responds the same way regardless of whether the email is registered (no enumeration).
3. If registered, AS generates a single-use, time-bound, CSPRNG-strong token, hashes it, stores hash + expiry against the account, and emails the *clear* token (or a code) to the address.
4. User clicks / enters; AS verifies and shows the new-password form.
5. User submits the new password; AS validates against policy (§28), updates the KDF digest, invalidates all active sessions, sends a confirmation email to the user.

Critical details

- **Single-use.** Once redeemed (success or failure beyond a small retry budget), invalidate.
- **Short-lived.** 30 minutes is a common default.
- **Hashed at rest.** A leak of the database should not give attackers usable reset tokens.
- **Confirmation email.** Notify the *real* user that a reset happened, with an "I did not request this — recover my account" link.

- **Invalidate sessions.** A reset is an implicit "I have been compromised" signal. Kill all active sessions and refresh tokens.
- **Generic responses.** Do not say "no account exists for that email"; always respond as if the email was sent.

Account recovery (no password, no email access)

The hard case: the user has lost both the password and the verified email. Knowledge-based-auth (mother's maiden name) is broken. Options:

- **Backup codes** — printed at MFA enrollment, stored offline.
- **Out-of-band verification** — phone call, video selfie, identity document upload (heavy; reserved for high-value accounts).
- **Trusted-contact recovery** — a designated contact vouches.
- **No recovery** — the account is lost. (For some threat models this is the right answer.)

Threat model

Attack	Mitigation
Reset-token theft via email	TLS to email provider; tokens hashed at rest; short TTL.
Reset-flow as enumeration oracle	Symmetric responses.
Reset-flow as session-hijack vector	Sessions invalidated on reset.
Reset-flow as DoS (mass reset emails to victims)	Per-account / per-IP rate limit.

References

- OWASP *Forgot Password Cheat Sheet*.
- NIST SP 800-63B §6.1.

---

31. Profile, consent, and audit

User-modifiable state — name, picture, marketing preferences, granted consents — needs:

- **Audit on every change.** Old value, new value, who, when, IP/device.
- **Sensitive-change confirmation.** Email change requires verification of the new email *and* notification to the old email.

- **Consent records.** Per-purpose, time-stamped, scoped. GDPR Art. 7 requires demonstrability — you must be able to prove the user consented to each purpose at a specific time.
- **Privileged-attribute changes need step-up.** Changing the email or 2FA requires a recent strong authentication (re-prompt password or WebAuthn).

References

- GDPR Art. 7 (Conditions for Consent), Art. 30 (Records of Processing).
- ISO/IEC 29184 (Privacy notices and consent).

---

32. Account closure

The end of the lifecycle. Two modes:

- **Soft delete** — mark `deleted_at`, retain rows, schedule purge after the legal retention window. Simpler; supports "I changed my mind" undo.
- **Hard delete** — irreversible. Required by GDPR right-to-erasure once retention window has passed.

Conflicts with retention requirements:

- Audit logs of *security* events (logins, password changes, MFA enrollments) often need to be retained for forensic / compliance reasons even after account closure.
- Resolution: *pseudonymize* — replace the `user_id` in audit logs with a non-reversible token. The events remain, the linkable identity does not.

Recommended: scheduled-purge soft delete with pseudonymization on hard delete.

References

- GDPR Art. 17 (Right to Erasure).
- ISO/IEC 27001 *Information Security Management Systems* (audit retention guidance).

Part VIII — Multi-Factor & Passwordless

Single-factor authentication — knowledge-only — is no longer adequate for non-trivial threat models. Multi-factor authentication (MFA) introduces a second factor from a different *class*. Passwordless flows (passkeys, magic links, WebAuthn-only sign-in) eliminate the password and rely on possession + verification factors.

33. MFA factor classes

The classical taxonomy:

- **Knowledge** — something you know (password, PIN, recovery code).
- **Possession** — something you have (TOTP authenticator app, hardware security key, phone with passkey).
- **Inherence** — something you are (fingerprint, face, voice).

MFA combines two of these from *different* classes. Two passwords are not MFA; password + TOTP are.

Factor strengths

Factor	Phishing-resistant	Lost-easily	UX
Password	No	No (memorized)	Familiar
SMS OTP	No (SS7, SIM swap)	Yes (phone loss)	Easy
Email OTP	Partial (depends on email security)	Yes	Easy
TOTP	Partial (still phishable if user types it)	Yes (device loss)	Familiar
Push notification	Partial (notification fatigue → approval)	Yes	Easy
WebAuthn / passkey	**Yes** (origin-bound)	Depends (synced vs device-bound)	Excellent
Hardware security key	**Yes**	Yes (key loss)	Good
Biometric (platform)	Yes (locally bound)	Yes (device loss)	Excellent

The only factor that is *fundamentally* phishing-resistant is WebAuthn (and its hardware-key cousin), because the browser-and-authenticator binding is to the origin, not to a user-entered credential. SMS, email, TOTP, and push all rely on the user *not* being tricked into entering a code into a malicious page.

Why SMS is still in the table

It is widely deployed, accessible, and better than no second factor. It is not adequate for high-value accounts. Use it as a fallback option, not a primary.

References

- NIST SP 800-63B §5.
- *NIST SP 800-63B Rev 4* (deprecates SMS for AAL3, allows for AAL2 with restrictions).

---

34. TOTP & HOTP

HOTP (HMAC-based One-Time Password, RFC 4226) is a counter-based OTP. The authenticator and server share a secret; each OTP is `truncate(HMAC-SHA-1(secret, counter))` modulo 10^digits. Counter increments per OTP.

TOTP (Time-based, RFC 6238) replaces the counter with `floor(unix_seconds / 30)` — the OTP rotates every 30 seconds.

The authenticator (Google Authenticator, Authy, Aegis, 1Password) stores the secret. Provisioning is the QR code:

```
otpauth://totp/ServiceName:user@example.com?secret=
<base32>&issuer=ServiceName&algorithm=SHA1&digits=6&period=30
```

Drift window

Clocks drift. The server SHOULD accept codes from `±1` step (i.e., the previous and next 30-second window) to tolerate small drift without UX friction. Larger drift windows weaken security; ±1 is the standard compromise.

Storage

- The secret is high-value: anyone with it can produce valid OTPs forever. Store encrypted at rest; protect with the same care as password peppers.
- OTPs themselves should be one-time: track recently-used OTPs (with expiry tied to the drift window) and reject reuse.

Threat model

Attack	Mitigation
Phishing the OTP	Inevitable — only WebAuthn fixes this. Reduce blast radius with short OTP windows.
Compromised authenticator app	User reports loss → admin invalidates the TOTP secret, user re-enrolls.
Brute force OTP (one million combinations)	Rate limit OTP attempts: 5 per minute, lockout after 10.
Server-side secret leak	Encrypt at rest; pepper.

References

- IETF RFC 4226 *HOTP*.
- IETF RFC 6238 *TOTP*.

---

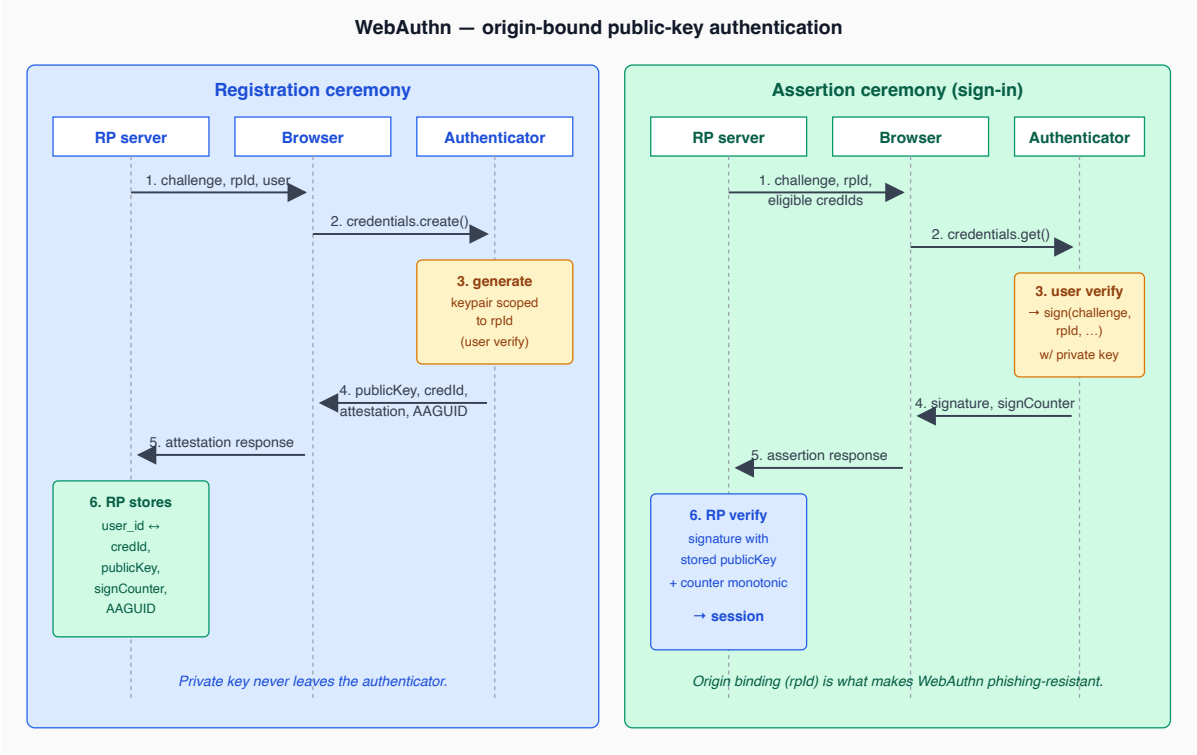
35. WebAuthn / FIDO2

WebAuthn (W3C) plus CTAP2 (FIDO Alliance) is the modern phishing-resistant standard. Together they specify:

- A browser API (`navigator.credentials.create()` / `.get()`) that prompts the user and talks to an authenticator.
- A wire format between the browser and the authenticator (CTAP2 over USB / NFC / BLE for roaming keys; platform-internal for built-in authenticators).
- A server-side verification flow.

The authenticator is a separate component holding a private key. The relying party never sees private key material. Origin binding (`rpId`) is enforced by the browser, which is what makes WebAuthn phishing-resistant: the authenticator will only respond to the origin it was registered against, no matter what the user clicks on.

Two ceremonies



Registration

1. RP server generates a random challenge, sends it to the client with `rpId` and user info.
2. Browser invokes `navigator.credentials.create()`; authenticator generates a fresh keypair scoped to `rpId`.
3. Authenticator returns the public key, a credential ID, optional attestation (proof of authenticator type/manufacturer), and the AAGUID (authenticator model identifier).
4. RP server stores `(user, credential_id, public_key, sign_counter, transports)`.

Assertion (sign-in)

1. RP server generates a challenge, optionally including the credential IDs eligible for this user (or empty for *discoverable credentials*).
2. Browser invokes `navigator.credentials.get()`; authenticator finds the matching key, signs `(challenge, rpId, ...)`.
3. Authenticator returns signature + sign counter.
4. RP server verifies signature with the stored public key, checks the sign counter is monotonic (defends against cloned authenticators), grants session.

User verification

- **uv (user verification)** — the authenticator authenticated the *user* (PIN, biometric) before signing. Required for AAL2.
- **up (user presence)** — the authenticator confirmed *some* user is physically present (button tap). Lighter requirement.

Attestation

The authenticator can attach an attestation statement proving its make/model. Three modes:

- **None** — no attestation. Most common in consumer flows.
- **Indirect** — attestation, but anonymized through a CA.
- **Direct** — full attestation. Used for compliance environments where only certain authenticator models are allowed.

Recommendation: WebAuthn with **none** attestation by default; allow operators to enforce direct attestation in regulated deployments.

Platform vs roaming authenticators

- **Platform** — built into the device (Touch ID, Windows Hello, Android biometric). Always available, cannot move to another device unless synced.
- **Roaming** — separate hardware (YubiKey, SoloKey). Portable across devices.

A user typically registers multiple authenticators: a platform one for daily use, a roaming one for backup.

References

- W3C *Web Authentication: An API for accessing Public Key Credentials, Level 3*.
- FIDO Alliance *Client to Authenticator Protocol (CTAP) 2*.
- *WebAuthn Recovery* (best-practices guidance).

---

36. Passkeys

A passkey is a WebAuthn credential with two extra properties built into the modern UX:

- **Discoverable credential** (resident key) — the credential ID is stored on the authenticator and can be enumerated, so the user does not have to type a username before signing in. The browser shows "use your passkey for example.com".
- **Sync** (optional) — the passkey is replicated across the user's devices via a sync provider (iCloud Keychain, Google Password Manager, 1Password). Loses some

"device-bound" guarantee in exchange for usability.

Synced vs device-bound

	Synced	Device-bound
Backed up	Yes (sync provider)	No
Lost device → lost passkey	No (use sync)	Yes (use a backup credential)
Phishing-resistant	Yes	Yes
Hardware-bound (compromise of the device key compromises one device only)	No	Yes
Suitable for high-assurance / FAPI	Less so	Yes

For consumer flows: synced passkeys are excellent UX with adequate security. For high-assurance flows: device-bound (often hardware-token) passkeys plus a recovery procedure.

AAGUID

The Authenticator Attestation GUID identifies the authenticator model. Useful for:

- Showing the user which device a credential was registered from.
- Allowlisting / denylisting models in enterprise deployments.
- Telemetry and analytics.

The FIDO Metadata Service publishes a registry of AAGUIDs.

Threat model

Attack	Mitigation
Phishing	Origin-binding makes WebAuthn assertion fail on attacker-origin pages.
Server-side credential leak	Public keys only — no useful information for the attacker.
Authenticator cloning	Sign counter monotonicity (limited; some authenticators don't increment).
Sync-provider compromise (synced passkeys)	Compromise of iCloud / Google account compromises every passkey synced through it. Rare but non-zero.
Lost device	User signs in with another registered authenticator (the user must have two).

References

- *Passkeys: Sign in with the passwordless future* — FIDO Alliance / web.dev.

---

37. Magic links & email OTP

A magic link or email OTP authenticates by proving control of the email. Sometimes useful as a passwordless flow, sometimes useful as a recovery fallback. Always tempting; rarely the right primary mechanism.

When they help

- Returning-user UX: skip the password if the email is verified and the device is recognized.
- Account recovery (we already use this in §30).
- Low-stakes accounts (newsletter signups, ephemeral guest sessions).

When they don't

- High-value accounts. Email is a notoriously phished-and-compromised channel.
- Adversarial environments where the user's email may be hostile (corporate email scanners that "click" links to inspect them, breaking the link).

Email codes are appropriate for verification and recovery; magic-link login is *not* recommended as the primary credential for non-trivial systems.

---

38. Step-up authentication

Some operations need stronger authentication than the user's current session provides. Examples: changing the email address on file, viewing payment history, transferring funds, modifying MFA enrollment.

The step-up pattern:

1. User attempts a sensitive action.
2. RS checks the session's `acr` / `amr` claims against the action's requirement.
3. If insufficient, redirect (or in-app prompt) to perform an additional authentication.
4. AS issues an updated token reflecting the new `acr` / `amr` and `auth_time`.
5. User retries the action; RS now permits.

OIDC supports this via `acr_values` on the authorization request — the RP asks the OP to authenticate the user with at least the named ACR.

The hidden complexity is that step-up may need to *layer over* the existing session, not invalidate it. Naïve implementations log the user out and back in, losing context. Correct implementations re-authenticate while preserving the session.

References

- OpenID Connect Core 1.0 §3.1.2.1 (`acr_values`).
- *OAuth 2.0 Step-up Authentication Challenge Protocol* (Internet-Draft).

Part IX — AAAF: Authentication, Authorization, Accounting

39. The acronym, expanded

Classic networking spoke of AAA: Authentication, Authorization, Accounting. The "F" for Framework is a deliberate addition: a recognition that these are not three independent layers but a coherent framework with cross-cutting concerns (audit applies to authn and authz; principals are the input to authz; consent decisions are recorded in accounting).

Framing the responsibility split cleanly:

- **Authentication** — establish a verified principal. Output: a token. Lives at the AS.
- **Authorization** — given a verified principal, decide allow/deny on a resource. Lives at the RS, with policies sometimes at a PDP.
- **Accounting** — record what happened. Cross-cutting; lives in a structured event log.

In a microservice mesh, these split as:

- AS issues tokens.
- API gateway terminates TLS, may pre-validate tokens, forwards.
- RS (any service) re-validates tokens, applies authorization, emits audit events.
- A PDP service (optional) makes authorization decisions for complex policies.
- A SIEM ingests audit events.

Why "framework" matters: a successful design treats these as one concern with one mental model, not three teams arguing across boundaries.

---

40. Authentication architecture

The on-the-wire token is JOSE (Part II). The ceremony is OAuth/OIDC (Parts III–V). What remains is *how the resource server hydrates the principal from the token* and *where in the request pipeline that happens*.

Where token validation lives

Three patterns, in increasing complexity:

1. **At the gateway only.** Gateway validates tokens, forwards user context as headers (`X-User-Id`, `X-User-Roles`). Services trust the gateway. Simpler ops; risky if any service is reachable bypassing the gateway.
2. **At the service only.** Each service validates the token. Robust to gateway bypass; duplicates work and JWKS-fetching across services.
3. **Both.** Gateway pre-validates (fast-fail), service re-validates (defense in depth). Recommended for non-trivial deployments.

The OneAuth repo example ships `one-auth-jar` for pattern 2 / 3: an embedded filter that validates JWS signatures against a cached JWKS and hydrates the principal into Spring Security context.

Principal hydration

Once the token validates, the service constructs a principal:

- `sub` → user identifier.
- Custom claims → roles / scopes / tenant.
- Token claims (`acr`, `amr`, `auth_time`) → authentication context.

The principal is then made available to handlers (in Spring: `@AuthenticationPrincipal`; the equivalent abstraction in any framework).

Service-to-service

When service A calls service B, A presents *its own* token (client credentials flow) or *the user's* token, propagating via OAuth 2.0 Token Exchange (RFC 8693) or simple forwarding. The choice depends on whether service B should authorize based on the user or based on service A's identity.

Both forms are commonly supported:

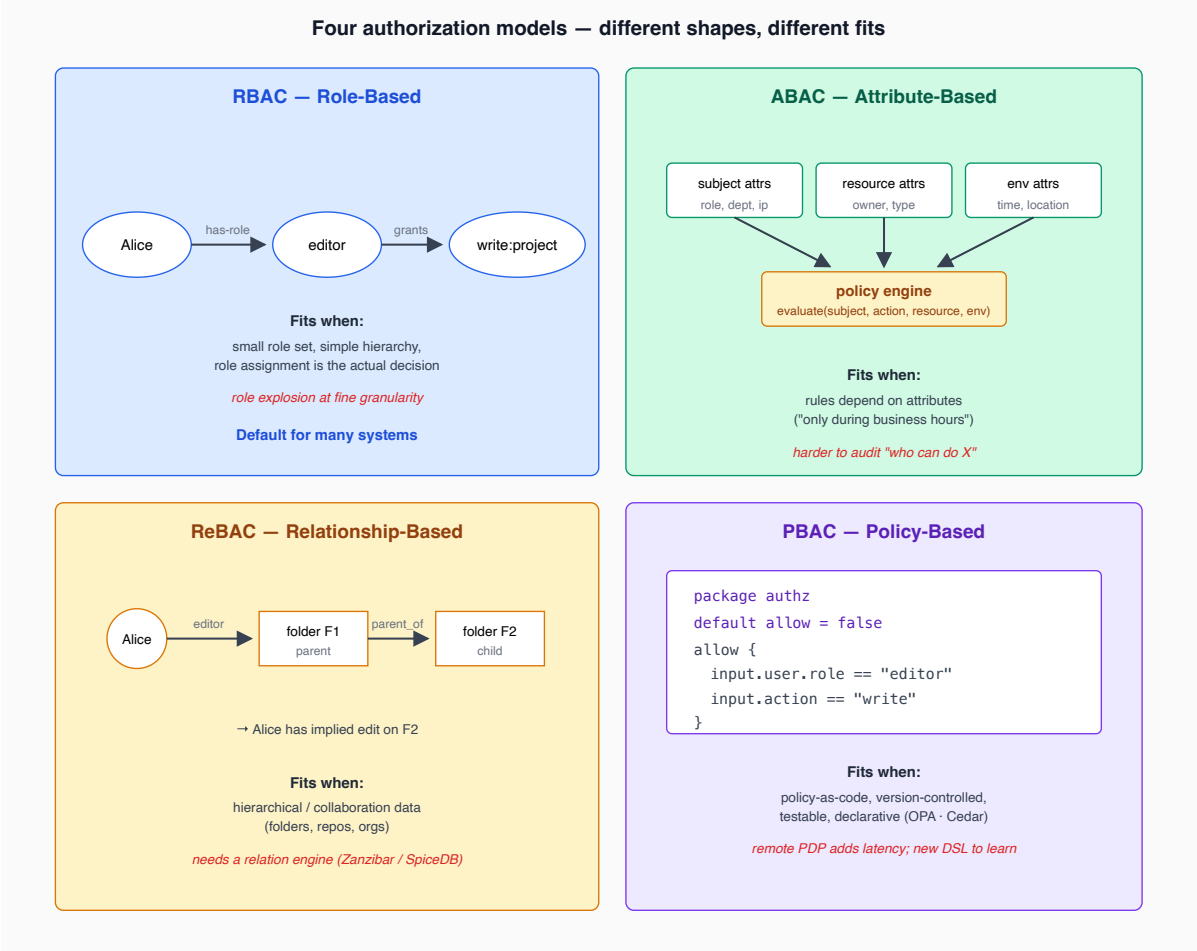
- **User-token forward** for read-on-behalf-of flows.
- **Service token** (via an `X-Service-Token` header or equivalent) for backend-only operations.

References

- IETF RFC 9068 *JWT Profile for OAuth 2.0 Access Tokens*.
- IETF RFC 8693 *OAuth 2.0 Token Exchange*.

---

41. Authorization models



RBAC — Role-Based Access Control

Subjects are assigned roles (`admin` , `editor` , `viewer`); roles are granted permissions; permissions check on resources. Simple, well-understood, sufficient for many systems.

Strengths: simple mental model, easy to audit ("who has admin?"), well-supported in every framework. Weaknesses: role explosion in fine-grained settings ("editor for project X but viewer for project Y" leads to per-project roles); hard to express attribute-based rules ("only during business hours from corporate IPs").

ABAC — Attribute-Based Access Control

Decisions are computed from attributes of the subject, resource, action, and environment. Express "the doctor may read the chart only if they are assigned to the patient and within the hospital network".

Strengths: flexible, expressive, central policy. Weaknesses: harder to audit ("who can do X?" is a query over policies, not a lookup), more complex tooling.

ReBAC — Relationship-Based Access Control

Inspired by Google's Zanzibar paper. Decisions are graph queries: "Alice is editor of folder F1; folder F1 is parent of folder F2; therefore Alice has edit on F2". Stored as `(subject, relation, object)` tuples.

Strengths: natural fit for hierarchical / collaboration data (drives, repos, organizations); efficient at scale. Weaknesses: requires a relationship engine (SpiceDB, OpenFGA); harder to express attribute-based rules.

PBAC — Policy-Based Access Control

A declarative policy language (OPA/Rego, AWS Cedar) evaluates allow/deny based on the request. Often the *engine* under ABAC.

Strengths: policies as code, version-controlled, testable. Weaknesses: a new language to learn; remote PDP adds latency.

Recommendation

RBAC + scope-based authorization is the right default for most consumers. For complex deployments, expose hooks so a downstream service can plug in a PDP (OPA, SpiceDB) without rewriting the auth layer.

References

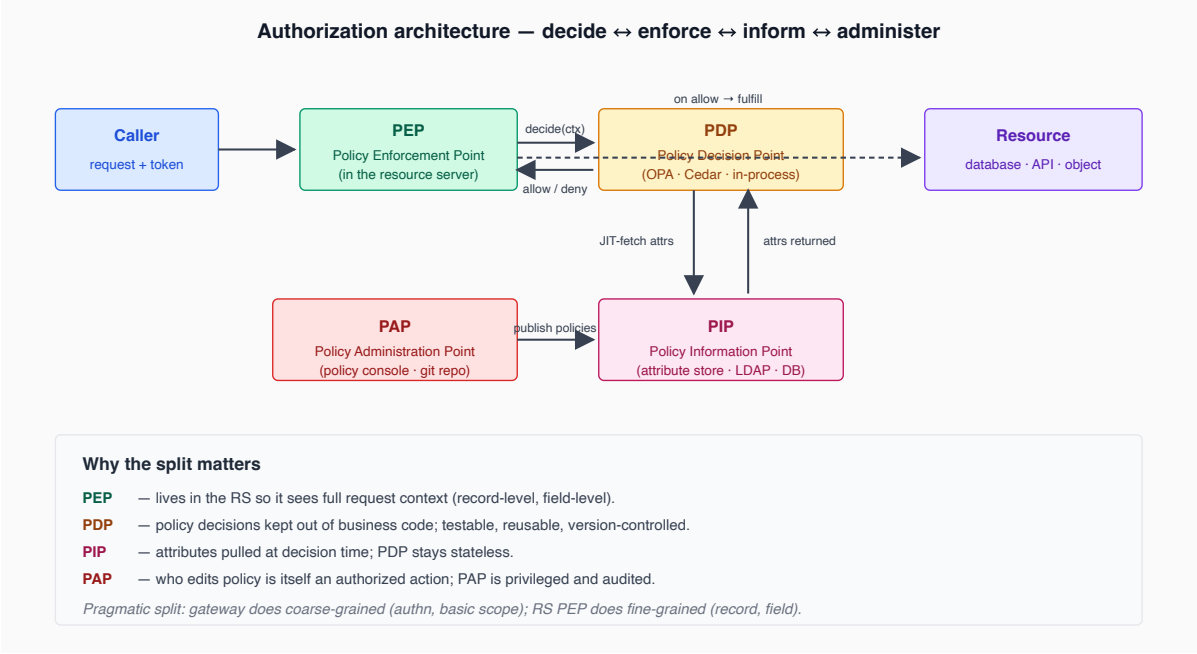
- NIST SP 800-162 *ABAC Considerations*.
- *Zanzibar: Google's Consistent, Global Authorization System* (USENIX 2019).
- *Open Policy Agent* documentation; *AWS Cedar* documentation.

---

42. PDP / PEP / PIP / PAP

XACML's vocabulary is widely adopted and applies regardless of whether you actually use XACML:

- **PEP — Policy Enforcement Point.** Where the decision is *applied* (the API endpoint, the controller, the gateway).
- **PDP — Policy Decision Point.** Where the decision is *computed* (in-process logic, OPA, Cedar engine).
- **PIP — Policy Information Point.** Where the PDP gets data it needs (database, attribute store, JIT-fetched user attributes).
- **PAP — Policy Administration Point.** Where policies are *authored* (a policy console, a Git repo).



Why enforcement belongs in the resource server

A common architecture mistake is to put all enforcement at the gateway. The argument seems persuasive: centralize, simplify the services. The argument fails because:

- The gateway speaks HTTP; many resources have richer semantics (record-level, field-level) the gateway cannot see.
- If a service is ever called *not* through the gateway (internal traffic, cron, debug), enforcement is bypassed.
- The gateway becomes a bottleneck for evolution: adding new policies requires gateway changes.

Pragmatic split: the gateway does coarse-grained decisions (authn, basic scope check); the service does fine-grained decisions (record-level, business rules).

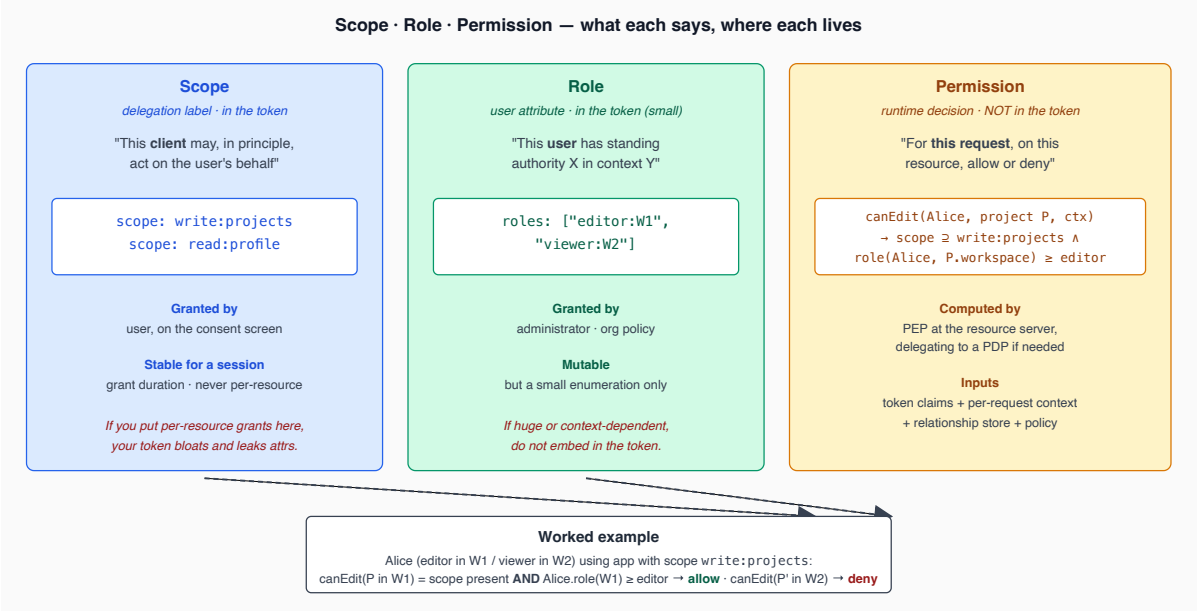
References

- OASIS *XACML 3.0*.

---

43. Scope vs role vs permission

Three different abstractions, frequently conflated. Conflation causes either over-sharing (too much in the token) or underspecification (decisions hidden in opaque runtime code).



The token carries scope and (small) role. The PDP at the resource server computes the actual permission per request from those claims plus context. Conflate scope with role and the token bloats; ignore scope and a malicious client can act outside what the user authorized.

---

44. Accounting / audit

The often-deferred third pillar. Unlike authn and authz, an audit-log gap is invisible until the day you need it.

What to log, schema, and tamper-evidence

Privacy

Audit events contain personal data and must be governed by the same data-protection regime as the rest of the user data. Apply pseudonymization on user deletion (§32).

References

- *CloudEvents Specification*.
- *OpenSSF Security Events*.
- ISO/IEC 27001 §A.12.4 (Logging and monitoring).

---

45. Service-to-service authorization

Inside a microservice mesh, services authenticate to each other and authorize their requests. Two patterns:

Workload identity

Each service has its own identity, separate from any user. Industry standard: SPIFFE (Secure Production Identity Framework for Everyone) and its workload-identity document, the SPIFFE Verifiable Identity Document (SVID). SVIDs come in two flavors:

- **X.509-SVID** — an X.509 certificate, presented at the TLS layer (mTLS).
- **JWT-SVID** — a JWT presented at the application layer.

Service tokens issued by an AS are conceptually JWT-SVIDs: short-lived JWTs, audience-bound to the receiving service, scope-restricted, signed by the AS.

Audience binding

Service tokens MUST be audience-bound to the *specific* recipient. A token good for every internal service is a privilege-escalation grenade waiting to drop. Resource indicators (RFC 8707) are the right mechanism.

Short-lived service tokens vs static API keys

Static API keys are the historical pattern: long-lived, copy-pasted into config files, rotated rarely (if ever). They fail because:

- Leaks are catastrophic and undetectable.
- Rotation requires coordinated config updates.
- They cannot be audience-bound, scope-limited, or sender-constrained.

Short-lived service tokens fix all three: minted on demand, expire in minutes, narrowly scoped, and (with sender-constraint) bound to the calling workload's identity.

References

- *SPIFFE / SPIRE* (Secure Production Identity Framework for Everyone).
- IETF RFC 8693 *OAuth 2.0 Token Exchange*.

Part X — Trust Distribution & Token Lifecycle

46. JWKS endpoint design

The JWKS endpoint is the trust boundary between the AS and the rest of the world. Every consumer fetches it; every cache decision affects security and operability. Design rules:

Cache-Control

Set `Cache-Control: max-age=N` proportional to your rotation cadence. Three considerations balance:

- **Too long** — rotation propagates slowly; a leaked key is honored for longer.
- **Too short** — JWKS endpoint becomes a hot path; outage of the AS becomes outage of every RS.
- **Just right** — typically 1 hour for high-tempo deployments, 6–24 hours for low-tempo.

`kid`-miss behavior

When an RS sees a token with a `kid` it does not recognize:

1. The cached JWKS may be stale (rotation just happened).
2. The token is forged.
3. The signer is misconfigured.

The recommended response: **revalidate the JWKS exactly once, then retry validation**. If it still fails, reject the token. Limit revalidation rate to defend against `kid`-miss-as-DoS.

Per-tenant filtering

Multi-tenant deployments may want each tenant to see only its own keys; an `?appId=...` filter on the JWKS endpoint is one way to do this. The trade-off: a compromised tenant configuration cannot be used to validate tokens for another tenant.

Offline mode

For air-gapped deployments, the JWKS is shipped as a file (`classpath:/.well-known/jwks.json`). Pros: zero hot-path AS dependency. Cons: rotation requires a redeploy.

Threat model

Attack	Mitigation
JWKS spoofing	Always TLS; HSTS; consider cert pinning for very-high-assurance.
Stale-cache-as-attack	TTL + `kid` -miss revalidate.
`kid` injection	Resolve `kid` only against the cached JWKS; never as a path / URL.
JWKS endpoint as DoS amplifier	Cache aggressively at CDN / reverse proxy.

References

- IETF RFC 7517 *JWK*.
- IETF RFC 8615 *Well-Known URIs*.

---

47. Key rotation strategy

Keys are not "set and forget". They are operational artifacts with lifecycles. The recommended rotation procedure:

1. **Generate next key.** New keypair, fresh `kid` . Private key in HSM/KMS; public key prepared for publishing.
2. **Publish overlap.** Add the new public key to the JWKS *before* using it for signing. RSs cache the JWKS (with TTL) and pick up the new key.
3. **Cut over signing.** AS starts signing with the new key. Old key remains in JWKS for verification of in-flight tokens.
4. **Retire old.** After tokens signed with the old key have expired (max access-token TTL plus margin), remove the old key from the JWKS.

Cadence:

- **Routine rotation:** quarterly to annually for signing keys. More frequent for content-encryption keys.

- **Emergency rotation:** any time of suspected compromise. Skip the overlap if the leak is confirmed; add it back briefly only if needed for active sessions to drain.

What to do when a private signing key leaks

This is the incident-response drill. Steps, in order:

1. **Confirm the leak.** Logs, threat-intel feed, customer report.
2. **Mint a new key.** Begin signing with it immediately.
3. **Remove the leaked key from the JWKS.** This invalidates every token signed with it — including legitimate ones in flight. This is the cost; sessions will need to re-auth.
4. **Force a re-auth on every active session.** Revocation event broadcast to every RS (or natural failure as the JWKS no longer contains the key).
5. **Audit.** What did the attacker have access to? What tokens did they sign?
6. **Post-mortem.** How did the key leak? Tighten the secret-management story.

References

- NIST SP 800-57 *Recommendation for Key Management*.

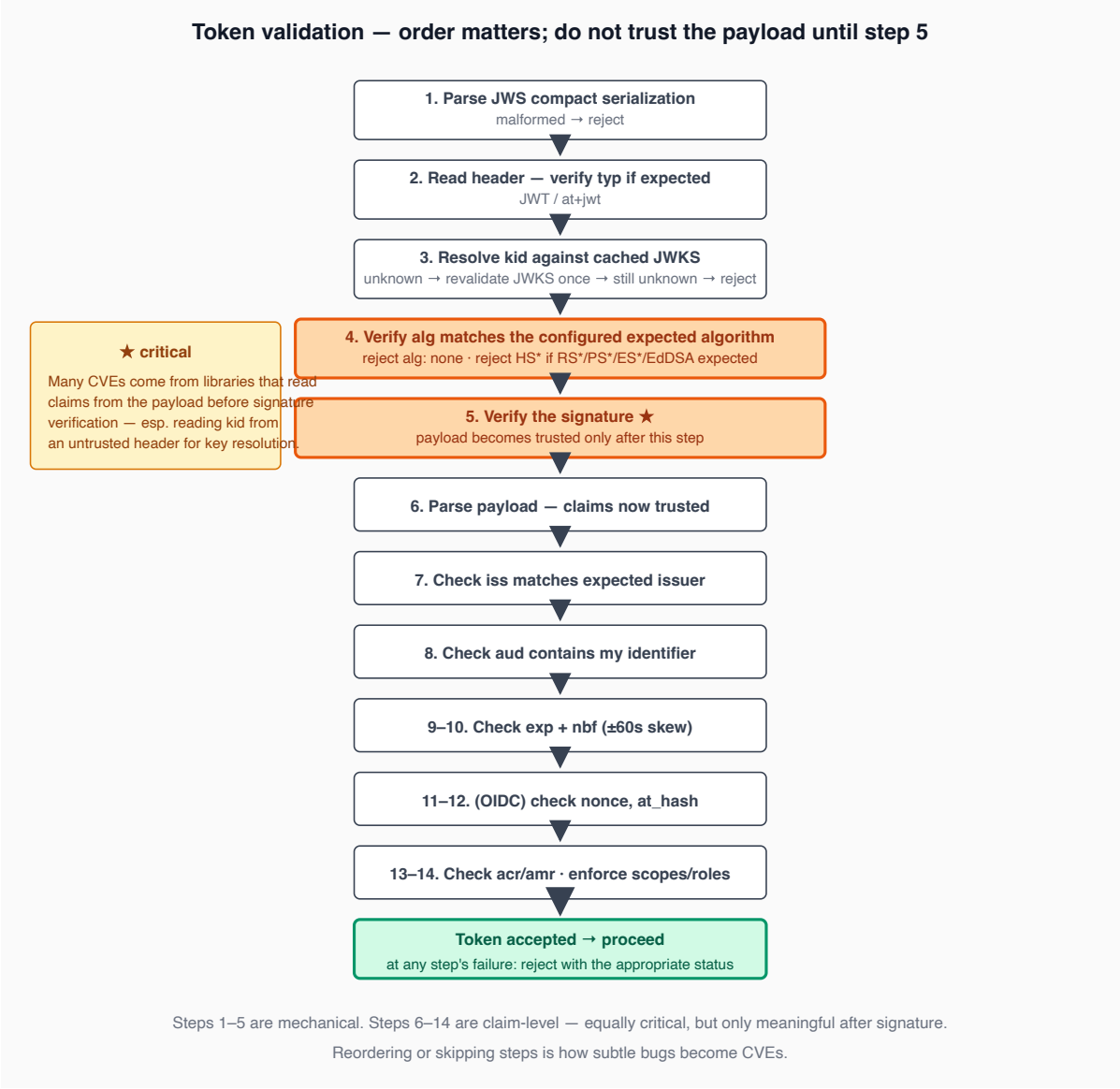
---

48. Token validation algorithm

A canonical, ordered procedure. Skipping or reordering steps is how subtle bugs become CVEs.

```
1. Parse the JWS compact serialization. Reject malformed input.
2. Read the header. Verify `typ` (when expected: `JWT` or `at+jwt`).
3. Resolve `kid` against the cached JWKS. Reject unknown `kid` (after one
revalidate).
4. Verify `alg` matches the configured expected algorithm. Reject
everything else.
 In particular: reject `alg: none`, reject HS* if RS*/PS*/ES*/EdDSA is
expected.
5. Verify the signature.
6. Parse the payload. (Now and only now is the payload trusted.)
7. Check `iss` against the expected issuer. Reject mismatch.
8. Check `aud` contains this resource server's identifier. Reject mismatch.
9. Check `exp` is in the future (with optional clock skew, ≤ 60s).
10. Check `nbf` is in the past (if present).
11. (OIDC) Check `nonce` matches expected value.
12. (OIDC) Check `at_hash` matches the access token (if validating an ID
token).
13. Check `acr` / `amr` against the action's requirements.
14. Enforce scopes / roles for the action.
```

do not trust the payload until after step 5. Many vulnerabilities have come from libraries that read claims (especially `kid`) from the payload before signature verification, opening avenues for tampering.



Steps 1–5 are mechanical. Steps 6–14 are claim-level — equally critical, but only meaningful after signature.
Reordering or skipping steps is how subtle bugs become CVEs.

Clock skew

Clocks drift. A common policy: allow up to 60 seconds of skew on `exp` and `nbf`. Larger skew weakens token expiry; smaller skew breaks legitimate users. NIST and most BCPs converge on 60 seconds.

References

- IETF RFC 7519 §7 *JWT Validation*.
- IETF RFC 8725 *JWT BCP* §3.

49. Token revocation

the standard pattern: **short access-token TTL + refresh on demand**. With a 5-minute access TTL, the maximum time a stolen token is useful is 5 minutes. Below that you cannot cost-effectively revoke; above that, refresh-token rotation defends against long-term theft.

Refresh-token revocation

Refresh tokens *must* be server-side state (the AS records each one). Revocation is straightforward — mark the row revoked. On next refresh attempt, fail.

Access-token revocation (JWT)

JWTs are stateless: any holder of a valid JWT can use it until `exp` regardless of what the AS thinks. Three options:

- **Live with short TTL.** Acceptable in most deployments.
- **Denylist.** AS publishes a list of revoked `jti`s; RSs check it. Works at modest scale; turns the AS into a hot dependency.
- **Switch to opaque tokens + introspection.** Each request to the RS includes a token-introspection call to the AS. RS gets ground truth on every request. Adds latency; couples RSs to AS uptime.

Recommendation: short TTL (5–15 minutes) plus refresh rotation. For deployments needing immediate revocation, use opaque tokens + introspection on a per-tenant basis.

Sign-out semantics

"Sign out everywhere" is a common UI affordance. Implementation:

1. Revoke all refresh tokens for the user.
2. Optionally maintain a `min-iat` per user — tokens issued before this timestamp are rejected. Distribute via a small cache that all RSs check.
3. Wait out the access-token TTL window for in-flight tokens.

References

- IETF RFC 7009 *Token Revocation*.
- IETF RFC 7662 *Token Introspection*.
- IETF RFC 9700 §4.14 (refresh-token replay detection).

Part XI — Sender-Constrained & Strong Auth

The next frontier above bearer tokens — an active development area for any modern AS.

50. DPoP — Demonstrating Proof-of-Possession

DPoP (RFC 9449) is the SPA-friendly sender-constraint mechanism. Per request, the client sends:

- An access token (in `Authorization: DPoP <token>`).
- A *DPoP proof* JWT in the `DPoP` header.

The proof JWT:

- Is signed by a key the client generated and never shared.
- Includes `htm` (HTTP method), `htu` (HTTP URL), `iat`, and `jti`.
- Is short-lived (seconds).

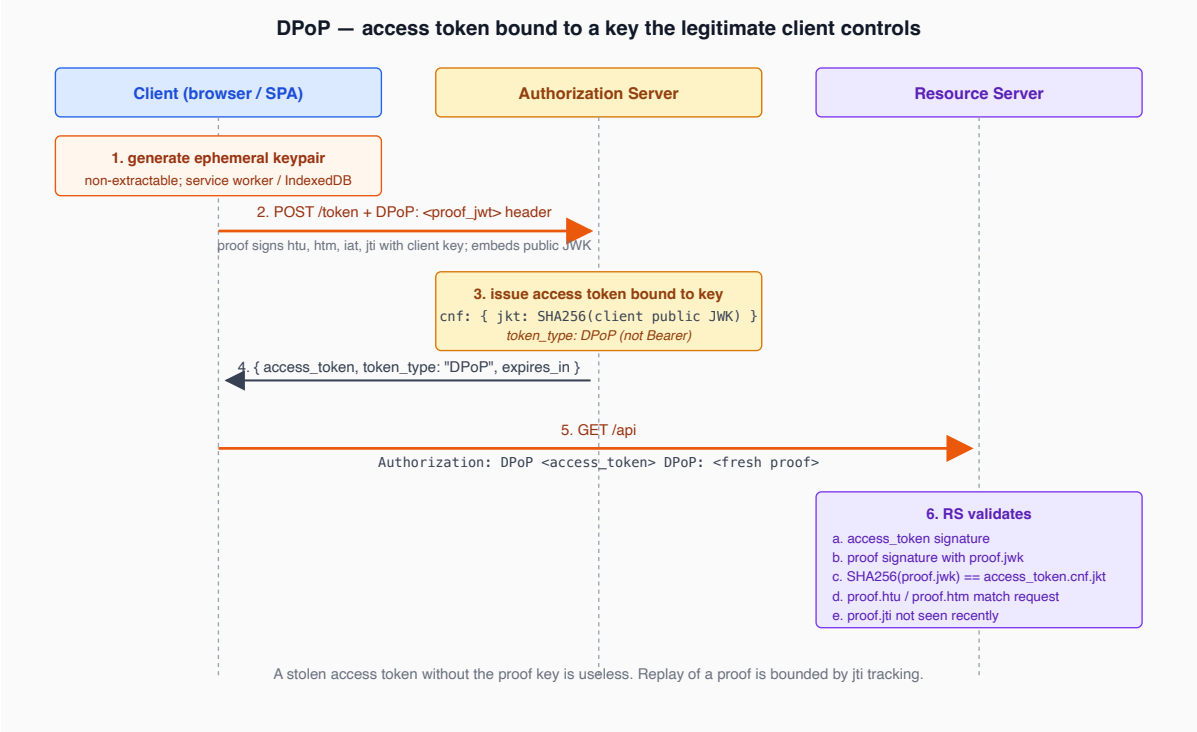
The access token, at issuance, is bound to the public-key thumbprint via the `cnf` claim:

```
{ "cnf": { "jkt": "<sha256 of client public JWK>" } }
```

The RS verifies:

1. The access token signature is valid.
2. The access token's `cnf.jkt` matches the thumbprint of the proof's signing key.
3. The proof's signature is valid (using the proof's `jwk` header).
4. The proof's `htm` / `htu` match this request.
5. The proof's `iat` is recent.
6. The proof's `jti` has not been seen recently (replay defense).

A stolen access token without the proof key is useless. Replay of a proof is bounded by `jti` tracking.



Where to store the DPoP key in a browser

The whole point fails if the key sits next to the access token in `localStorage`. Options:

- **`CryptoKey` with `extractable: false`** — the browser holds the key in user-agent memory; JS can use it for `sign()` but cannot read its bytes. Survives page reload via IndexedDB (still non-extractable).
- **Service worker** — keys live in the worker's scope, isolated from page scripts.
- **Best:** combine. Generate non-extractable keys, store in IndexedDB, perform sign operations in a service worker so the page never has a `CryptoKey` reference at all.

This is harder than localStorage. It is also the only way DPoP delivers its claimed protection in the SPA threat model.

References

- IETF RFC 9449 *DPoP*.

---

51. Mutual TLS (mTLS)

RFC 8705 specifies two flavors:

Certificate-bound access tokens (`tls_client_auth`)

The token endpoint binds the issued access token to the client's TLS certificate (the certificate's SHA-256 hash). The RS, also accepting mTLS connections, verifies the request's client cert hash matches the token's `cnf.x5t#S256`.

Heavy: every party — AS, RS, client — must run mTLS. Common in B2B and FAPI-aligned banking deployments. Inappropriate for browser clients (browsers do not expose client certs to JavaScript).

Self-signed certificate authentication (`self_signed_tls_client_auth`)

The client self-issues a certificate; the JWKS-equivalent (`jwks_uri` in client metadata) lists the certificate's hash. Useful when there is no PKI but the client can be enrolled with a known cert hash.

When mTLS is reasonable vs over-engineered

- **Reasonable:** B2B integrations, FAPI deployments, internal service-to-service over mTLS-mesh (Istio, Linkerd).
- **Over-engineered:** consumer SPAs (use DPoP), public APIs with diverse clients (use DPoP).

References

- IETF RFC 8705 *OAuth 2.0 Mutual-TLS*.

---

52. FAPI 2.0 profile

FAPI (Financial-grade API) is a profile by the OpenID Foundation for high-assurance use cases (open banking, healthcare). FAPI 2.0 mandates:

- OAuth 2.1 baseline.
- Sender-constrained tokens: DPoP or mTLS.
- PAR (RFC 9126).
- JAR (RFC 9101) — request objects.
- Signed responses (no plain JSON).
- Specific algorithm restrictions (PS256, ES256, EdDSA).
- Strong logout flows.

Most non-banking applications do not need FAPI. Banking and healthcare in regulated jurisdictions often do.

References

- *FAPI 2.0 Security Profile*.

---

53. PAR & JAR

The authorization request crosses the front channel — the user's browser. Anyone who can read the URL can read the request. PAR and JAR move the sensitive parts off the front channel.

PAR — Pushed Authorization Requests (RFC 9126)

The client POSTs the authorization request to a `/par` endpoint (back channel). The AS returns a `request_uri` opaque handle. The client redirects the browser with just `request_uri` and `client_id`. The browser carries an opaque reference, not the request contents.

Benefits:

- Request integrity — the AS knows the request came from the client (the back-channel POST is authenticated).
- Request confidentiality — PII / scopes / etc., not exposed in browser logs.
- Request size — long requests do not hit URL length limits.

JAR — JWT-Secured Authorization Request (RFC 9101)

The client packs the authorization request into a signed JWT and sends it in `request` (or `request_uri`). The AS verifies the JWT signature against the client's pre-registered key. Even if delivered front-channel, the request cannot be tampered.

PAR and JAR compose: a JAR delivered via PAR is the strongest form.

References

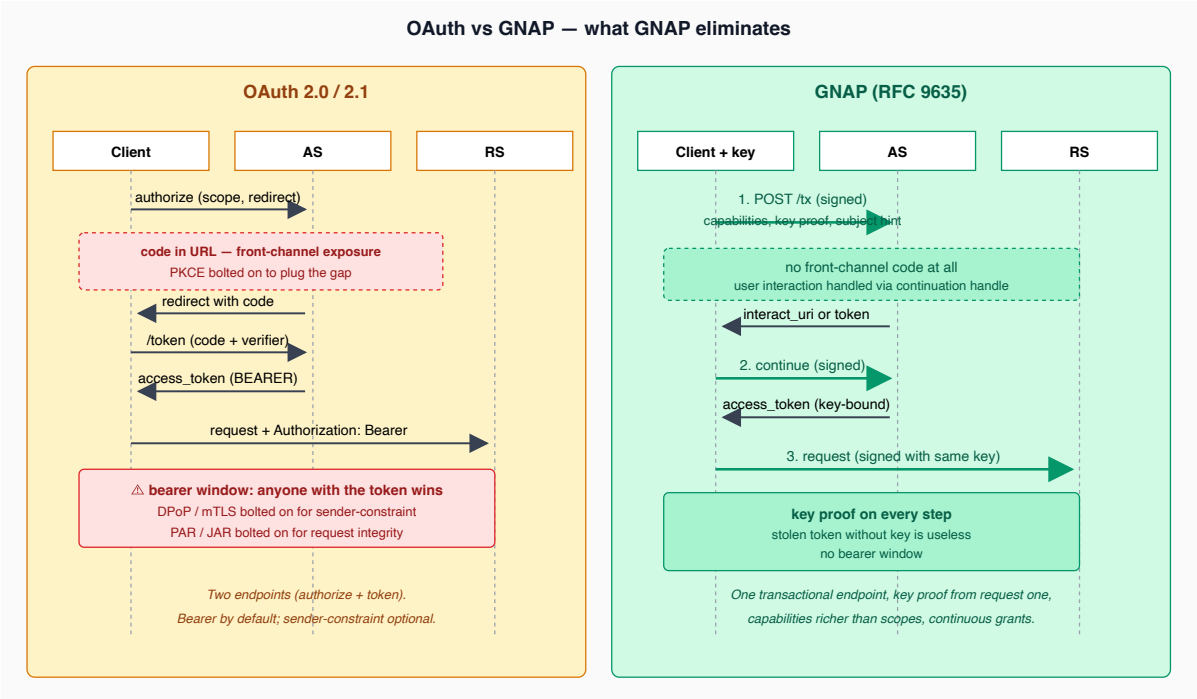
- IETF RFC 9126 *Pushed Authorization Requests*.
- IETF RFC 9101 *JWT-Secured Authorization Request*.

Part XII — Future-Looking Protocols

54. GNAP — Grant Negotiation and Authorization Protocol

OAuth 2.0 was designed in 2010, before mobile dominated, before smartphones had biometric authenticators, before machine identities outnumbered humans. GNAP (RFC 9635) is a clean-slate redesign by the IETF. Key differences:

- **Single-request grants.** OAuth uses an authorization request followed by a token request; GNAP combines them into one transactional negotiation.
- **Key proof from request one.** Every GNAP request is signed by a client-controlled key. No bearer-style window.
- **Capability requests.** Clients request *capabilities* (more expressive than scopes) — "I want to be able to read /accounts and write to /payments under $500".
- **Continuous grants.** A grant can be modified, extended, or queried after issuance via the continuation API.
- **Multi-instance clients.** A client running across many devices can be a single GNAP client; each instance proves itself with its own key.



Conceptual mapping

OAuth concept	GNAP equivalent
Client	Client + key
Resource owner	Subject
Scope	Access (capability)
Authorization code	Continuation reference
Access token	Token (now key-bound by default)
AS	AS (richer state machine)

Why it matters

GNAP solves real problems OAuth has accreted: bearer-token theft, scope inflation, fragmented standards (PKCE, DPoP, PAR, JAR all bolt onto OAuth). It is unlikely to displace OAuth in the next five years, but it is the natural successor.

Pragmatic stance: track the standard, do not implement until consumer libraries mature.

References

- IETF RFC 9635 *GNAP Core Protocol*.
- IETF RFC 9636 *GNAP for Resource Servers*.

---

55. Verifiable Credentials & Self-Sovereign Identity

A different model. Instead of the AS being the source of truth, the *user* holds cryptographically-verifiable credentials issued by various authorities. The user presents the credential to a verifier directly.

The roles:

- **Holder** — the user, in their wallet.
- **Issuer** — the authority that originally vouched (university, government, bank).
- **Verifier** — the relying party.

The wallet holds W3C Verifiable Credentials (VCs). DIDs (Decentralized Identifiers) replace `iss` and `sub` with cryptographic identifiers not tied to any one provider.

OIDC has bridged into VC space via:

- **OIDC4VCI** — OpenID for Verifiable Credential Issuance.
- **OIDC4VP** — OpenID for Verifiable Presentations.

Where this fits a modern AS

Not central to most identity systems' near-term roadmaps, but the OIDC bridges are the most plausible path to interop. Track for selective enablement (e.g., issuing a VC of "verified employee at Org X" that can be presented elsewhere).

References

- W3C *Verifiable Credentials Data Model 2.0*.
- W3C *Decentralized Identifiers (DIDs) v1.0*.
- *OIDC4VCI | OIDC4VP*.

---

56. CAEP & SSF — out-of-band session signals

Continuous Access Evaluation Profile (CAEP) and Shared Signals Framework (SSF) are OpenID Foundation specs for *push-based* identity events between systems. Use cases:

- An employee is terminated at the IdP. CAEP pushes a "session-revoke" signal to every RP currently relying on the IdP. RPs revoke local sessions immediately, without waiting for the access-token TTL.
- A device is reported stolen. CAEP pushes "device-claims-changed".
- Risk score crosses a threshold. CAEP pushes "credential-change-required".

SSF is the transport (subscriber model, signed JSON Web Tokens carrying events). CAEP is one profile of events on SSF.

Recommendation: implement SSF *receiver* (to react to social-IdP signals) before SSF *transmitter*.

References

- *OpenID Continuous Access Evaluation Profile (CAEP)*.
- *OpenID Shared Signals Framework*.

Part XIII — Operational Security

57. The OAuth threat catalog

RFC 6819 (2013) and RFC 9700 (2024) catalog every known OAuth attack. The greatest hits:

OAuth threat catalog — what attackers try, what stops them

Attack	Vector	Defense
Redirect URI manipulation CWE-601	typo squat · open redirector · substring match weakness	exact-string redirect_uri allowlist; no prefix · no wildcard · no substring
Authorization code injection RFC 9700 §4.5	attacker injects stolen code into victim's session	PKCE (verifier ties code to client session) + state (binds response to request)
Mix-up attack multi-IdP clients	code from IdP A submitted to IdP B's token endpoint	iss in authorization response (RFC 9207); client tracks expected IdP per request
IdP confusion malicious provider	attacker IdP forwards attacker-controlled token	strict IdP allowlist · iss validation; UI clearly shows user which IdP
Refresh token theft long-lived credential	XSS · log scrape · backup leak	rotation (RFC 9700 §4.14) + replay-detect → revoke session · sender-constrain (DPoP/mTLS)
Token leakage side channels	URL → Referer → logs XSS exfil · browser cache	never tokens in URL · HttpOnly cookies · CSP; never log Authorization headers
CSRF on state CWE-352	attacker initiates auth flow on victim's browser	state parameter · CSPRNG-generated · bound to user session · validated on response
JWT algorithm confusion CVE-2015-9235 et al.	alg: none accepted · or HS256 forged using public-key bytes	pin algorithm in verifier · reject alg: none; type-tag keys with their algorithm family

Catalog drawn from RFC 6819 (Threat Model) and RFC 9700 (Security BCP). Each defense is a positive control; most attacks have multiple defenses applicable — defense in depth is the strategy.

No single layer suffices. Combine PKCE + state + sender-constraint + exact-match + CSP + audit.

Redirect URI manipulation

The AS sends the code to a `redirect_uri` controlled by the attacker. Vectors: typo squatting (registered redirect URI is overly broad), open redirector chained off the legitimate site, partial-string match weakness.

Defense: exact-string `redirect_uri` matching against a pre-registered list. No prefix, no wildcard, no substring.

Authorization code injection

The attacker obtains a code somehow (XSS, malicious app, MITM) and injects it into a victim's session.

Defense: PKCE (the verifier ties the code to the requesting client's session) plus `state` (the auth response is bound to the original auth request).

Mix-up attack

A multi-IdP client receives a code from IdP A and mistakenly submits it to IdP B. Possible when the client doesn't track which IdP each in-flight request was for.

Defense: include the issuer (`iss`) in the authorization response (RFC 9207); client verifies on receipt.

IdP confusion

A malicious IdP is registered alongside legitimate ones; the user is induced to authenticate via the malicious one, which forwards an attacker-controlled token.

Defense: strict IdP allowlist; `iss` validation; user UI shows which IdP they are signing in to.

Refresh-token theft

The refresh token is long-lived for a long-lived session.

Defense: rotation (RFC 9700 §4.14), sender-constraint (DPoP, mTLS), short-TTL access tokens.

Token leakage

Tokens leak via:

- HTTP referrer headers (token in URL).
- Browser history.
- Server access logs.
- Caches and proxies.
- XSS exfiltration.

Defense: never put tokens in URLs (always headers or POST bodies); strict CSP; HttpOnly cookies for sessions; never log Authorization headers.

CSRF on `state`

Attacker initiates an auth flow on the victim's browser. Without `state` , the victim's session ends up authenticated as the attacker's account.

Defense: `state` parameter, CSPRNG-generated, bound to the user's session, validated on response.

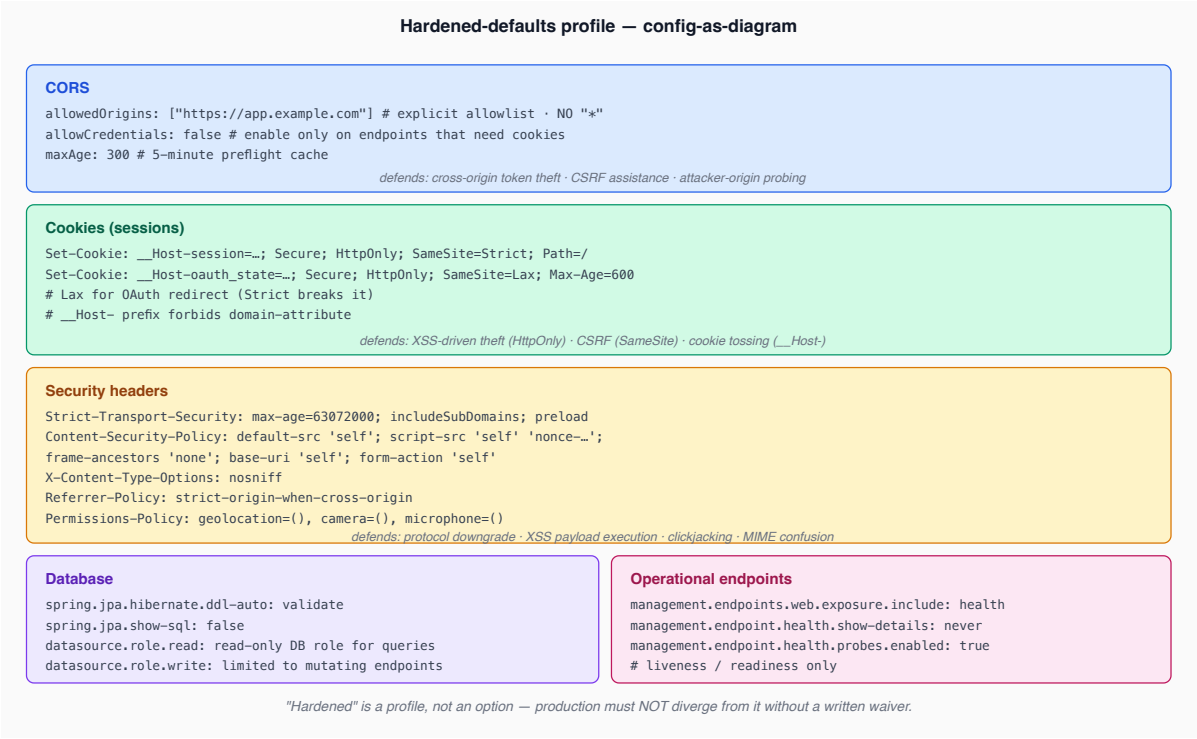
References

- IETF RFC 6819 *OAuth 2.0 Threat Model*.
- IETF RFC 9700 *OAuth 2.0 Security BCP*.
- IETF RFC 9207 *OAuth 2.0 Authorization Server Issuer Identification*.

---

58. Hardened defaults

Ship a hardened-defaults profile distinct from the dev profile (the OneAuth repo example provides `application-security-hardened.yml`). The full picture:



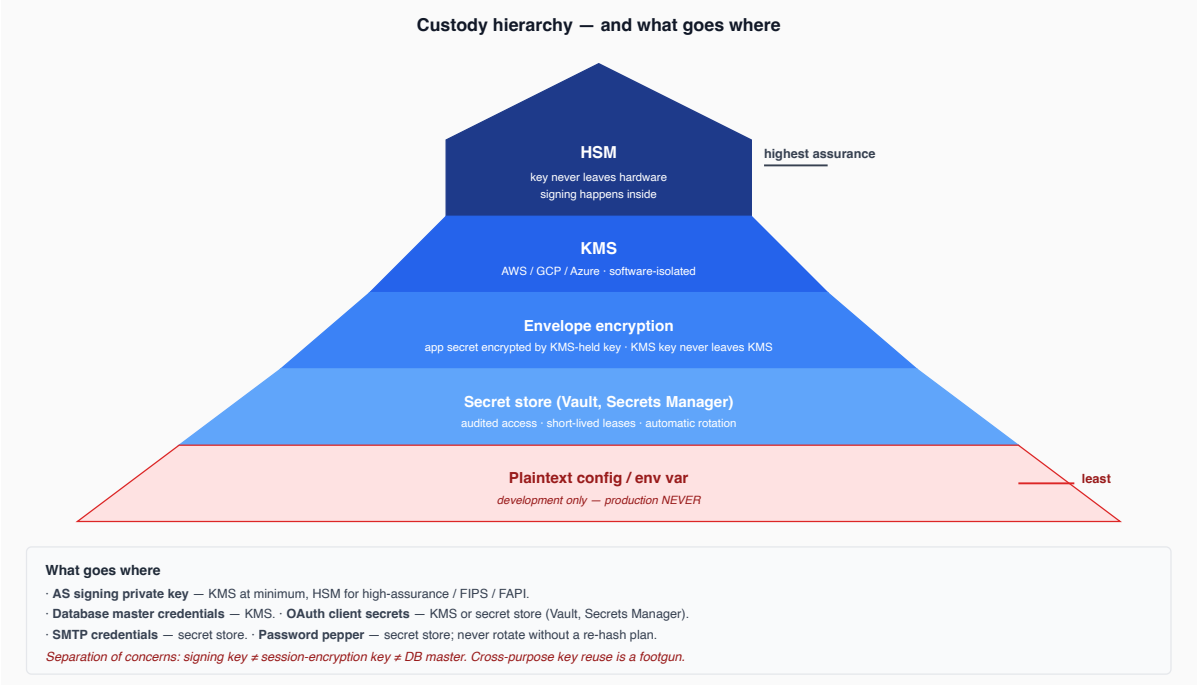
References

- OWASP *Secure Headers Project*.
- IETF RFC 6797 *HSTS*.

---

59. Secret & key custody

Secrets are the load-bearing parts of the system. Custody is non-negotiable.



Rotation

- Application secrets: quarterly.
- Signing keys: per the rotation procedure (§47).
- Pepper: rotation requires re-hashing on next user login (lazy migration); plan accordingly.

References

- NIST SP 800-57 *Key Management*.
- *AWS KMS / GCP KMS / Azure Key Vault* documentation.

---

60. Rate limiting & abuse

Authentication endpoints are abuse magnets. Rate limiting is not optional. Each endpoint has a different shape:

Endpoint	Per-IP limit	Per-account limit	Per-(IP, account) limit
Login	30 / minute	10 / 15 minutes	5 / 15 minutes
Registration	5 / minute	— (no account yet)	—
Password reset request	5 / hour	3 / hour	—
Verification email resend	10 / hour	3 / hour	—
MFA OTP attempt	30 / minute	5 / 15 minutes	5 / 15 minutes

Endpoint	Per-IP limit	Per-account limit	Per-(IP, account) limit
Token refresh	100 / minute	60 / minute	—

Sliding window vs token bucket

- **Sliding window** — count requests in the last N seconds. Accurate but memory-heavy.
- **Token bucket** — tokens replenish at rate R; each request consumes one. Less memory, allows controlled burst.
- **Fixed window** — count requests per calendar minute. Cheap, allows burst at boundary.

Recommendation: token bucket for steady-state shaping plus a fixed cap.

Combining limit dimensions

Per-IP defends against single-host attacks; per-account defends against distributed attacks; per-(IP, account) defends against either. **All three** are needed:

- Per-IP only: an attacker with a botnet bypasses.
- Per-account only: an attacker bombing one account from many IPs bypasses for legit users on that account (account-DoS); switch the legit user out from under themselves.
- Per-(IP, account) only: the first probe against an account-IP pair has no history; cannot block the first request.

Lockout strategy

Hard lockouts (after N failures, account is locked for T minutes) defend against guessing but enable account-DoS:

```
attacker makes 10 wrong logins for victim → account locked for 15 minutes
victim cannot log in for 15 minutes → DoS
```

Mitigations:

- Soft lockout: increase friction (CAPTCHA, slow down) but never *block* a correct password.
- Lock per-(IP, account) only: locking one IP from the account does not lock the legitimate user's IP.
- Reset on successful auth: the threshold is for *guessing*, not for legitimate retries.

References

- OWASP *Rate Limiting / Brute-Force Protection*.

61. Logging, monitoring, IR

The accounting layer's operational counterpart.

Baseline alerts

- Login failures spike (per-account, per-IP, global).
- New registrations spike.
- Password reset requests spike.
- Successful logins from impossible-travel IPs.
- Privileged-action audit events without preceding step-up.
- 5xx rate from auth endpoints.
- JWKS endpoint latency or error rate.

Impossible-travel

User logs in from San Francisco, then 30 minutes later from Lagos. Either VPN, or compromise. Surface, do not block. Block on combination with other signals (new device, sensitive operation).

Incident-response runbook: signing key leak

the incident-response drill from Part X §47, written down:

```
0. Confirm leak (logs, threat-intel, customer report)
1. Generate new signing key in KMS
2. Begin signing with new kid
3. Remove leaked kid from JWKS publishing ← every token signed with it now
invalid
4. Force re-auth on every active session (sessions broken by step 3
anyway)
5. Audit: what tokens were signed with the leaked key? what did the holder
access?
6. Customer notification per regulatory regime
7. Post-mortem within 7 days
```

References

- NIST SP 800-61 *Computer Security Incident Handling Guide*.

Part XIV — Putting it together: the OneAuth reference architecture

62. Layer map

Where each protocol layer lives in the OneAuth repo example's components:

Layer	Component
JOSE primitives	`one-auth-jar` (validation), `one-auth-app` (issuance)
OAuth 2.0 / 2.1	`one-auth-app` (AS)
OIDC	`one-auth-app` (OP)
Social federation	`one-auth-app` (RP, then re-issues local tokens)
User lifecycle	`one-auth-app`
MFA / passwordless	`one-auth-app` (planned)
AAAF authn (RS-side)	`one-auth-jar` (filter, principal hydration)
AAAF authz (RS-side)	`one-auth-jar` (scope/role checks); downstream services for fine-grained
Accounting	downstream pipeline (planned)
JWKS publication	`one-auth-app`
Token validation (RS)	`one-auth-jar`
Sender-constraint	TBD (DPoP first, planned)
Frontend integration	`one-auth-ui` SDK (`@oneauth/react` , `@oneauth/nextjs`)

63. Build order

A complete OneAuth-class system, ordered the way you would actually build it. Each phase depends on the previous one — wiring later items first leaves the foundation incomplete and forces rework. The closing tenets in §64 are the rules every step rolls up to.

Phase 1 — Hardened foundations

The crypto, key custody, and HTTP-level defaults that everything else trusts. Cutting corners here means re-issuing every token later.

1. **Hardened-defaults profile.** CORS allowlist, cookie flags (`HttpOnly`, `Secure`, `SameSite`), security headers (CSP, HSTS, X-Frame-Options, Referrer-Policy), restricted operational endpoints. The profile ships *before* the first real endpoint — bolting it on later is how regressions land in production.
2. **JWS signing keys.** RS256 as the today-default, ES256 / EdDSA available for new deployments (4× smaller tokens, faster verify). Private keys live in KMS / HSM; never on disk. One signing alg pinned per verifier — no negotiation.
3. **JWKS endpoint, per-appId.** `/.well-known/jwks.json` published per audience with cache headers. Offline mode for air-gapped consumers. `kid` discipline: signer always sets it, verifier treats it as a hint and resolves only against the trusted JWKS.
4. **Argon2id password storage.** Production-tuned memory / time / parallelism cost; per-user salt; pepper from KMS. On any later migration from a weaker hash, lazy re-hash on next successful login — never bulk-rehash with the cleartext you don't have.
5. **Rate limiting on auth endpoints.** Per-IP, per-account, and per-(IP, account) — all three; any one alone is bypassable. Soft lockout (CAPTCHA, slowdown), never block a correct password. Enforcement in code, not just config.

Phase 2 — User lifecycle

The minimum surface to register, verify, and recover an account safely.

6. **Email verification (code-based).** Six-digit code, session-bound, 10-minute TTL, resend throttle. Code-based survives link pre-fetchers, corporate scanners, and Referer leaks — all of which silently consume one-click links.
7. **Password reset.** Single-use, time-bound token; all active sessions invalidated on reset. Symmetric responses on both reset request and reset completion — no enumeration oracle.
8. **Breached-password check.** HIBP k-anonymity (range API, never the full hash) at registration and password change. Reject prior-breached candidates before the credential ever exists in the system.

Phase 3 — OAuth 2.1 core

The grants and token-handling rules that turn the AS into something a real client can integrate against.

9. **Authorization Code + PKCE (S256).** PKCE mandatory for every client — public *and* confidential. The verifier never leaves the legitimate client; the AS rejects redemption without it.

10. **Strict OAuth 2.1 redirect_uri enforcement.** Exact-string match. No prefix, no wildcard, no path glob. Every registered URI reviewed; substring matching is how attackers redirect codes to evil.com.
11. **Refresh-token rotation + replay-detect revocation.** Every refresh exchange issues a new opaque token; reuse of the prior one revokes the entire chain. Storage rules differ per client type: confidential → secret store; browser → HttpOnly cookie set by a BFF, never `localStorage`.
12. **Client credentials grant.** Three client-auth options offered — HTTP Basic, JWT bearer assertion (RFC 7523), mTLS (RFC 8705). Pick by deployment fit; document the trade-offs.
13. **Service tokens — audience-bound.** RFC 8707 `resource` indicator on every S2S exchange. One token per audience; multi-audience tokens are an exfiltration vector.

Phase 4 — OIDC

Federated-trust shape on top of OAuth.

14. **OIDC ID token issuance.** Standard claims: `iss`, `sub`, `aud`, `exp`, `iat`, `nonce`, plus `acr` / `amr` / `azp` / `at_hash` once the strong-auth phase lights them up. Validation order is non-negotiable: header check → signature → `iss` / `aud` → time → `nonce` → claim-level checks.
15. **OIDC discovery — `/.well-known/openid-configuration`.** Full metadata: endpoints, supported alg, scopes, claims, response types. Clients should drive integration off this document, not hard-coded URLs.
16. **OIDC UserInfo endpoint.** Bearer-protected `/userinfo`; returns claims gated by granted scope (`openid`, `profile`, `email`). The standard equivalent of any local "current user" endpoint.

Phase 5 — Federation

External identity providers, with a local re-issue boundary.

17. **Social federation: Google, GitHub, Facebook.** The local AS acts as RP; after federated sign-on the local AS re-issues local tokens — RSes never see the upstream IdP's signature. Account-linking step-up gate: linking a new IdP to an existing account requires a fresh authentication, not just a session cookie.

Phase 6 — Authorization & accounting

Decisions about *who can do what*, and the durable record of every decision.

18. **RBAC — scope + role.** Scope is token-bound (delegated authority); role is subject-bound (inherent authority). PEP at the resource server; PDP local for first-party.

Centralize the model now — retrofitting authority semantics is more painful than retrofitting authentication.

19. **Structured audit-event pipeline.** CloudEvents-style schema, tamper-evident storage. Required events: login success/fail, MFA enroll, consent change, privileged action, key rotation. Equal priority with the API surface — done after the fact, never done well.

Phase 7 — Strong authentication

Layered on top once the rest is stable; the path is *additive* toward passkey-only.

20. **MFA: TOTP (RFC 6238).** Provisioning URI, recovery codes, anti-replay window (one code, one use, ±1 step skew).
21. **MFA: WebAuthn / passkeys.** Phishing-resistant; resident keys preferred. The end-state is passkey-only sign-in — every choice in this phase keeps that path open.
22. **Step-up authentication via `acr` / `amr`.** Sensitive operations (move money, change recovery email, rotate API keys) require a recent `amr` matching the required `acr`. Drives the AAL ladder (NIST SP 800-63B) in concrete terms.

Phase 8 — Sender-constraint and federated trust

Bearer is the universal default and the universal foot-gun. This phase narrows the blast radius of a leaked token, in priority order by where the leaks actually happen.

23. **JWE for sensitive payloads.** Nested JWS-in-JWE for tokens carrying confidential claims. Key wrap: ECDH-ES+A256KW (preferred) or RSA-OAEP-256. Content encryption: A256GCM.
24. **DPoP — browser clients first.** Per-request proof bound to access token via `cnf.jkt`; replay defense via `jti` + `htm` + `htu` + `iat` window. Browser is highest leak surface, so DPoP lands here before mTLS.
25. **mTLS — B2B service tokens.** Client certificate identity bound to `cnf.x5t#S256` (RFC 8705). Suits high-stakes server-to-server where a PKI already exists.
26. **SSF receiver for federated session-revoke.** Consume CAEP / Shared Signals events from upstream IdPs — when a federated identity is revoked or the session is killed there, propagate locally instead of waiting for the next bearer-token expiry.
27. **GNAP exploration.** Track upstream library maturity (RFC 9635 / RFC 9636); pilot for high-stakes flows where the bearer-token window is unacceptable. Keep the AS architecturally separable so a parallel GNAP grant path can land without rewriting OAuth.

64. Closing — design tenets

The non-negotiables. These tenets are what every design decision in this book rolls up to.

1. **Standards-first.** If a problem is solved by an IETF/OIDF spec, follow the spec. Custom protocols are a last resort and a maintenance burden.
2. **Offline verification.** Resource servers do not depend on the AS being reachable on the hot path. JWKS caching with sound rotation is the load-bearing decision.
3. **Sender-constrained where stakes are high.** Bearer is the universal default and the universal foot-gun. Promote DPoP / mTLS deliberately, do not wait for an incident.
4. **Key rotation as a routine event.** Quarterly, not "in response to a leak". The first rotation should be drilled before it matters.
5. **Audit as a first-class output.** Equal priority with the API surface. Done after the fact, it is never done well.
6. **Defense in depth, not single-layer purity.** Hardened defaults, rate limits, anomaly signals, MFA, sender-constraint — each defeats only some attacks. Combine them.
7. **Symmetry on auth endpoints.** No timing oracles, no error-message oracles. Distinguishing legitimate from illegitimate must require the legitimate credential.
8. **Forward-compatible with passkeys.** The end-state is no passwords. Every design choice about MFA should keep that path open.

Appendices

Appendix A — RFC index

Every standard cited in this book.

Spec	Title
RFC 4226	HOTP — HMAC-Based One-Time Password Algorithm
RFC 4949	Internet Security Glossary, Version 2
RFC 5322 / RFC 6531	Internet Message Format / SMTP Extension for Internationalized Email
RFC 6238	TOTP — Time-Based One-Time Password Algorithm
RFC 6749	OAuth 2.0 Authorization Framework
RFC 6750	OAuth 2.0 Bearer Token Usage
RFC 6797	HTTP Strict Transport Security (HSTS)
RFC 6819	OAuth 2.0 Threat Model and Security Considerations
RFC 6979	Deterministic Usage of DSA and ECDSA
RFC 7009	OAuth 2.0 Token Revocation
RFC 7515	JSON Web Signature (JWS)
RFC 7516	JSON Web Encryption (JWE)
RFC 7517	JSON Web Key (JWK)
RFC 7518	JSON Web Algorithms (JWA)
RFC 7519	JSON Web Token (JWT)
RFC 7520	Examples of Protecting Content using JOSE
RFC 7523	JWT Profile for OAuth 2.0 Client Authentication
RFC 7591	OAuth 2.0 Dynamic Client Registration Protocol
RFC 7592	OAuth 2.0 Dynamic Client Registration Management

Spec	Title
RFC 7636	PKCE — Proof Key for Code Exchange
RFC 7638	JSON Web Key (JWK) Thumbprint
RFC 7662	OAuth 2.0 Token Introspection
RFC 7800	Proof-of-Possession Key Semantics for JWTs (`cnf` claim)
RFC 8032	Edwards-Curve Digital Signature Algorithm (EdDSA)
RFC 8252	OAuth 2.0 for Native Apps
RFC 8471 – RFC 8473	Token Binding (for completeness)
RFC 8615	Well-Known URIs
RFC 8628	OAuth 2.0 Device Authorization Grant
RFC 8693	OAuth 2.0 Token Exchange
RFC 8705	OAuth 2.0 Mutual-TLS Client Authentication
RFC 8707	OAuth 2.0 Resource Indicators
RFC 8725	JSON Web Token Best Current Practices
RFC 9068	JWT Profile for OAuth 2.0 Access Tokens
RFC 9101	JWT-Secured Authorization Request (JAR)
RFC 9106	Argon2 Memory-Hard Function for Password Hashing
RFC 9126	OAuth 2.0 Pushed Authorization Requests (PAR)
RFC 9207	OAuth 2.0 Authorization Server Issuer Identification
RFC 9449	OAuth 2.0 Demonstrating Proof of Possession (DPoP)
RFC 9635	Grant Negotiation and Authorization Protocol (GNAP)
RFC 9636	GNAP for Resource Servers
RFC 9700	OAuth 2.0 Security Best Current Practice
OIDC Core 1.0	OpenID Connect Core
OIDC Discovery 1.0	OpenID Connect Discovery
OIDC RP-Initiated Logout · Front-Channel · Back-Channel	OpenID Connect logout flows

Spec	Title
W3C WebAuthn L3	Web Authentication: An API for accessing Public Key Credentials
W3C VC Data Model 2.0	Verifiable Credentials Data Model
W3C DID 1.0	Decentralized Identifiers
FIDO CTAP 2	Client to Authenticator Protocol
NIST SP 800-57	Recommendation for Key Management
NIST SP 800-61	Computer Security Incident Handling Guide
NIST SP 800-63B	Authentication and Authenticator Lifecycle (Rev 4)
NIST SP 800-63C	Federation and Assertions
NIST SP 800-90A	Random Bit Generators
NIST SP 800-162	Guide to Attribute Based Access Control
NIST SP 800-207	Zero Trust Architecture
ISO/IEC 27001	Information Security Management Systems
FAPI 2.0 Security Profile	Financial-grade API Profile
OASIS XACML 3.0	eXtensible Access Control Markup Language
OpenID CAEP · Shared Signals Framework	Continuous Access Evaluation / Shared Signals
OIDC4VCI · OIDC4VP	OpenID for Verifiable Credentials Issuance / Presentation
W3C CSP Level 3	Content Security Policy
OWASP Cheat Sheets	Authentication · Password Storage · Forgot Password · JWT · OAuth
SPIFFE / SPIRE	Secure Production Identity Framework for Everyone
Open Policy Agent · AWS Cedar	Policy-as-code engines for PBAC
HaveIBeenPwned API	k-anonymity breached-password lookup

Appendix B — Claim & header registry

The claims and headers used throughout the book and the registry that defines them.

Name	Where	Meaning	Registry
`iss`	Payload	Issuer	IANA JWT Claims

Name	Where	Meaning	Registry
`sub`	Payload	Subject	IANA JWT Claims
`aud`	Payload	Audience	IANA JWT Claims
`exp`	Payload	Expiration	IANA JWT Claims
`nbf`	Payload	Not Before	IANA JWT Claims
`iat`	Payload	Issued At	IANA JWT Claims
`jti`	Payload	JWT ID	IANA JWT Claims
`nonce`	Payload	Replay defense	OIDC Core
`acr`	Payload	Auth Context Class Ref	OIDC Core
`amr`	Payload	Auth Methods References	OIDC Core
`azp`	Payload	Authorized Party	OIDC Core
`at_hash`	Payload	Access Token Hash	OIDC Core
`cnf`	Payload	Confirmation (sender-constraint)	RFC 7800
`scope` / `scp`	Payload	OAuth scopes	RFC 9068
`email` / `email_verified` / `name` / `picture`	Payload	OIDC profile / email scopes	OIDC Core
`alg`	Header	Signing/encryption algorithm	RFC 7515 / 7516
`kid`	Header	Key identifier	RFC 7515
`typ`	Header	Type	RFC 7515
`cty`	Header	Content type	RFC 7515
`jwk`	Header	Embedded public key (DPoP)	RFC 7515
`jkt`	`cnf.jkt`	JWK thumbprint	RFC 9449
`htm`	Header (DPoP proof)	HTTP method	RFC 9449

Name	Where	Meaning	Registry
`htu`	Header (DPoP proof)	HTTP URI	RFC 9449

Appendix C — Cipher suite recommendations (2026)

Use	Preferred	Acceptable	Avoid
Token signing	EdDSA (Ed25519)	ES256, PS256	RS256 (acceptable, plan migration), HS*, none
JWE key management	ECDH-ES+A256KW	RSA-OAEP-256	RSA-OAEP, RSA1_5
JWE content encryption	A256GCM	A128GCM, XC20P	AES-CBC + HMAC
TLS	1.3 only	1.2 with AEAD ciphers	1.0, 1.1
Hashing	SHA-256, SHA-3-256	SHA-512	MD5, SHA-1
Password KDF	Argon2id	scrypt, bcrypt cost ≥ 12	PBKDF2 < 600k iters, plain hashes
MAC (when required)	HMAC-SHA-256	HMAC-SHA-512	HMAC-SHA-1

Appendix D — Glossary

(Cross-references the front-matter glossary; full alphabetical list.)

- **AAAF** — Authentication, Authorization, Accounting (Framework).
- **AAGUID** — Authenticator Attestation GUID. Identifies a WebAuthn authenticator model.
- **AAL** — Authenticator Assurance Level (NIST SP 800-63B). AAL1 / AAL2 / AAL3 in increasing strength.
- **ABAC** — Attribute-Based Access Control.
- **ACR** — Authentication Context Class Reference (OIDC).
- **AEAD** — Authenticated Encryption with Associated Data.
- **AMR** — Authentication Methods References (OIDC).
- **AS** — Authorization Server (OAuth).
- **at_hash** — Access Token Hash claim (OIDC).
- **azp** — Authorized Party claim (OIDC).

- **BCP** — Best Current Practice (IETF).
- **BFF** — Backend for Frontend.
- **CAEP** — Continuous Access Evaluation Profile.
- **CEK** — Content Encryption Key (JWE).
- **CIBA** — Client-Initiated Backchannel Authentication.
- **CSPRNG** — Cryptographically Secure Pseudo-Random Number Generator.
- **CTAP** — Client to Authenticator Protocol (FIDO).
- **DCR** — Dynamic Client Registration (RFC 7591).
- **DID** — Decentralized Identifier (W3C).
- **DPoP** — Demonstrating Proof-of-Possession (RFC 9449).
- **FAL** — Federation Assurance Level (NIST SP 800-63C).
- **FAPI** — Financial-grade API.
- **GNAP** — Grant Negotiation and Authorization Protocol.
- **HSM** — Hardware Security Module.
- **IAL** — Identity Assurance Level (NIST SP 800-63A).
- **IdP** — Identity Provider.
- **JAR** — JWT-Secured Authorization Request (RFC 9101).
- **JOSE** — JSON Object Signing and Encryption.
- **JWA** — JSON Web Algorithms (RFC 7518).
- **JWE** — JSON Web Encryption (RFC 7516).
- **JWK** — JSON Web Key (RFC 7517).
- **JWKS** — JSON Web Key Set.
- **JWS** — JSON Web Signature (RFC 7515).
- **JWT** — JSON Web Token (RFC 7519).
- **KDF** — Key Derivation Function.
- **kid** — Key ID (JOSE header).
- **KMS** — Key Management Service.
- **MAC** — Message Authentication Code.
- **MFA** — Multi-Factor Authentication.
- **OIDC** — OpenID Connect.
- **OP** — OpenID Provider.
- **PAP** — Policy Administration Point.
- **PAR** — Pushed Authorization Request (RFC 9126).
- **PBAC** — Policy-Based Access Control.
- **PDP** — Policy Decision Point.
- **PEP** — Policy Enforcement Point.
- **PIP** — Policy Information Point.

- **PKCE** — Proof Key for Code Exchange (RFC 7636).
- **RBAC** — Role-Based Access Control.
- **ReBAC** — Relationship-Based Access Control.
- **RO** — Resource Owner.
- **ROPC** — Resource Owner Password Credentials grant (deprecated).
- **RP** — Relying Party.
- **RS** — Resource Server.
- **SPIFFE** — Secure Production Identity Framework for Everyone.
- **SSF** — Shared Signals Framework.
- **SSI** — Self-Sovereign Identity.
- **SVID** — SPIFFE Verifiable Identity Document.
- **TOTP** — Time-based One-Time Password (RFC 6238).
- **VC** — Verifiable Credential (W3C).
- **WebAuthn** — Web Authentication API (W3C).

End of book. Comments, corrections, and chapter-length improvements welcome via GitHub.